



MÁSTER EN INGENIERÍA INDUSTRIAL

TRABAJO FIN DE MÁSTER

Arquitectura Distribuida para UAVs de Bajo Coste: Integración de Micro-Companion Computer, Conectividad IoT y Entorno de Simulación

AUTOR: Félix García Fernández

DIRECTOR: Emilio Manuel Domínguez Adan

Madrid, Junio de 2026

Autorizada la entrega del proyecto del alumno:

Félix García Fernández

EL DIRECTOR DEL PROYECTO

Emilio Manuel Domínguez Adan

Fdo.: Fecha: / /

Vº Bº DEL COORDINADOR DE PROYECTOS

Alvaro Sánchez Miralles

Fdo.: Fecha: / /

Resumen

Este trabajo propone una arquitectura de bajo coste para dotar a un dron comercial de capacidades autónomas e IoT, sustituyendo el habitual *companion computer* (Raspberry Pi o similar) por un microcontrolador ESP32 de unos pocos euros. El firmware resultante (AFC) se ha diseñado en capas con inversión de dependencias para compilar sin cambios tanto en la placa física como en un entorno de simulación completo (SITL), y se ha validado mediante ensayos en SITL y en hardware real (HIL): protocolo de comunicaciones verificado a nivel de bytes, footprint y consumo de CPU/memoria en la placa física, y un ciclo completo de identificación de planta y diseño de control PID ejecutado y validado de principio a fin.

Palabras clave: UAV de bajo coste, arquitectura distribuida, ESP32, IoT, SITL/HIL, identificación de sistemas, control PID.

1. Introducción

Dotar a un dron de bajo coste de autonomía y conectividad IoT exige normalmente incorporar un *companion computer* de gama alta, lo que dispara el coste, el peso y el consumo energético del vehículo. Este trabajo explora una alternativa: usar un microcontrolador ultra-barato (ESP32) como *micro-companion computer*, apoyándose en un controlador de vuelo básico (Betaflight) para la estabilización de bajo nivel. La seguridad se resuelve por arquitectura — si el microcontrolador falla o se satura, el dron conserva su estabilización, en vez de caer — y la conectividad se resuelve abstrayendo las comunicaciones a TCP/UDP, rompiendo la dependencia del radiocontrol tradicional y su límite de línea de visión directa.

2. Objetivos

El sistema debía cumplir tres objetivos. Primero, un software lo más flexible posible: independencia del sistema operativo (FreeRTOS o Linux) y del transporte de comunicaciones (UART, I2C, TCP/UDP) mediante inversión de dependencias. Segundo, control del dron sin mando manual: una Estación de Control en Tierra (GCS) que sustituya al mando RC y que sea indiferente a si la red subyacente es un socket local, una red doméstica o una red móvil global. Tercero, un entorno de desarrollo ágil: simulación completa (SITL) para iterar sin volar, verificación en hardware real (HIL) antes de arriesgar la placa, y sintonización de los parámetros de control *on-the-fly*, sin recompilar ni reflashear el firmware.

3. Arquitectura del sistema

El firmware AFC se organiza en cinco capas con inversión de dependencias (Figura 1): Platform (ESP32/FreeRTOS o Linux), Core (interfaces abstractas), Channel (protocolos MSP/IBus/I2C sobre un *blackboard* fuertemente tipado), AppLogic (control PID en cascada) y FlightApplication (ensamblaje). `TransportFactory` y `OSALFactory` son las únicas clases sensibles a la plataforma: el resto del código es idéntico en ambos entornos, y la selección ocurre una sola vez, en tiempo de compilación.

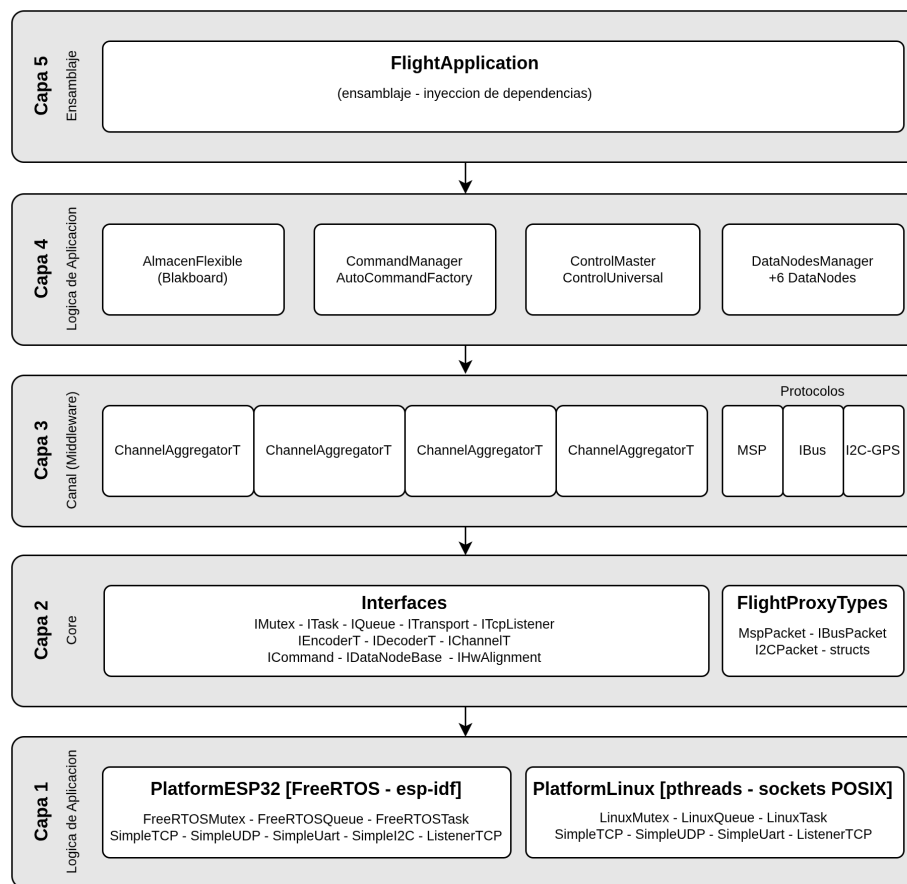


Figura 1. Arquitectura en capas del firmware AFC, con inversión de dependencias e implementación intercambiable de la Capa 1 (ESP32/FreeRTOS o Linux).

Sobre esta base se ha construido un entorno SITL (Betaflight 4.5.1 + Gazebo Garden + emuladores de sensores externos) que reproduce la dinámica de vuelo sin necesidad de la placa física, y una GCS en Python que sustituye al mando RC y añade telemetría, identificación de planta y diseño de control asistido por SIMC.

4. Ensayos y resultados

El protocolo de comunicaciones GCS–AFC (MSP sobre TCP, IBus sobre UDP) se ha verificado a nivel de bytes con capturas Wireshark (CRC8-DVB-S2 e IBus *checksum*). En hardware real (placa AM-036), el *footprint* estático ocupa un 27 % de la partición de aplicación (73 % libre) y el consumo de CPU se mantiene en un solo dígito por tarea durante el vuelo. El ensayo más completo ejecuta de principio a fin el lazo de altura: localización del punto de *hovering*, escalón de *throttle*, identificación FOPDT (modelo de primer orden más tiempo muerto) con $K = 7,588$ (ganancia), $\tau = 1,804$ s (constante de tiempo), $\theta = 0,470$ s (retardo) y $R^2 = 0,978$ (coeficiente de determinación del ajuste); sintonización SIMC (método de Skogestad) con $K_p = 0,1007$ (ganancia proporcional) y $K_i = 0,0558$ (ganancia integral); y validación con un *doblet* ± 1 m/s (Figura 2), sin *overshoot* apreciable.

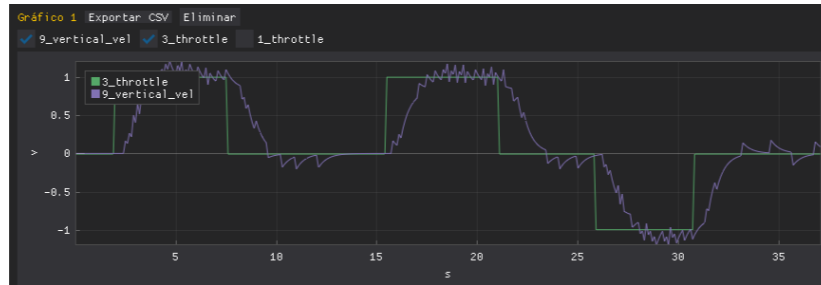


Figura 2. Ensayo de validación (*doblet* ± 1 m/s) del lazo de velocidad vertical con el PID sintonizado por SIMC ya cargado en el AFC.

Como demostración adicional de la flexibilidad de la arquitectura, el sistema se extiende con navegación autónoma por ruta de varios waypoints, con una transición suave entre ellos generada en la GCS, validada en un circuito hexagonal de 6 waypoints con 10 repeticiones (error de seguimiento $xte_{rms} = 0,60 \pm 0,22$ m).

5. Conclusiones

El resultado principal de este trabajo es que la plataforma funciona de punta a punta: un microcontrolador de bajo coste asume el papel de *companion computer* sin renunciar a telemetría, identificación de planta ni control remoto, con margen de recursos de sobra para crecer. La arquitectura en capas ha permitido desarrollar y depurar la mayor parte del sistema en simulación, reservando el hardware físico para la validación final, y el protocolo de comunicaciones diseñado para este trabajo se ha puesto a prueba, con éxito, en dos entornos de red distintos. Quedan como trabajo futuro la robustez de red ante fallos de conexión, la caracterización de latencia entre SITL y HIL, y la migración del enlace celular a LTE.

6. Referencias

- [1] Betaflight Contributors, “Betaflight — Open Source Flight Controller Firmware,” 2025. <https://github.com/betaflight/betaflight>

- [2] Skogestad, S. “Simple Analytic Rules for Model Reduction and PID Controller Tuning,” *Journal of Process Control*, vol. 13, no. 4, pp. 291–309, 2003.

Abstract

This work proposes a low-cost architecture to add autonomous and IoT capabilities to a commercial drone, replacing the usual *companion computer* (Raspberry Pi or similar) with a microcontroller (ESP32) worth a few euros. The resulting firmware (AFC) is designed in layers with dependency inversion so that it compiles unchanged both on the physical board and in a full simulation environment (SITL), and it has been validated through SITL and real-hardware (HIL) tests: byte-level verification of the communications protocol, static footprint and CPU/memory usage on the physical board, and a complete plant identification and PID control design cycle executed and validated end to end.

Keywords: low-cost UAV, distributed architecture, ESP32, IoT, SITL/HIL, system identification, PID control.

1. Introduction

Adding autonomy and IoT connectivity to a low-cost drone usually requires a high-end *companion computer*, which drives up the vehicle's cost, weight and power consumption. This work explores an alternative: using an ultra-cheap microcontroller (ESP32) as a *micro-companion computer*, relying on a basic flight controller (Betaflight) for low-level stabilization. Safety is addressed through architecture — if the microcontroller fails or becomes saturated, the drone keeps its stabilization instead of falling — and connectivity is addressed by abstracting communications to TCP/UDP, breaking the traditional radio control's dependency on direct line of sight.

2. Objectives

The system had to meet three objectives. First, software as flexible as possible: independence from the operating system (FreeRTOS or Linux) and from the communications transport (UART, I2C, TCP/UDP) through dependency inversion. Second, control of the drone without a manual radio: a Ground Control Station (GCS) that replaces the RC controller and is indifferent to whether the underlying network is a local socket, a home network or a global mobile network. Third, an agile development environment: full simulation (SITL) to iterate without flying, real-hardware verification (HIL) before risking the board, and *on-the-fly* control tuning, without recompiling or reflashing the firmware.

3. System architecture

The AFC firmware is organized into five layers with dependency inversion (Figure 1): Platform (ESP32/FreeRTOS or Linux), Core (abstract interfaces), Channel (MSP/IBus/I2C protocols over a strongly-typed *blackboard*), AppLogic (cascaded PID control) and FlightApplication (assembly). `TransportFactory` and `OSALFactory` are the only platform-sensitive classes: the rest of the code is identical on both targets, and the implementation is selected once, at compile time.

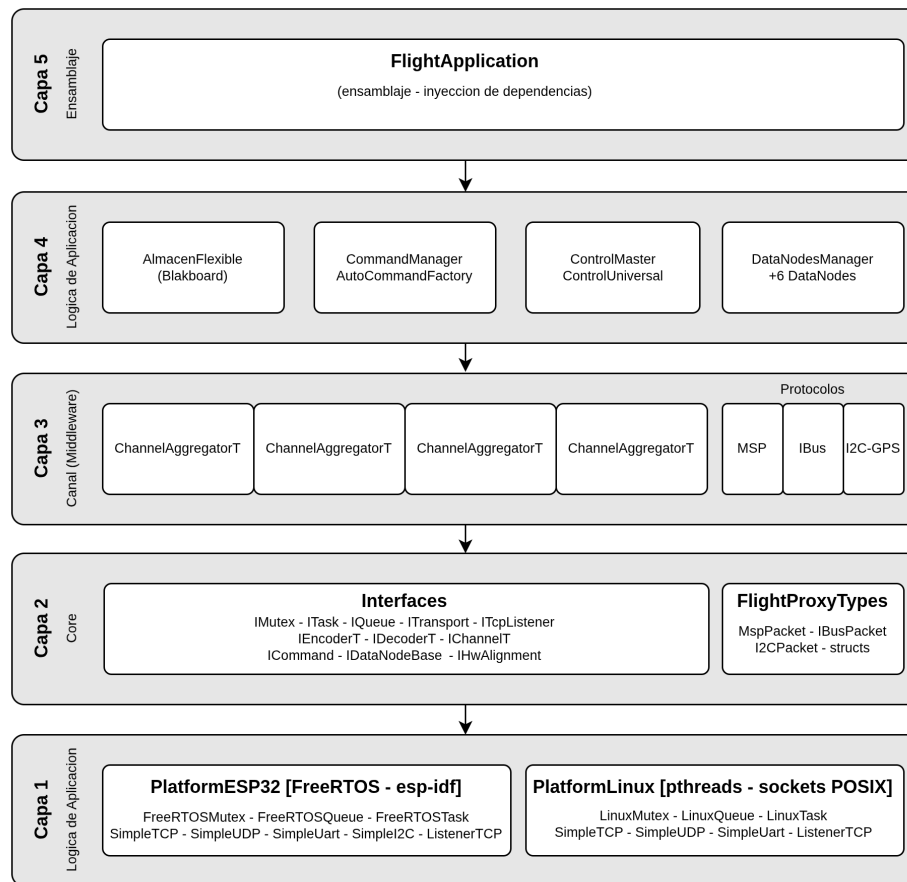


Figure 1. Layered architecture of the AFC firmware, with dependency inversion and an interchangeable Layer 1 implementation (ESP32/FreeRTOS or Linux).

On this basis, a SITL environment (Betaflight 4.5.1 + Gazebo Garden + external sensor emulators) has been built that reproduces flight dynamics without the physical board, together with a Python GCS that replaces the RC controller and adds telemetry, plant identification and SIMC-assisted control design.

4. Tests and results

The GCS–AFC communications protocol (MSP over TCP, IBus over UDP) has been verified at byte level with Wireshark captures (CRC8-DVB-S2 and IBus *checksum*). On real hardware (AM-036 board), the static *footprint* occupies 27 % of the application partition (73 % free) and CPU usage stays in the single digits per task during flight. The most complete test runs the altitude loop end to end: locating the *hovering* point, a *throttle* step, FOPDT identification (first-order plus dead-time model) with $K = 7.588$ (gain), $\tau = 1.804$ s (time constant), $\theta = 0.470$ s (dead time) and $R^2 = 0.978$ (coefficient of determination of the fit); SIMC tuning (Skogestad’s method) with $K_p = 0.1007$ (proportional gain) and $K_i = 0.0558$ (integral gain); and validation with a ± 1 m/s *doublet* (Figure 2), with no appreciable overshoot.

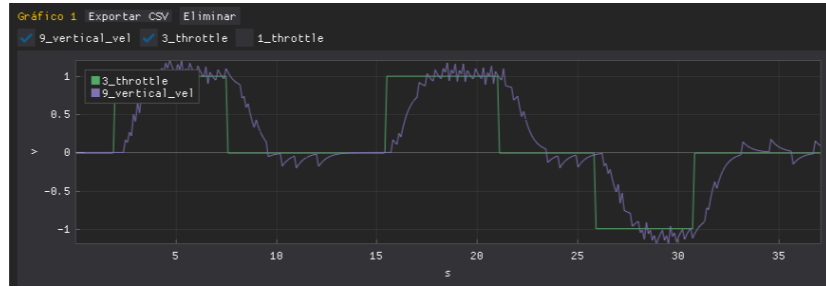


Figure 2. Validation test (± 1 m/s *doublet*) of the vertical velocity loop with the SIMC-tuned PID already loaded on the AFC.

As a further demonstration of the architecture’s flexibility, the system is extended with autonomous multi-waypoint route navigation, with a smooth transition between waypoints generated on the GCS, validated on a hexagonal 6-waypoint circuit over 10 repetitions (tracking error $xe_{rms} = 0.60 \pm 0.22$ m).

5. Conclusions

The main result of this work is that the platform works end to end: a low-cost microcontroller takes on the role of *companion computer* without giving up telemetry, plant identification or remote control, with plenty of headroom left to grow. The layered architecture has made it possible to develop and debug most of the system in simulation, reserving the physical hardware for final validation, and the communications protocol designed for this work has been successfully tested on two different network environments. Network robustness against connection failures, latency characterization between SITL and HIL, and migration of the cellular link to LTE remain as future work.

6. References

- [1] Betaflight Contributors, “Betaflight — Open Source Flight Controller Firmware,” 2025. <https://github.com/betaflight/betaflight>

- [2] Skogestad, S. “Simple Analytic Rules for Model Reduction and PID Controller Tuning,” *Journal of Process Control*, vol. 13, no. 4, pp. 291–309, 2003.

Índice general

1. Introducción	11
1.1. Glosario del sistema	11
1.1.1. Hardware	11
1.1.2. Entorno y simulación	12
2. Estado del Arte	13
2.1. UAVs de bajo coste: contexto y evolución	13
2.1.1. El ecosistema de UAVs <i>open-source</i> y de bajo coste	13
2.1.2. Controladores de vuelo <i>open-source</i> : Betaflight frente a las alternativas	14
2.2. Arquitecturas de control distribuidas para UAVs	15
2.2.1. Sistemas monolíticos vs. arquitecturas distribuidas	15
2.2.2. <i>Companion computers</i> : estado del arte	15
2.2.3. El <i>Micro-Companion Computer</i> : ESP32 como alternativa de bajo coste	16
2.3. Conectividad IoT y operaciones BVLOS	16
2.3.1. El dron como dispositivo de red: <i>Internet of Drones</i>	16
2.3.2. Conectividad celular para UAVs: LTE/4G como habilitador	17
2.4. Protocolos de comunicación en sistemas UAV	17
2.4.1. MSP (<i>MultiWii Serial Protocol</i>)	17
2.4.2. IBus como protocolo de referencia de actitud	18
2.5. Arquitectura software: capas, abstracciones y patrones	18
2.5.1. Arquitectura en capas y HAL/OSAL en sistemas embebidos	18
2.5.2. Principio de Inversión de Dependencias (DIP)	19
2.5.3. El patrón <i>Blackboard</i> en sistemas de control	19
2.6. Entornos de simulación para UAVs	20
2.6.1. Software-in-the-Loop (SITL)	20
2.6.2. Hardware-in-the-Loop (HIL)	20
2.6.3. Gazebo como motor de simulación física	21
2.7. Síntesis: posicionamiento del trabajo	21
3. Motivación y Objetivos	22
3.1. Motivación	22
3.1.1. Seguridad mediante arquitectura distribuida	22
3.1.2. El dron como dispositivo IoT: romper la barrera de la distancia	22
3.2. Objetivos del proyecto	23

3.2.1. Software lo más flexible posible	23
3.2.2. Control del dron sin mando manual	23
3.2.3. Entorno de desarrollo ágil	23
4. Recursos y Metodología	24
4.1. Recursos hardware	24
4.1.1. Plataforma base	24
4.1.2. Modificaciones — sistema UAV completo	24
4.1.3. Equipo de desarrollo	24
4.2. Recursos software	25
4.2.1. Firmware y sistemas embebidos	25
4.2.2. Entorno de simulación	25
4.2.3. Estación de control en tierra (GCS)	25
4.2.4. Herramientas de desarrollo	25
4.3. Metodología de trabajo	25
4.3.1. Sintonización de control <i>on-the-fly</i>	26
5. Plan de Trabajo y Cronograma	27
5.1. Fases del proyecto	27
5.2. Cronograma	28
6. Arquitectura del Firmware AFC	29
6.1. Visión general de la arquitectura	29
6.1.1. Diagrama de capas	30
6.1.2. Principios de diseño aplicados	30
6.2. Capa 1 — Plataforma e Infraestructura	31
6.2.1. Implementaciones de transporte	32
6.2.1.1. Transportes de datos básicos	32
6.2.1.2. TcpListener — transporte generador	33
6.2.2. Implementaciones del OSAL	33
6.2.2.1. FreeRTOS (hardware ESP32)	33
6.2.2.2. Linux (entorno de simulación)	33
6.2.3. Compilación condicional de plataforma	34
6.3. Capa 2 — Core e Interfaces	34
6.3.1. Interfaces de plataforma	34
6.3.1.1. ITransport	34
6.3.1.2. Interfaces OSAL: IMutex, IQueue, ITask	35
6.3.2. Interfaces de protocolo	35
6.3.2.1. IEncoder	35
6.3.2.2. IDecoder	35
6.3.3. Factorías: TransportFactory y OSALFactory	35
6.3.4. Tipos de datos globales	35
6.4. Capa 3 — Middleware de Comunicaciones	36
6.4.1. Implementaciones de protocolo	36
6.4.1.1. MSP — MultiWii Serial Protocol	36

6.4.1.2.	IBus	37
6.4.2.	El concepto de Channel	37
6.4.2.1.	Composición: ITransport + IEncoder + IDecoder	37
6.4.2.2.	Flujo de datos interno	37
6.4.3.	Enrutamiento avanzado de canales	38
6.4.3.1.	ChannelServer	38
6.4.3.2.	ChannelAggregator	38
6.4.3.3.	ChannelDisregator	39
6.4.3.4.	PersistentChannel	39
6.5.	Capa 4 — Lógica de Aplicación	39
6.5.1.	El patrón Blackboard: AlmacenFlexible	40
6.5.1.1.	Índice centralizado de datos	40
6.5.1.2.	Metadatos temporales	41
6.5.2.	Gestor de Comandos (CommandManager)	42
6.5.2.1.	Anatomía de un ICommand	42
6.5.2.2.	Procesamiento de paquetes MSP	42
6.5.3.	Gestor de Vuelo (ControlManager)	43
6.5.3.1.	Ciclo de trabajo	43
6.5.3.2.	Leyes de control implementadas	43
6.5.3.3.	Integración con el Blackboard	46
6.5.3.4.	Navegación por ruta	47
6.5.4.	Gestor de Nodos de Datos (DataNodesManager)	47
6.5.4.1.	Anatomía de un DataNode	47
6.5.4.2.	Nodos implementados	47
6.5.4.3.	Orquestación y temporización	48
6.6.	Capa 5 — Ensamblaje	48
6.6.1.	FlightApplication	48
6.6.2.	Inyección de dependencias y arranque	49
6.7.	Flujos de datos principales	50
6.7.1.	Entrada y ejecución de comandos desde la GCS	50
6.7.2.	Ingesta continua de referencias (IBus/UDP)	51
6.7.3.	Procesamiento interno del control	51
6.7.4.	Envío de referencias al BFC	51
6.8.	Portabilidad y entorno de simulación	51
7.	Entorno de Simulación	53
7.1.	Motivación y objetivos del entorno de simulación	54
7.2.	Visión de conjunto de la arquitectura de simulación	55
7.3.	Componentes del entorno de simulación	57
7.3.1.	Motor de física — Gazebo Garden	57
7.3.2.	Simulación del controlador de vuelo — Betaflight SITL	59
7.3.3.	Emulación de sensores externos — ESS (<i>gzDataExtractor</i>)	60
7.4.	Integración del firmware AFC en el entorno de simulación	61
7.5.	Orquestación con Docker Compose	61

7.5.1. Servicios, red y dependencias	62
7.5.2. Scripts auxiliares	62
7.6. Modo HIL — Hardware in the Loop	62
7.7. Flujo de datos <i>end-to-end</i> en simulación	63
8. Sistema de Control en Tierra (GCS)	65
8.1. Introducción y objetivos	65
8.2. Arquitectura general	66
8.3. Capa de comunicaciones: protocolo MSP V2	67
8.3.1. Estructura del paquete MSP V2	67
8.3.2. Encoder, decoder y CRC8-DVB-S2	67
8.3.3. Gestión de la conexión TCP (Connection)	68
8.4. Mando virtual (Remote)	69
8.4.1. Bucle de envío y estados arm/disarm	69
8.4.2. Tests automatizados: step test y staircase test	69
8.4.3. Interfaz de usuario (RemoteTab)	69
8.5. Telemetría en tiempo real	70
8.5.1. Suscripción a variables y buffer circular	70
8.5.2. Ciclo de muestreo (TelemetryDataGatherer)	70
8.5.3. Interfaz: gráficos configurables y exportación CSV	71
8.6. Configuración del controlador (Settings)	71
8.6.1. Nivel de control activo	71
8.6.2. Parámetros PID por lazo	72
8.7. Identificación de planta	73
8.7.1. Modelos soportados y señal de excitación	73
8.7.2. Ajuste por mínimos cuadrados no lineales	74
8.7.3. Flujo de trabajo en la interfaz (IdenTab)	74
8.8. Diseño del controlador PID	75
8.8.1. Reglas SIMC (Skogestad)	75
8.8.2. Simulación discreta: saturación y tiempo muerto	76
8.8.3. Flujo completo: de la identificación a la carga de parámetros	76
8.9. Bus interno de eventos (SignalBus)	77
8.10. Flujo de trabajo integrado	78
8.11. API para automatización	79
9. Ensayos	80
9.1. Ensayos en SITL	80
9.1.1. Verificación de los flujos de comunicación	80
9.1.1.1. MSP (TCP, puerto 12345)	80
9.1.1.2. IBus (UDP, puerto 12346)	81
9.2. Ensayos HIL	82
9.2.1. Uso de CPU en el ESP32	82
9.2.2. Uso de memoria (heap y stack)	82
9.2.2.1. Footprint estático	82

9.2.2.2. Uso en <i>runtime</i>	83
9.3. Ciclo completo de identificación y diseño de control	85
9.3.1. Localización del punto de <i>hovering</i>	85
9.3.2. Ensayo de escalón	86
9.3.3. Identificación de la planta	87
9.3.4. Diseño del controlador y validación	88
9.3.5. Sintonización del resto de lazos	89
9.4. Ensayo de navegación por ruta	90
10. Conclusiones y Trabajo Futuro	94
10.1. Cumplimiento de los objetivos	94
10.1.1. Software lo más flexible posible	94
10.1.2. Control del dron sin mando manual	94
10.1.3. Entorno de desarrollo ágil	95
10.2. Conclusiones	95
10.3. Trabajo futuro	95
11. Alineación con los Objetivos de Desarrollo Sostenible	96
11.1. ODS 9 — Industria, innovación e infraestructura	96
11.2. ODS 4 — Educación de calidad	96
11.3. ODS 11 — Ciudades y comunidades sostenibles	97
11.4. ODS 13 — Acción por el clima	97
Bibliografía	98

Índice de figuras

3. Vista general de capas del firmware AFC	30
4. Principio de inversión de dependencias en la arquitectura AFC	31
5. Flujo de datos interno en un ChannelT<T>: ciclos de recepción y emisión	38
6. Enrutamiento de canales: ChannelServer y ChannelAggregator (rama GCS) y ChannelDisregator (rama BFC)	39
7. Flujo completo de un comando MSP desde la GCS hasta la respuesta	43
8. Máquina de estados de ControlStateMachin	45
9. DataNodes activos con sus canales y su relación con el blackboard	48
10. Diagrama de wiring de FlightApplication: canales, blackboard, managers y conexiones de callbacks	50
11. Compilación dual ESP32/Linux: capas compartidas y capa Platform intercambiable	52
12. Arquitectura del entorno de simulación completo. Los cuatro bloques internos (AFC en Linux, SITL, Gazebo y ESS) se ejecutan en el mismo host; los dos servicios contenerizados (SITL y gateway) comparten <i>namespace</i> de red para que 127.0.0.1 sea común. La GCS se conecta desde el exterior por los puertos publicados por el contenedor <i>gateway</i>	54
13. Estructura de trama del protocolo I2C virtual sobre TCP implementado en I2CSim.py. La cabecera de 4 bytes es suficiente para identificar el dispositivo, el registro y el tipo de operación. El servidor responde con la misma cabecera más el <i>payload</i> de datos del sensor.	60
14. Arquitectura interna de la GCS. Las flechas continuas representan comunicación de red con el AFC; las flechas discontinuas representan solicitudes internas via SignalBus entre pestañas.	66
15. Estructura de trama MSP V2. El <i>checksum</i> cubre desde <i>flag</i> hasta el final del <i>payload</i> , excluyendo los dos bytes de cabecera.	67
16. Pestaña <i>Remote</i> de la GCS, con los cuatro <i>sliders</i> de mando virtual y los paneles de configuración del <i>step test</i> y el <i>staircase test</i>	70
17. Pestaña de <i>Telemetría</i> de la GCS, con el menú de variables agrupadas por comando MSP a la izquierda y la zona de gráficos dinámica a la derecha.	71
18. Pestaña <i>Settings</i> de la GCS, con el selector de nivel de control activo y los parámetros PID de cada lazo.	73
19. Pestaña de <i>Identificación</i> de la GCS, con los datos cargados, la región de análisis delimitada y el resultado del ajuste FOPDT superpuesto.	75

20. Pestaña <i>Control Design</i> de la GCS, con las gráficas de simulación de la variable de proceso y la señal de control para los parámetros SIMC sintonizados.	77
21. Flujo de trabajo integrado de la GCS: desde el ensayo de escalón hasta la validación del lazo de control sintonizado. La flecha discontinua indica la posibilidad de iterar ajustando <i>factor_agresividad</i> sin repetir el ensayo.	79
22. Captura Wireshark de la petición MSP (GCS → AFC), puerto TCP 12345. Data (9 bytes): 24 58 3e 00 2e 01 00 00 16.	80
23. Captura Wireshark de la respuesta MSP (AFC → GCS) al mismo intercambio. Data (37 bytes), con <i>cmd</i> = 302 igual que en la petición.	81
24. Captura Wireshark de una trama IBus (GCS → AFC), puerto UDP 12346. Data (32 bytes).	81
25. Instrumentación HIL en <i>runtime</i> (AM-036, 43 s): heap libre (arriba), uso de pila por tarea (centro) y uso de CPU por tarea (abajo, ya presentado en §9.2.1).	84
26. Staircase test sobre el canal de <i>throttle</i> . El escalonado de <i>l_throttle/3_throttle</i> (paneles inferior y superior) permite localizar <i>u_offset</i> a partir de la respuesta de <i>9_vertical_vel</i> (panel central).	86
27. Ensayo de escalón: <i>throttle</i> (verde) y <i>9_vertical_vel</i> (violeta).	87
28. Pestaña <i>Identificación</i> con el ajuste FOPDT superpuesto a los datos medidos de <i>throttle</i> → velocidad vertical.	88
29. Pestaña <i>Control Design</i> : simulación de la variable de proceso (arriba) y de la señal de control (abajo) antes de cargar los parámetros al AFC.	89
30. Ensayo de validación (<i>doblet</i> ±1 m/s) con el PID sintonizado ya cargado en el AFC.	89
31. Las 10 trayectorias reales superpuestas a la referencia, sobre el circuito hexagonal de 6 waypoints.	91
32. Métricas de seguimiento del ensayo, media ± desviación sobre las 10 corridas.	92
33. Trayectoria media de las 8 corridas dentro de rango, con las corridas individuales de fondo.	93

Índice de tablas

1. Comparativa de las principales plataformas de autopiloto <i>open-source</i> [1, 2]	14
2. Comparativa del TFM frente a trabajos relacionados	21
3. Cronograma estimado del proyecto	28
4. Correspondencia interfaz–implementación para OSAL y Transport en las dos plataformas	34
5. Catálogo de DataIDs del blackboard (AlmacenFlexible) con tipo, productor y consumidor	42
6. Bloques del entorno de simulación, su rol y los protocolos de comunicación.	56
7. <i>Plugins</i> del sistema Gazebo Garden habilitados en el mundo de simulación.	58
8. Tópicos de Gazebo Garden consumidos por la ESS.	59
9. Pestañas de la GCS, clase principal, canal hacia el AFC y función.	66
10. Campos de la trama MSP V2.	67
11. Lazos de control configurables desde la pestaña <i>Settings</i>	72
12. Modelos de planta soportados por la GCS para la identificación paramétrica.	74
13. Servicios registrados en el <i>SignalBus</i> de la GCS.	78
14. Bytes capturados del intercambio MSP GET (<i>id</i> = 2) y verificación de CRC8-DVB-S2.	81
15. Uso de CPU por tarea durante 43 s de vuelo HIL (AM-036).	82
16. Uso de memoria estático del binario <i>TransponderESP32</i>	83
17. Distribución del <i>footprint</i> por origen del código.	83
18. Parámetros identificados del modelo FOPDT <i>throttle</i> → velocidad vertical.	87
19. Parámetros PID sintonizados por SIMC para el lazo de velocidad vertical.	88
20. Sintonización de todos los lazos de control del sistema.	90
21. Resultados del ensayo de navegación por ruta (<i>n</i> = 10).	90

Índice de extractos de código

1. Macro REGISTER_FP_STRUCT y ejemplo de tipo generado (IMUData) 36
2. Registro de productores y consumidores en AlmacenFlexible desde ControlMaster 40
3. Miembros privados de FlightApplication — topología de canales del sistema 49

Acrónimos

<i>AFC</i>	<i>Advanced Flight Controller</i> — Controladora avanzada (ESP32)
<i>BFC</i>	<i>Basic Flight Controller</i> — Controladora de vuelo base (Betaflight)
<i>BVLOS</i>	<i>Beyond Visual Line of Sight</i> — Vuelo más allá del alcance visual
<i>ES</i>	<i>External Sensors</i> — Sensores externos
<i>ESC</i>	<i>Electronic Speed Controller</i> — Controlador electrónico de velocidad
<i>ESS</i>	<i>External Sensors Simulation</i> — Emulación de sensores externos
<i>FOPDT</i>	<i>First Order Plus Dead Time</i> — Modelo de primer orden con retardo
<i>GCS</i>	<i>Ground Control Station</i> — Estación de control en tierra
<i>HIL</i>	<i>Hardware-in-the-Loop</i>
<i>ICAI</i>	Instituto Católico de Artes e Industrias
<i>IBus</i>	<i>Intelligent Bus</i> — Protocolo de referencia de actitud
<i>IoT</i>	<i>Internet of Things</i> — Internet de las Cosas
<i>IPDT</i>	<i>Integrator Plus Dead Time</i> — Modelo integrador con retardo
<i>MCU</i>	<i>Microcontroller Unit</i> — Microcontrolador
<i>MSP</i>	<i>MultiWii Serial Protocol</i>
<i>OSAL</i>	<i>Operating System Abstraction Layer</i>
<i>SIMC</i>	<i>Simplified IMC</i> — Reglas de sintonización de Skogestad
<i>SITL</i>	<i>Software-in-the-Loop</i>
<i>SOPDT</i>	<i>Second Order Plus Dead Time</i> — Modelo de segundo orden con retardo
<i>TFM</i>	Trabajo Fin de Máster
<i>UAV</i>	<i>Unmanned Aerial Vehicle</i> — Vehículo aéreo no tripulado

Capítulo 1

Introducción

LOS sistemas de aeronaves no tripuladas (UAV) han experimentado una notable democratización gracias a la proliferación de hardware de bajo coste y ecosistemas de software abierto. Sin embargo, dotar a estos sistemas de capacidades autónomas, conectividad global y un entorno de desarrollo ágil sigue siendo un reto cuando el presupuesto y el peso son restricciones críticas.

Este Trabajo Fin de Máster presenta una arquitectura distribuida para UAVs de bajo coste basada en tres pilares: un *micro-companion computer* construido sobre el microcontrolador ESP32, conectividad IoT mediante red móvil (SIM800L) y un entorno completo de simulación que combina SITL (*Software-in-the-Loop*), Gazebo y emulación de sensores externos.

La memoria se organiza de la siguiente forma: el Capítulo 2 revisa el estado del arte en UAVs de bajo coste, arquitecturas distribuidas, conectividad IoT y patrones software, y el Capítulo 3 justifica las decisiones de diseño y fija los objetivos del proyecto. El Capítulo 4 describe los recursos hardware y software empleados junto con la metodología seguida, y el Capítulo 5 detalla el plan de trabajo y el cronograma. A continuación, los capítulos técnicos desarrollan la propuesta: el Capítulo 6 presenta la arquitectura del firmware del AFC, el Capítulo 7 describe el entorno de simulación SITL/Gazebo empleado para su validación, y el Capítulo 8 detalla la Ground Control Station desarrollada para operar y configurar el sistema. El Capítulo 9 recoge los ensayos realizados en SITL y HIL. Cierran la memoria las conclusiones y el trabajo futuro, y finalmente el Capítulo 11 alinea la propuesta con los Objetivos de Desarrollo Sostenible.

1.1. Glosario del sistema

A continuación se definen los términos principales del sistema, empleados de forma consistente a lo largo del documento.

1.1.1. Hardware

UAV — Sistema completo Conjunto total que resulta de combinar la plataforma base con todas las modificaciones añadidas.

Base drone — Plataforma base Dron original encargado del vuelo a bajo nivel y la estabilización. Compuesto por la BFC y el *Powertrain*.

BFC — *Basic Flight Controller* Controladora de vuelo base (*MAMBA F405MINIMK2*) ejecutando Betaflight. Única responsable de la estabilización del dron.

Powertrain — Hardware de vuelo Elementos físicos de actuación: ESCs, motores brushless, hélices y *frame*.

AFC — *Advanced Flight Controller* Controladora avanzada, módulo AM-036 (ESP32-WROVER-B + SIM800L). Actúa como *micro-companion computer* del sistema.

ES — Sensores externos Periféricos que alimentan al AFC: módulo GPS + Magnetómetro WS-M181 (UART + I2C) y barómetro de precisión BMP390 (I2C).

1.1.2. Entorno y simulación

GCS — *Ground Control Station* Estación de control en tierra. Interfaz desde la que se envían referencias y configuraciones, y se recibe telemetría.

SITL — *Software-in-the-Loop* Instancia de Betaflight compilada y ejecutada como proceso de PC, simulando el comportamiento de la BFC.

Gazebo Motor de simulación física que calcula la dinámica de vuelo del UAV. Recibe los comandos de motor del SITL y responde con datos de IMU y estado del vehículo.

ESS — *External Sensors Simulation* Módulo que lee los datos de posición, orientación y altitud de Gazebo y los convierte en señales I2C/UART simuladas sobre sockets TCP, de modo que el AFC los interpreta igual que si fueran los sensores físicos reales.

HIL — *Hardware-in-the-Loop* Modo de ensayo en el que la placa AFC física real se conecta al entorno de simulación en PC, permitiendo verificar los recursos del MCU bajo carga real.

Capítulo 2

Estado del Arte

2.1. UAVs de bajo coste: contexto y evolución

2.1.1. El ecosistema de UAVs *open-source* y de bajo coste

EN la última década, el mercado de vehículos aéreos no tripulados ha experimentado un crecimiento sin precedentes, impulsado por la reducción de costes del hardware de consumo y la maduración de ecosistemas de software libre. Esta convergencia ha dado lugar a plataformas de investigación que, hace apenas una década, habrían requerido presupuestos propios de la industria aeronáutica profesional.

Sørensen et al. [3] demostraron que es posible construir un sistema UAV de despliegue de sensores por debajo de 800 \$, con un diseño modular y extensible basado íntegramente en componentes de código abierto. Su trabajo ilustra cómo la elección de hardware de bajo coste no implica una merma en las capacidades funcionales del sistema, siempre que el diseño arquitectónico sea el adecuado.

Sin embargo, la proliferación de plataformas abiertas ha generado un problema de fragmentación. El survey de Ebeid et al. [1] ofrece una tabla comparativa de las principales plataformas de autopiloto open-source —Betaflight, INAV, ArduPilot y PX4— y concluye que la falta de estandarización dificulta el desarrollo cruzado entre plataformas. Cada ecosistema impone sus propias abstracciones, protocolos y herramientas, lo que eleva la curva de aprendizaje y limita la reutilización de código entre proyectos. Muñoz-Morera et al. [4] señalan que esta escasez de plataformas asequibles y accesibles es una de las principales barreras que limita la investigación en autonomía de UAVs. La Tabla 1 resume las características principales de cada plataforma.

Tabla 1. Comparativa de las principales plataformas de autopiloto *open-source* [1, 2]

Plataforma	Uso principal	Protocolo	SITL/HIL	Nav. autón.	Hardware
Betaflight	FPV / <i>racing</i>	MSP	No	No	Bajo
INAV	FPV + <i>waypoints</i>	MSP v2	Parcial	Básica	Bajo
ArduPilot	Misión completa	MAVLink	Completo	Completa	Medio–Alto
PX4	Investigación / misión	MAVLink	Completo	Completa	Medio–Alto

2.1.2. Controladores de vuelo *open-source*: Betaflight frente a las alternativas

Los cuatro grandes ecosistemas de autopiloto *open-source* presentan perfiles muy distintos en cuanto a funcionalidad, complejidad y recursos de hardware requeridos. Ebeid et al. [2] proporcionan una tabla comparativa detallada de controladores de vuelo y simuladores. ArduPilot y PX4, los más completos funcionalmente, incorporan soporte nativo de SITL, integración con Gazebo y un protocolo de comunicación rico como MAVLink. Sin embargo, esta riqueza tiene un precio: ambas plataformas exigen hardware de mayor potencia y están diseñadas para aeronaves de mayor envergadura, lo que las hace subóptimas para plataformas FPV (*First Person View*) de bajo coste.

Betaflight [5] está diseñado y optimizado para el vuelo manual de altas prestaciones en modalidades FPV. Su scheduler determinista es capaz de ejecutar el lazo de control a frecuencias de hasta 8 kHz, lo que lo hace insuperable en términos de respuesta en plataformas pequeñas [1]. Sin embargo, esta especialización viene acompañada de limitaciones significativas: carece de herramientas SITL/HIL robustas [2] y no incorpora soporte nativo para companion computers o navegación autónoma.

Beaudoin et al. [6] analizan las limitaciones de ArduPilot/PX4 desde la perspectiva del coste, el peso y el consumo energético, concluyendo que estas plataformas resultan inadecuadas para drones de pequeño formato. Este análisis refuerza la elección de Betaflight como BFC en el presente trabajo: se aprovecha su rendimiento en el lazo rápido de control y se compensan sus limitaciones de autonomía mediante un sistema complementario externo.

En resumen, el ecosistema UAV *open-source* ofrece hoy plataformas asequibles y maduras, pero ninguna resuelve simultáneamente las dos necesidades del presente trabajo: un lazo de control de alta frecuencia y bajo coste, y la posibilidad de incorporar autonomía y conectividad mediante un companion computer. ArduPilot y PX4 cubren la segunda necesidad a costa de la primera, mientras que Betaflight resuelve la primera pero carece de la segunda. Esta brecha es la que motiva la arquitectura distribuida propuesta en este TFM, que se desarrolla en la siguiente sección.

2.2. Arquitecturas de control distribuidas para UAVs

2.2.1. Sistemas monolíticos vs. arquitecturas distribuidas

La concentración de toda la lógica de vuelo en un único procesador —estabilización de motores, lectura de sensores de posición, navegación autónoma y transmisión de telemetría— conlleva un riesgo inherente en sistemas de tiempo real. Panunzio y Vardanega [7] señalan que *“una gran proporción de los requisitos de los sistemas embebidos de tiempo real proviene de las dimensiones extra-funcionales del tiempo y el determinismo espacial, la fiabilidad, la seguridad funcional y la seguridad de la información”*. Cuando un único procesador asume simultáneamente tareas de distinta criticidad temporal, la saturación de cualquiera de ellas puede comprometer el lazo de control de tiempo crítico.

La solución arquitectónica establecida en la industria es la división de responsabilidades entre dos procesadores especializados. Ebeid et al. [8] lo formulan de manera directa: *“consideramos un UAS con dos computadoras a bordo: la primera gestiona las tareas críticas para el vuelo (autopiloto), la segunda —computadora de misión o companion— gestiona las tareas no críticas para el vuelo”*. Esta separación garantiza que, incluso en caso de fallo o sobrecarga del sistema secundario, el lazo de estabilización primario permanece intacto.

Kounev et al. [9] demuestran la aplicación del principio de inversión de control como mecanismo de desacoplamiento entre componentes en sistemas embebidos de seguridad crítica. Este principio, que articula también el diseño del firmware del AFC en este TFM, es el que permite que el sistema secundario sea reemplazable o actualizable sin afectar al sistema primario de vuelo.

2.2.2. *Companion computers*: estado del arte

Un companion computer es una computadora secundaria a bordo del UAV, responsable de las tareas de misión de alto nivel: planificación de rutas, visión por computador, comunicaciones de largo alcance y procesamiento de datos de sensores externos. Ebeid et al. [8] describen el perfil típico de estas computadoras como *“equipos embebidos de pequeño factor de forma, como PC104, mini-ITX, Atom o Edison”*.

Ahmed y Jenihhin [10] documentan el caso de un companion computer basado en el NVIDIA Jetson AGX Xavier para aplicaciones de visión por computador y aprendizaje automático a bordo, concluyendo que *“los dispositivos de procesamiento añaden peso, aumentan el consumo de energía y conllevan a un sistema UAV más complejo y costoso”*. Esta penalización en peso, consumo y coste es la principal limitación que impide el uso de companion computers convencionales en plataformas de pequeño formato.

La revisión de Ebeid et al. [1] confirma que la combinación de Betaflight con companion computers del estilo Jetson Tx2 o Raspberry Pi es técnicamente viable y está documentada en la

literatura. Sin embargo, ningún trabajo propone una solución estructurada y arquitectónicamente completa basada en un MCU de bajo coste para este rol.

2.2.3. El *Micro-Companion Computer*: ESP32 como alternativa de bajo coste

El vacío identificado en la literatura es claro: no existe un enfoque consolidado que emplee microcontroladores de bajo coste como companion computers. Este trabajo ocupa ese espacio proponiendo el ESP32 como *micro-companion computer*. El ESP32 (Espressif Systems) es un SoC de doble núcleo Xtensa LX6 a 240 MHz con Wi-Fi 802.11b/g/n integrado, múltiples interfaces de bus (SPI, I2C, UART), soporte nativo de FreeRTOS y disponibilidad de 4 MB o más de Flash y PSRAM. Su precio de mercado ronda los 3–8 \$, frente a los 35–80 \$ de una Raspberry Pi o los varios centenares de una NVIDIA Jetson.

Beaudoin et al. [6] ya exploraron el uso del ESP32 en el contexto UAV portando ArduPilot a esta plataforma. Sus conclusiones reconocen que “*la capa de abstracción de hardware del ESP32 facilita la integración de comunicaciones y sensores*”, pero detectan limitaciones en entornos aéreos que requieren optimización específica de protocolos. Otro trabajo [11] demuestra el uso del ESP32 como controlador de un dron con interfaz web, aunque en una arquitectura monolítica sin separación de responsabilidades. Ninguno de estos trabajos propone la arquitectura en capas completa con OSAL, inversión de dependencias y patrón Blackboard que se desarrolla en el presente TFM.

2.3. Conectividad IoT y operaciones BVLOS

2.3.1. El dron como dispositivo de red: *Internet of Drones*

Los sistemas de control RC tradicionales operan en bandas ISM (2.4 GHz o 5.8 GHz) con un alcance práctico de 1–2 km en condición de línea de visión directa. Yang et al. [12] identifican cuatro limitaciones fundamentales del paradigma RC clásico: rango restringido, ancho de banda limitado, ausencia de mecanismos de tracking y tiempo de vuelo acotado. El concepto de *Internet of Drones* (IoD) propone tratar al UAV como un nodo más de la red global de Internet, habilitando operaciones más allá de la línea de visión (BVLOS, *Beyond Visual Line of Sight*).

Lin et al. [13] argumentan que “*las redes móviles ofrecen conectividad inalámbrica de área amplia, alta velocidad y segura, lo que puede mejorar significativamente el control y la seguridad de las operaciones de pequeños UAVs y habilitar casos de uso BVLOS*”. Amorim et al. [14] concluyen que “*para las operaciones BVLOS, se requiere una conectividad inalámbrica estable y fiable para el mando y el control*”, señalando las redes celulares como el principal habilitador de este paradigma.

La propuesta de este TFM —integrar un módulo SIM800L en el AFC para conectar el UAV a la red móvil— responde directamente a esta necesidad, convirtiendo el dron en un dispositivo IoT operable globalmente mediante protocolos TCP/UDP estándar.

2.3.2. Conectividad celular para UAVs: LTE/4G como habilitador

Nguyen et al. [15] desarrollaron y evaluaron un prototipo de sistema de control basado en LTE para pequeños drones de baja altitud, midiendo latencia, comportamiento en handover y potencia de señal en condiciones reales. Sus resultados validan la viabilidad técnica de usar redes celulares para control y telemetría de UAVs. Colpaert et al. [16] analizan los problemas de interferencia en altitud con redes terrestres LTE, concluyendo que “*los UAVs a grandes alturas sufren interferencias significativas*” procedentes de múltiples estaciones base en visión directa, lo que limita la cobertura práctica y constituye un reto abierto.

La elección del módulo SIM800L (GSM/GPRS 2G) en este trabajo es una decisión de ingeniería de compromiso deliberada. Si bien el protocolo es de una generación anterior a LTE, el paradigma TCP/IP sobre el que opera es idéntico, y su coste (inferior a 5 \$) y su consumo son incomparablemente menores. El trabajo de [17] valida el paradigma de usar servicios cloud como relay para comunicaciones UAV/UGV sobre red celular, confirmando que los protocolos de transporte estándar (TCP/UDP) son suficientes para telemetría y control. Los retos específicos de interferencia en altura documentados en [16] quedan fuera del alcance del presente prototipo de investigación, pero constituyen una línea de trabajo futura relevante.

2.4. Protocolos de comunicación en sistemas UAV

2.4.1. MSP (*MultiWii Serial Protocol*)

El *MultiWii Serial Protocol* (MSP) es el protocolo binario estándar de comunicación entre la controladora de vuelo y los dispositivos periféricos en el ecosistema Betaflight/INAV. La documentación oficial de ArduPilot [18] lo define como “*el principal protocolo de comunicación utilizado por todas las pilas de vuelo derivadas de Betaflight. Es un protocolo de mensajes binarios basado en petición/respuesta, utilizado para control, telemetría y sensores*”. El protocolo surgió en los primeros años de la comunidad MultiWii (principios de la década de 2010) y ha evolucionado hasta su versión 2, introducida por INAV 1.7 en 2017, que añadió soporte de identificadores de 16 bits y un CRC más robusto [19]. La tasa de refresco efectiva en hardware real alcanza los 1200 Hz [20], lo que lo hace adecuado para la transmisión de referencias de control en tiempo real.

La elección de MSP frente a MAVLink, el protocolo habitual de ArduPilot/PX4, responde a criterios de peso y acoplamiento: MAVLink está orientado a plataformas más pesadas y su implementación completa requiere una pila de código significativamente mayor. En el contexto del AFC sobre ESP32, MSP proporciona las mismas capacidades de control y telemetría con una huella de memoria notablemente menor.

2.4.2. IBus como protocolo de referencia de actitud

El protocolo IBus (*Intelligent Bus*) es el protocolo serie bidireccional de FlySky para la transmisión de datos de mando entre el receptor RC y la controladora de vuelo. Liang [21] describe sus características fundamentales: es un protocolo bidireccional que puede transmitir hasta 14 canales de servo a 115200 baudios en tramas de 32 bytes. Su bajo *overhead* y su simplicidad lo hacen ideal para inyectar referencias de actitud a alta frecuencia [22].

En la arquitectura del AFC, el protocolo IBus se reutiliza para inyectar las referencias de actitud (roll, pitch, throttle, yaw) en la BFC, sustituyendo completamente al receptor RC físico. Esta elección es natural porque Betaflight espera recibir estos datos en formato IBus por UART, y replicar ese comportamiento desde el AFC no requiere ninguna modificación al firmware de la BFC. En el entorno de simulación, el mismo protocolo IBus viaja sobre UDP en lugar de UART física, manteniendo la compatibilidad total con el protocolo sin alterar una sola línea del firmware de la BFC.

2.5. Arquitectura software: capas, abstracciones y patrones

2.5.1. Arquitectura en capas y HAL/OSAL en sistemas embebidos

La arquitectura en capas aplicada a firmware embebido persigue aislar la lógica de negocio de los detalles de implementación del hardware. Los conceptos de *Hardware Abstraction Layer* (HAL) y *Operating System Abstraction Layer* (OSAL) son los mecanismos estándar para lograr esta separación. Según [23], “la capa de abstracción permite que el código de aplicación se mueva de un microcontrolador o plataforma hardware a otro sin modificación. Crear una capa de abstracción es un requisito para la portabilidad y una aplicación del principio de inversión de dependencias”.

La NASA ha adoptado este patrón como piedra angular de su *Core Flight System* (cFS). Su implementación del OSAL [24] proporciona una API uniforme para VxWorks, RTEMS y Linux/POSIX, permitiendo que el mismo código de aplicación se ejecute en el hardware de vuelo real y en un PC de escritorio para pruebas. Este ejemplo de un sistema aeroespacial de misión crítica real avala la solidez del enfoque. Koç y Ünlü [25] aplican la misma filosofía en el dominio UAV, describiendo un sistema responsable de “proveer abstracción de hardware, activar comprobaciones de seguridad y gestionar condiciones inesperadas”.

En el firmware AFC de este TFM, las capas PlatformESP32 y PlatformLinux constituyen la implementación del OSAL y el HAL para cada plataforma, permitiendo que el mismo código de las capas superiores se ejecute sin cambios tanto en el microcontrolador físico como en el entorno de simulación Linux.

2.5.2. Principio de Inversión de Dependencias (DIP)

El Principio de Inversión de Dependencias (DIP, *Dependency Inversion Principle*) fue enunciado por Robert C. Martin en 1994 y forma parte de los principios SOLID del diseño orientado a objetos. Su formulación canónica [26] establece que “*los módulos de alto nivel no deben depender de módulos de bajo nivel. Ambos deben depender de abstracciones*”. Aplicado al firmware embebido, esto significa que la lógica de aplicación no debe referenciar directamente los drivers de hardware, sino que ambos deben depender de una interfaz abstracta común.

Kounev et al. [9] demuestran la aplicación de la inversión de control en sistemas embebidos de seguridad crítica, argumentando que “*la inversión de control es una técnica utilizada para garantizar el bajo acoplamiento entre componentes interrelacionados*”. En estos sistemas, el desacoplamiento no es solo una buena práctica de ingeniería, sino una necesidad de seguridad funcional: permite sustituir, actualizar o probar componentes de forma aislada sin comprometer la integridad del sistema [7].

En el AFC, el DIP se materializa en la interfaz `ITransport`: la lógica de aplicación invoca métodos de envío y recepción de bytes sin saber si por debajo existe una UART física, un bus I2C o un socket TCP. Esta abstracción es la que hace posible que el mismo firmware funcione en el ESP32 real y en Linux sin ninguna recompilación.

2.5.3. El patrón *Blackboard* en sistemas de control

El patrón Blackboard es un patrón de diseño de software de naturaleza comportamental originado en el campo de la inteligencia artificial. Su primera aplicación documentada fue en el sistema de reconocimiento de voz Hearsay-II en los años 1970 [27]. El patrón define tres componentes: una pizarra o memoria global compartida, un conjunto de fuentes de conocimiento independientes que leen y escriben en ella, y un componente de control que coordina la ejecución.

Weyns et al. [28] aplican el patrón Blackboard directamente en sistemas no tripulados, describiendo cómo permite el desacoplamiento entre componentes productores y consumidores de datos, así como la detección de fallos en tiempo de ejecución. Foxwell [29] ofrece una formulación orientada a robótica: “*un conjunto de algoritmos que leen información del mundo y publican resultados en la Blackboard; un conjunto de módulos que leen esos resultados y actualizan el robot en consecuencia*”. La patente USPTO 6.256.679 [30] documenta la aplicación del patrón en un sistema embebido aeronáutico en capas para la medición de combustible, demostrando su eficiencia en plataformas de recursos limitados.

En el AFC, el componente `AlmacenFlexible` implementa este patrón con una extensión propia: cada vez que un productor escribe un valor, el sistema registra automáticamente el timestamp y la frecuencia de actualización, permitiendo que los consumidores comprueben la frescura del dato antes de utilizarlo en el lazo de control.

2.6. Entornos de simulación para UAVs

2.6.1. Software-in-the-Loop (SITL)

La técnica *Software-in-the-Loop* (SITL) consiste en compilar el firmware del controlador de vuelo como un proceso de PC que se comunica con un simulador físico. Esta aproximación permite validar algoritmos de control, depurar lógica de comunicaciones y realizar pruebas de integración sin arriesgar el hardware. Ebeid et al. [2] señalan que PX4 y ArduPilot lideran en el soporte de SITL, con integraciones maduras con simuladores como Gazebo y jMAVSim, mientras que Betaflight carece de herramientas SITL/HIL robustas.

Antonini et al. [31] describen un pipeline completo de SITL para UAVs que combina PX4 con Gazebo para la simulación de dinámica de seis grados de libertad, validando la arquitectura en escenarios de visión por computador. Razón et al. [32] documentan la utilidad del framework SITL para la verificación de modelos de dinámica de vuelo y algoritmos de control, y describen el ciclo metodológico SITL → HIL → vuelo real como práctica establecida de verificación escalonada.

El presente TFM extiende el concepto de SITL más allá de la BFC: el entorno completo de simulación comprende Betaflight SITL + Gazebo + ESS + AFC ejecutándose en Linux. El AFC no distingue si está hablando con hardware real o con el simulador, ya que las interfaces de comunicación son idénticas gracias a la capa de abstracción OSAL.

2.6.2. Hardware-in-the-Loop (HIL)

El ensayo *Hardware-in-the-Loop* (HIL) es un paso intermedio entre SITL y el vuelo real: el hardware físico se conecta al entorno de simulación virtual, de modo que el procesador real ejecuta el firmware mientras recibe datos de sensores simulados y sus órdenes actúan sobre un modelo virtual de la planta. Nguyen et al. [33] describen una arquitectura HIL completa basada en Gazebo y Pixhawk, concluyendo que “*el HILS es un enfoque eficaz en el diseño de sistemas UAV que permite a los ingenieros probar el algoritmo de control en una placa hardware con un modelo UAV en software*”. Razón et al. [32] documentan el ciclo completo SITL → HIL → vuelo real como metodología de verificación escalonada.

El valor específico de HIL en este TFM es verificar que el ESP32 dispone de recursos de CPU y memoria suficientes para ejecutar el firmware AFC bajo carga real, con todos los nodos de datos activos y la red celular operativa. La arquitectura con OSAL hace que la transición de SITL a HIL sea casi transparente: solo cambia la plataforma de ejecución del AFC, mientras que las conexiones pasan de sockets locales a la red inalámbrica local.

2.6.3. Gazebo como motor de simulación física

Gazebo es el simulador físico de código abierto más utilizado en el ámbito de la robótica y los UAVs. Antonini et al. [31] describen su uso para la simulación de la dinámica UAV de seis grados de libertad, incluyendo modelos de sensores virtuales y cámara RGB-D. Nguyen et al. [33] lo emplean para generar “*un modelo no lineal de seis grados de libertad, modelo de sensores y visualización 3D para un UAV quadrotor*”.

En el entorno de simulación del TFM, Gazebo actúa como fuente de verdad física: recibe los comandos que el BFC/SITL envía a los motores, calcula la dinámica resultante y genera los datos de posición, orientación, velocidad y aceleración. El módulo ESS (*External Sensors Simulation*) toma estos datos y los convierte en señales I2C/UART simuladas sobre sockets TCP, de modo que el AFC los interpreta exactamente igual que si fueran los sensores físicos reales.

2.7. Síntesis: posicionamiento del trabajo

La revisión del estado del arte permite identificar con precisión el espacio que ocupa el presente TFM. La Tabla 2 cruza las dimensiones clave frente a los trabajos más representativos.

Tabla 2. Comparativa del TFM frente a trabajos relacionados

Trabajo	Bajo coste	BVLOS/IoT	SITL/HIL	Arq. capas	MCU como CC
Sørensen 2017 [3]	✓	—	—	—	—
Beaudoin 2020 [6]	✓	—	—	parcial	parcial
Ebeid 2021 [8]	—	—	—	✓	—
Antonini 2019 [31]	—	—	✓	—	—
Nguyen 2018 [33]	—	—	✓	—	—
Yang/Lin 2018 [12, 13]	—	✓	—	—	—
Este TFM	✓	✓	✓	✓	✓

Ninguno de los trabajos revisados combina simultáneamente las cinco dimensiones. La propuesta de este TFM es la primera que integra un MCU de bajo coste como companion computer dentro de una arquitectura en capas completa (OSAL + DIP + Blackboard), con soporte de conectividad celular y un entorno de desarrollo SITL/HIL completo.

Capítulo 3

Motivación y Objetivos

3.1. Motivación

MODIFICAR el código de ecosistemas cerrados o de gran envergadura —como ArduPilot o PX4— para añadir lógica de control propia o hardware no estándar es un proceso complejo y costoso. La solución habitual en proyectos de investigación y prototipado es incorporar un *companion computer* (p.ej. Raspberry Pi —plataforma originalmente educativa, extendida al ámbito DIY y de bajo presupuesto—), lo que dispara el coste, el peso y el consumo energético, impidiendo su uso en plataformas pequeñas.

La propuesta de este TFM es crear una arquitectura en la que un microcontrolador ultrabarato y ligero (ESP32) actúe como *micro-companion computer*. Se aprovecha un dron básico con Betaflight para el vuelo a bajo nivel y se le dota de capacidades autónomas, IoT y de experimentación personalizadas por una fracción del coste y el peso de un sistema tradicional.

3.1.1. Seguridad mediante arquitectura distribuida

Concentrar toda la lógica en un único chip —estabilización de motores, lectura de GPS, navegación y envío de datos— conlleva un riesgo de saturación del procesador. Si el cálculo de una ruta tarda un milisegundo de más, el dron pierde el equilibrio.

La solución adoptada es dividir el trabajo delegando la *supervivencia* del dron (el lazo de control rápido de los motores) a Betaflight (BFC). De esta forma, si el AFC falla, tarda en procesar algo complejo o se reinicia, el dron entra en un modo seguro pero no cae a plomo.

3.1.2. El dron como dispositivo IoT: romper la barrera de la distancia

Los sistemas tradicionales dependen de un mando por radiofrecuencia (RC) que obliga a mantener el dron en la línea de visión directa (máximo 1–2 km). La motivación es abstraer las comunicaciones a protocolos de internet (TCP/UDP) e integrar módulos como el SIM800L,

convirtiendo el dron en un nodo de red global operable desde cualquier punto del mundo a través de la red móvil.

3.2. Objetivos del proyecto

A continuación se enumeran los objetivos que debe cumplir el sistema desarrollado.

3.2.1. Software lo más flexible posible

1. **Abstracción del sistema operativo (OSAL).** El código principal debe ser indiferente a si se ejecuta en un MCU real usando FreeRTOS o en el simulador en un PC usando Linux.
2. **Abstracción de las comunicaciones (HAL/Transport).** Interfaz genérica de *send/receive* de bytes crudos para cualquier tipo de comunicación: UART física, I2C o sockets TCP/UDP.
3. **Inversión de dependencias.** Las abstracciones (OSAL y transportes) se inyectan en la lógica principal de la aplicación, que es así independiente del hardware y del sistema operativo.

3.2.2. Control del dron sin mando manual

1. **Estación de control en tierra (GCS).** Interfaz que reemplaza al mando RC tradicional, permitiendo enviar referencias, configuraciones y solicitar telemetría.
2. **Arquitectura independiente de la red.** El sistema debe ser indiferente a si la conexión GCS–AFC se realiza por sockets internos, red local o red móvil global.

3.2.3. Entorno de desarrollo ágil

1. **SITL (*Software-in-the-Loop*).** Entorno de simulación completo (Betaflight SITL + Gazebo + ESS) para iterar el código sin volar el dron.
2. **HIL (*Hardware-in-the-Loop*).** Conexión de la placa AFC física al ciclo de simulación para verificar recursos de CPU y memoria del MCU.
3. **Sintonización de control *on-the-fly*.** Inyección de nuevos parámetros de control al AFC desde la GCS sin necesidad de recompilar ni reflashear el firmware.

Capítulo 4

Recursos y Metodología

4.1. Recursos hardware

4.1.1. Plataforma base

- **BFC — Controladora de vuelo base:** MAMBA F405MINIMK2, ejecutando Betaflight.
- **Powertrain:** ESCs, motores brushless, hélices y *frame* FPV de 5”.
- **Receptor RC:** receptor Flysky con protocolo IBus/UART.

4.1.2. Modificaciones — sistema UAV completo

- **AFC — Controladora avanzada:** módulo AM-036 (ESP32-WROVER-B + SIM800L).
 - Procesador dual-core Xtensa LX6 a 240 MHz.
 - Wi-Fi 802.11b/g/n integrado, 4 MB Flash, 8 MB PSRAM.
 - Módulo GSM/GPRS SIM800L para conectividad celular.
- **GPS + Magnetómetro:** WS-M181 (UART + I2C).
- **Barómetro:** BMP390 (I2C).

4.1.3. Equipo de desarrollo

- PC con Linux para compilación cruzada y simulación.
- Programador/depurador JTAG para ESP32.
- Instrumentación básica de laboratorio (multímetro, osciloscopio).

4.2. Recursos software

4.2.1. Firmware y sistemas embebidos

- **Betaflight** [5]: firmware para la BFC.
- **ESP-IDF**: SDK oficial de Espressif para ESP32 (incluye FreeRTOS).
- **Firmware AFC** (este TFM): código C++ con arquitectura en 5 capas.

4.2.2. Entorno de simulación

- **Betaflight SITL**: instancia de Betaflight compilada para Linux.
- **Gazebo**: simulador físico de robots y UAVs.
- **ESS** (este TFM): emulación de sensores GPS, magnetómetro y barómetro sobre TCP.

4.2.3. Estación de control en tierra (GCS)

- **GCS** (este TFM): mando virtual IBus/UDP, configuración MSP/TCP, telemetría, identificación de planta y diseño de control.

4.2.4. Herramientas de desarrollo

- **CMake / Make**: sistema de construcción.
- **GCC / xtensa-esp32-elf-gcc**: compiladores para Linux y ESP32.
- **Git**: control de versiones.
- **GDB / OpenOCD**: depuración.

4.3. Metodología de trabajo

El proyecto sigue un enfoque iterativo e incremental. El ciclo de trabajo para cada componente es:

1. **Diseño de la interfaz**. Se define el contrato abstracto antes de cualquier implementación concreta.
2. **Implementación en Linux**. El componente se desarrolla y prueba en PC sin hardware.
3. **Validación en SITL**. Se integra en el entorno completo Betaflight SITL + Gazebo + ESS.
4. **Portado a ESP32**. Se implementa la misma interfaz para FreeRTOS + esp-idf.

5. **Verificación HIL.** El firmware completo corre en el hardware real conectado a la simulación, verificando recursos de CPU y memoria.

4.3.1. Sintonización de control *on-the-fly*

Para el diseño de las leyes de control se sigue un ciclo de identificación basado en datos:

1. Diseño del ensayo desde la GCS y recogida de telemetría.
2. Identificación paramétrica de la planta.
3. Diseño del controlador con los parámetros identificados.
4. Inyección de nuevos parámetros al AFC mediante la GCS sin recompilar.
5. Validación del comportamiento en lazo cerrado.

Capítulo 5

Plan de Trabajo y Cronograma

5.1. Fases del proyecto

El desarrollo se organiza en las siguientes fases secuenciales, con algún solapamiento en las etapas de validación:

1. **Fase 1 — Infraestructura base.** Definición de las interfaces del Core (OSAL, ITransport), implementación de las plataformas Linux y ESP32, y puesta en marcha del entorno de simulación SITL + Gazebo.
2. **Fase 2 — Capa de comunicaciones.** Implementación de los encoders/decoders MSP e IBus, ensamblaje del sistema de canales (Channel, ChannelServer, ChannelAggregator, ChannelDisregator, PersistentChannel) y validación en SITL.
3. **Fase 3 — Lógica de aplicación.** Implementación del Blackboard (AlmacenFlexible), CommandManager, ControlManager y DataNodesManager. Integración con el entorno de simulación completo.
4. **Fase 4 — GCS y control.** Desarrollo de la estación de control en tierra con mando virtual, telemetría, identificación de planta y diseño de control on-the-fly.
5. **Fase 5 — Integración y validación HIL.** Portado completo al hardware ESP32, verificación de recursos y pruebas de vuelo en entorno controlado.
6. **Fase 6 — Documentación.** Redacción de la memoria del TFM.

5.2. Cronograma

Tabla 3. Cronograma estimado del proyecto

Fase	Enero	Febrero	Marzo	Abril	Mayo	Junio
1 — Infraestructura base	✓	✓				
2 — Comunicaciones		✓	✓			
3 — Lógica de aplicación			✓	✓		
4 — GCS y control				✓	✓	
5 — Integración HIL					✓	
6 — Documentación				✓	✓	✓

Capítulo 6

Arquitectura del Firmware AFC

El Advanced Flight Controller (AFC) constituye el núcleo software del sistema UAV desarrollado en este trabajo. Es el componente responsable de integrar la telemetría procedente de los sensores externos, ejecutar las leyes de control de alto nivel y mantener la comunicación bidireccional tanto con la Ground Control Station (GCS) como con el Basic Flight Controller (BFC). Para alcanzar los objetivos de portabilidad, desacoplamiento y facilidad de desarrollo enunciados en el capítulo de objetivos —en particular, la compilación dual sobre ESP32/FreeRTOS y sobre Linux estándar para el entorno de simulación— el firmware se estructura mediante una arquitectura en cinco capas con inversión de dependencias. Esta organización permite que la lógica de aplicación sea completamente independiente del hardware y del sistema operativo, y que la variabilidad de plataforma quede confinada en una única capa intercambiable.

6.1. Visión general de la arquitectura

El modelo arquitectónico adoptado organiza el firmware en cinco capas con responsabilidades bien delimitadas: Plataforma e Infraestructura (Capa 1), Core e Interfaces (Capa 2), Comunicaciones (Capa 3), Lógica de Aplicación (Capa 4) y Ensamblaje (Capa 5). La motivación central de esta organización es la dualidad de plataformas de ejecución: el AFC debe funcionar indistintamente sobre un ESP32 con FreeRTOS, donde accede a drivers de hardware real mediante el framework esp-idf (UART, I2C), y sobre Linux con pthreads, donde todas las comunicaciones se simulan mediante sockets POSIX. Una arquitectura monolítica haría que cada cambio de plataforma obligase a revisar y adaptar la lógica de control; el modelo de capas confina esa variabilidad en la Capa 1 y deja las capas superiores completamente independientes del hardware y del sistema operativo.

6.1.1. Diagrama de capas

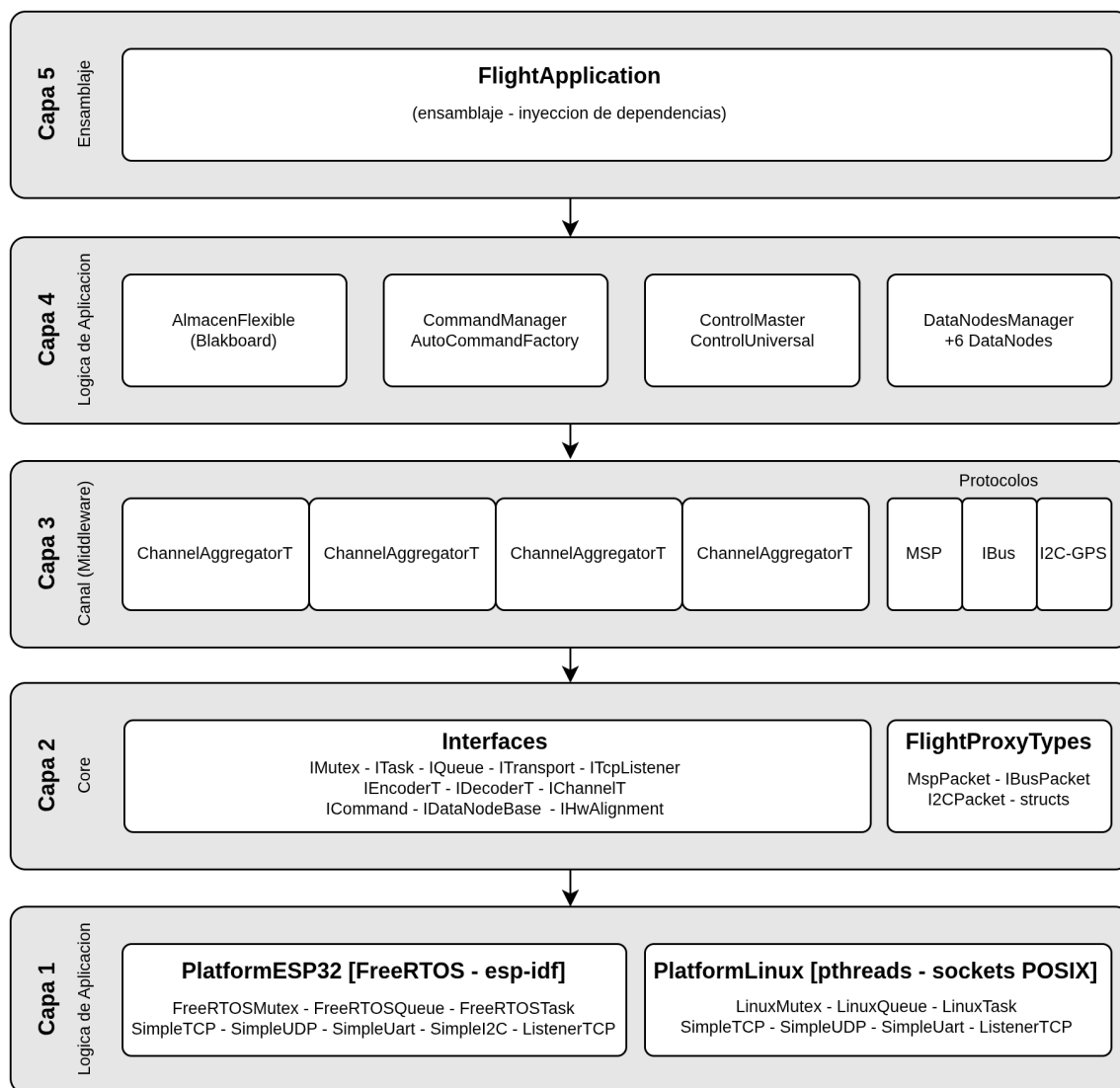


Figura 3. Vista general de capas del firmware AFC

6.1.2. Principios de diseño aplicados

Cuatro principios vertebran el diseño. En primer lugar, la **inversión de dependencias** (DIP): las capas superiores (AppLogic, Channel) dependen únicamente de los contratos abstractos del Core, nunca de las implementaciones concretas de Platform. En segundo lugar, la **inyección de dependencias en el ensamblaje**: es en la Capa 5 (FlightApplication) donde se instancian los objetos concretos de plataforma y se inyectan en los subsistemas que los necesitan, sin que ningún subsistema los instance directamente. En tercer lugar, la

portabilidad mediante factories: TransportFactory y OSALFactory son las únicas clases que instancian objetos dependientes de plataforma; el mecanismo de selección se resuelve en tiempo de compilación mediante CMake, sin directivas `#ifdef` dispersas por el código de negocio. En cuarto lugar, el **tipado fuerte de datos compartidos** mediante la macro `REGISTER_FP_STRUCT`: todos los tipos de datos que circulan por el blackboard se definen a través de esta macro, que genera en tiempo de compilación la estructura C++, su firma de tipos y sus metadatos de campo, habilitando la introspección automática del protocolo MSP.

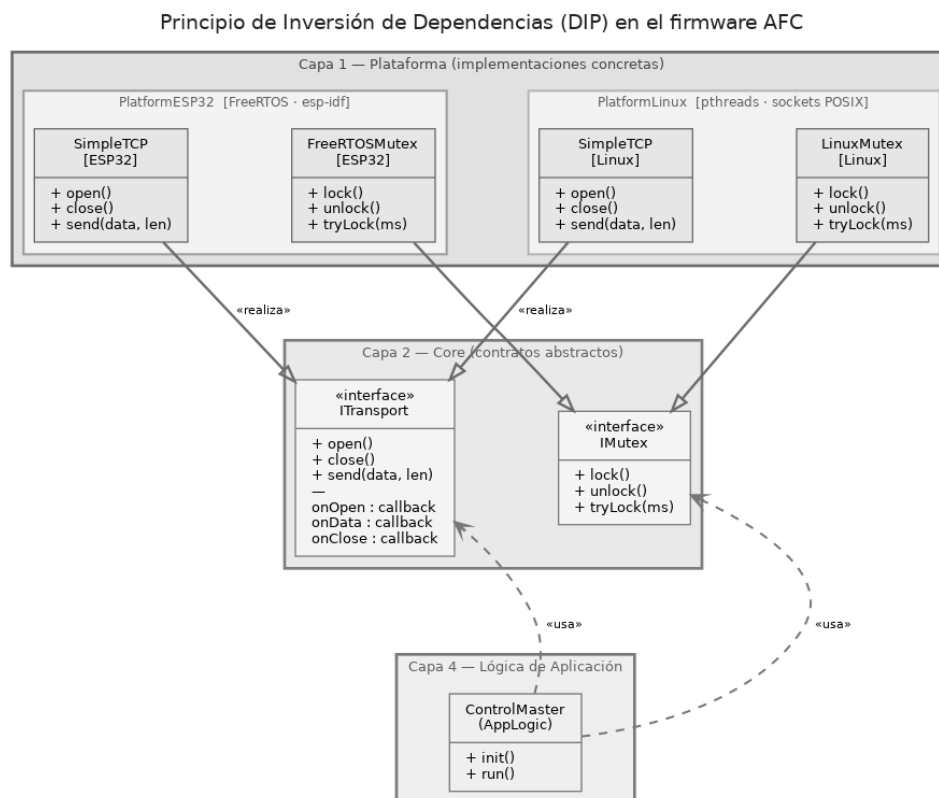


Figura 4. Principio de inversión de dependencias en la arquitectura AFC

6.2. Capa 1 — Plataforma e Infraestructura

La Capa 1 agrupa las carpetas `PlatformESP32` y `PlatformLinux` y constituye el punto de contacto directo con el hardware y el sistema operativo. Su única misión es implementar los contratos abstractos definidos en el Core, adaptándolos a las particularidades de cada plataforma. Es la única capa del sistema que contiene código dependiente del entorno; el resto de las capas desconoce por completo si se están ejecutando sobre un microcontrolador o sobre un ordenador de propósito general.

6.2.1. Implementaciones de transporte

6.2.1.1. Transportes de datos básicos

El contrato `ITransport`, definido en `Core/Transport/ITransport.h`, establece la interfaz mínima para cualquier medio de transmisión de bytes. Los métodos `open()`, `close()` y `send(data, len)` cubren el ciclo de vida activo del transporte, mientras que tres callbacks —`onOpen`, `onData` y `onClose`— permiten que el consumidor del transporte reaccione a eventos asíncronos de conexión, recepción y desconexión sin necesidad de bloquear la ejecución.

```
1 // Extracto de:
   components/Core/include/FlightProxy/Core/Transport/ITransport.h
2
3 class ITransport {
4 public:
5     virtual ~ITransport() = default;
6
7     // Metodos de control del ciclo de vida del transporte
8     virtual void open() = 0;
9     virtual void close() = 0;
10
11     // Envio de datos crudos (bytes)
12     virtual void send(const uint8_t* data, size_t len) = 0;
13
14     // Callbacks registrados por el consumidor del transporte
15     std::function<void()> onOpen; //
        conexion establecida
16     std::function<void(const uint8_t*, size_t)> onData; // datos
        recibidos
17     std::function<void()> onClose; //
        conexion cerrada
18 };
```

Listing 6.1. Interfaz `ITransport` — contrato de transporte (Capa 2, Core)

Sobre esta interfaz se implementan, en la plataforma ESP32, los transportes `SimpleTCP`, `SimpleUDP`, `SimpleUart` y `SimpleI2C` mediante los drivers nativos de `esp-idf`. En la plataforma Linux, los transportes (excepto `SimpleI2C`, que no tiene equivalente en Linux) se implementan sobre la biblioteca de sockets POSIX: `SimpleTCP` opera como cliente TCP, `SimpleUDP` como socket UDP, y `SimpleUart` también se encamina por TCP para poder participar en el entorno de simulación sin hardware físico.

6.2.1.2. TcpListener — transporte generador

La interfaz `ITcpListener` actúa como «transporte generador»: no envía datos propios, sino que permanece en escucha sobre un puerto TCP y, ante cada nueva conexión entrante, invoca un callback entregando un objeto `ITransport` nuevo y listo para usar. Sus implementaciones son `ListenerTCP` en ambas plataformas, con comportamiento equivalente.

6.2.2. Implementaciones del OSAL

El Operating System Abstraction Layer (OSAL) proporciona una interfaz genérica para las primitivas de concurrencia que la lógica de aplicación necesita, sin acoplarla a ningún RTOS ni a la biblioteca de hilos del sistema operativo anfitrión. Las tres interfaces centrales son:

- **IMutex** (`lock()`, `unlock()`, `tryLock(timeout_ms)`): garantiza el acceso exclusivo a recursos compartidos.
- **IQueue** (`send()`, `receive()`, `size()`): cola de transferencia de bloques de bytes sin tipo. La clase `QueueWrapper<T>`, común a ambas plataformas, envuelve un `IQueue` crudo y expone la versión tipada `IQueueT<T>` que es la que utiliza el código de negocio.
- **ITask** (`start()`, `stop()`, `join()`, `isRunning()`): abstracción de tarea concurrente.

6.2.2.1. FreeRTOS (hardware ESP32)

En la plataforma ESP32, `IMutex` se implementa como `FreeRTOSMutex` mediante `xSemaphoreCreateMutex` y las primitivas de toma y liberación del RTOS. `IQueue` se implementa como `FreeRTOSQueue` sobre `xQueueCreate`, con transferencia de bloques de bytes de tamaño fijo. `ITask` se implementa como `FreeRTOSTask`: una tarea FreeRTOS con prioridad, tamaño de stack y afinidad de núcleo configurables desde el constructor.

6.2.2.2. Linux (entorno de simulación)

En la plataforma Linux, `IMutex` se implementa como `LinuxMutex` sobre `pthread_mutex_t`. `IQueue` se implementa como `LinuxQueue` mediante una `std::queue` protegida por mutex y una variable de condición para el bloqueo de `receive()`. `ITask` se implementa como `LinuxTask` sobre un hilo POSIX (`std::thread`).

Tabla 4. Correspondencia interfaz–implementación para OSAL y Transport en las dos plataformas

Interfaz	Implementación ESP32	Implementación Linux
ITransport	SimpleUart, SimpleTCP, SimpleUDP, SimpleI2C	SimpleUart, SimpleTCP, SimpleUDP
ITcpListener	ListenerTCP	ListenerTCP
IMutex	FreeRTOSMutex	LinuxMutex
IQueue	FreeRTOSQueue	LinuxQueue
ITask	FreeRTOSTask	LinuxTask

6.2.3. Compilación condicional de plataforma

TransportFactory y OSALFactory son las únicas clases que instancian objetos concretos de Capa 1. Ambas disponen de una implementación por plataforma, y en el proceso de compilación CMake se selecciona la implementación correcta al escoger el target correspondiente. Este mecanismo garantiza que ninguna parte del código de la Capa 3 o superior contenga código condicional de plataforma: la selección es puramente estructural y ocurre una sola vez, en el descriptor de compilación.

6.3. Capa 2 — Core e Interfaces

El Core actúa como «muro de abstracción» entre la lógica de aplicación y los detalles de implementación de hardware y protocolo. No contiene código ejecutable ni lógica de negocio: su contenido son exclusivamente contratos (clases C++ con métodos virtuales puros), tipos de datos globales y las cabeceras de las factories. Esta separación es la que hace posible que AppLogic y Channel nunca dependan directamente de Platform.

El Core define un total de 14 interfaces: 13 con métodos definidos más IConnectivity, que es un marcador vacío sin métodos (placeholder para conectividad futura).

6.3.1. Interfaces de plataforma

6.3.1.1. ITransport

ITransport (descrito en la sección anterior) y ITcpListener constituyen los contratos de transmisión de bytes. ITransport tiene cuatro implementaciones en ESP32 (incluyendo SimpleI2C) y tres en Linux; ITcpListener tiene una implementación por plataforma (ListenerTCP).

6.3.1.2. Interfaces OSAL: IMutex, IQueue, ITask

Los contratos IMutex, IQueue, IQueueT<T> e ITask son los contratos de concurrencia implementados por la Capa 1 de Platform. Son consumidos directamente por la lógica de aplicación (Capa 4) y por el middleware de canales (Capa 3) para operaciones thread-safe, sin que ninguna de estas capas tenga conocimiento de la plataforma subyacente.

6.3.2. Interfaces de protocolo

6.3.2.1. IEncoder

IEncoderT<T> expone el método `encode(packet) -> vector<uint8_t>`, que serializa una estructura tipada en una secuencia de bytes lista para enviar por el transporte. Sus implementaciones concretas —MspProtocol, IbusProtocol, GpsProtocol, I2CProtocol— son comunes a ambas plataformas y residen en el propio Core.

6.3.2.2. IDecoder

IDecoderT<T> expone `feed(data, len)`, `onPacket(handler)` y `reset()`. El modelo de decodificación es uniforme: `feed()` acumula bytes entrantes y los hace pasar por la máquina de estados interna de cada protocolo; cuando el último byte completa un paquete válido, se invoca automáticamente el callback `onPacket` con la estructura ya deserializada y validada.

6.3.3. Factorías: TransportFactory y OSALFactory

Las factories del Core exponen métodos de fábrica tipados que devuelven punteros a las interfaces abstractas (`unique_ptr<ITransport>`, `unique_ptr<IMutex>`, etc.). La implementación concreta de cada factory —seleccionada por CMake en tiempo de enlace— instancia los objetos de la plataforma correcta. De este modo, cualquier subsistema que necesite crear objetos de Capa 1 lo hace a través de la factory, sin incluir ninguna cabecera de plataforma específica.

6.3.4. Tipos de datos globales

El Core estandariza todos los tipos de datos que circulan por el sistema mediante la macro `REGISTER_FP_STRUCT`, definida en `TypeSignature.h`. Cada invocación de la macro recibe el nombre del tipo y una X-macro con la lista de sus campos, y genera en tiempo de compilación tres artefactos: el struct en el namespace `FlightProxy::Core`, la especialización `TypeSignature<T>::Get()` —que devuelve la firma de tipos de los campos, por ejemplo `"s<f32,f32,f32,f32,f32,f32>"`— y la especialización `TypeFields<T>::Get()` —que devuelve los nombres de campo separados por coma—. Estos metadatos son consumidos por `AutoCommandFactory` para construir automáticamente los comandos MSP de introspección.

```
1 // Extracto de:
  components/Core/include/FlightProxy/Core/FlightProxyTypes.h
2 //
  components/Core/include/FlightProxy/Core/TypeSignature.h
3
4 #define IMU_DATA_FIELDS(X) \
5     X(float, accel_x)      \
6     X(float, accel_y)      \
7     X(float, accel_z)      \
8     X(float, gyro_x)       \
9     X(float, gyro_y)       \
10    X(float, gyro_z)
11
12 REGISTER_FP_STRUCT(IMUData, IMU_DATA_FIELDS)
13
14 // Código C++ generado por la macro:
15 namespace FlightProxy { namespace Core {
16     struct IMUData {
17         float accel_x; float accel_y; float accel_z;
18         float gyro_x;  float gyro_y;  float gyro_z;
19     };
20 }}
21 // TypeSignature<IMUData>::Get() --> "s<f32,f32,f32,f32,f32,f32>"
22 // TypeFields<IMUData>::Get()    -->
  "accel_x,accel_y,accel_z,gyro_x,gyro_y,gyro_z"
```

Código 1. Macro REGISTER_FP_STRUCT y ejemplo de tipo generado (IMUData)

Los 12 tipos definidos en `FlightProxyTypes.h` son: `StatusData`, `RCDData`, `RCNORMData`, `GPSData`, `MagData`, `BaroData`, `IMUData`, `AttitudeData`, `VelocityData`, `YawData`, `PidCtrlIn` y `PidCtrlOut`.

6.4. Capa 3 — Middleware de Comunicaciones

La Capa 3 da el salto de abstracción definitivo en el tratamiento de la información: las capas inferiores trabajan con bytes crudos; la Capa 3 trabaja con estructuras de datos tipadas. Se organiza en las carpetas `Channel` y `Core/Protocol`.

6.4.1. Implementaciones de protocolo

6.4.1.1. MSP — MultiWii Serial Protocol

MSP v2 (MultiWii Serial Protocol versión 2) es el protocolo principal de comunicación con la GCS y con el BFC. La serialización se basa en una cabecera de dos bytes (`$X`), un campo de

flags, un `CommandId` en 16 bits y un `CRC8-DVB-S2` al final del paquete. El `MspDecoder` implementa una máquina de estados que consume bytes entrantes byte a byte, valida el `CRC` al recibir el último byte del payload y solo entonces invoca el callback `onPacket` con la estructura ya deserializada. El `MspEncoder` realiza el proceso inverso: recibe una estructura tipada, construye la trama binaria completa y devuelve el vector de bytes listo para enviar.

6.4.1.2. IBus

IBus es el protocolo de radiocontrol de 14 canales en formato binario compacto, empleado para la transmisión continua de referencias de vuelo desde la GCS. Cada trama ocupa 32 bytes: un byte de longitud, un byte de tipo y 28 bytes de datos de canal ($14 \text{ canales} \times 2 \text{ bytes}$, little-endian), seguidos de 2 bytes de checksum. El `IbusDecoder` acepta también la variante de 30 bytes sin canal de telemetría. Los protocolos GPS e I2C siguen el mismo patrón de par encoder/decoder, especializados para los formatos de los módulos WS-M181 y BMP390 respectivamente.

6.4.2. El concepto de Channel

6.4.2.1. Composición: ITransport + IEncoder + IDecoder

`ChannelT<T>` es el contenedor fundamental de la capa: ensambla un `ITransport`, un `IEncoderT<T>` y un `IDecoderT<T>` en una única unidad cohesionada que implementa `IChannelT<T>`. Ninguno de los tres componentes conoce a los otros dos: el channel hace de pegamento, registrando los callbacks internos durante la construcción y gestionando el ciclo de vida del conjunto.

6.4.2.2. Flujo de datos interno

El ciclo de recepción opera en cinco pasos: (1) el transporte recibe bytes del medio físico y los entrega al channel mediante el callback `onData`; (2) el channel los pasa al decoder mediante `feed()`; (3) el decoder los acumula en su máquina de estados interna; (4) cuando el paquete está completo, el decoder invoca su `onPacket` interno; (5) el channel reenvía el paquete completo como estructura tipada al consumidor externo mediante su propio callback `onPacket`. El ciclo de emisión es el proceso inverso en cuatro pasos: (1) el canal recibe una estructura tipada vía `sendPacket()`; (2) la pasa al encoder, que la serializa en bytes; (3) el canal recibe los bytes del encoder; (4) los entrega al transporte para su envío por el medio físico.

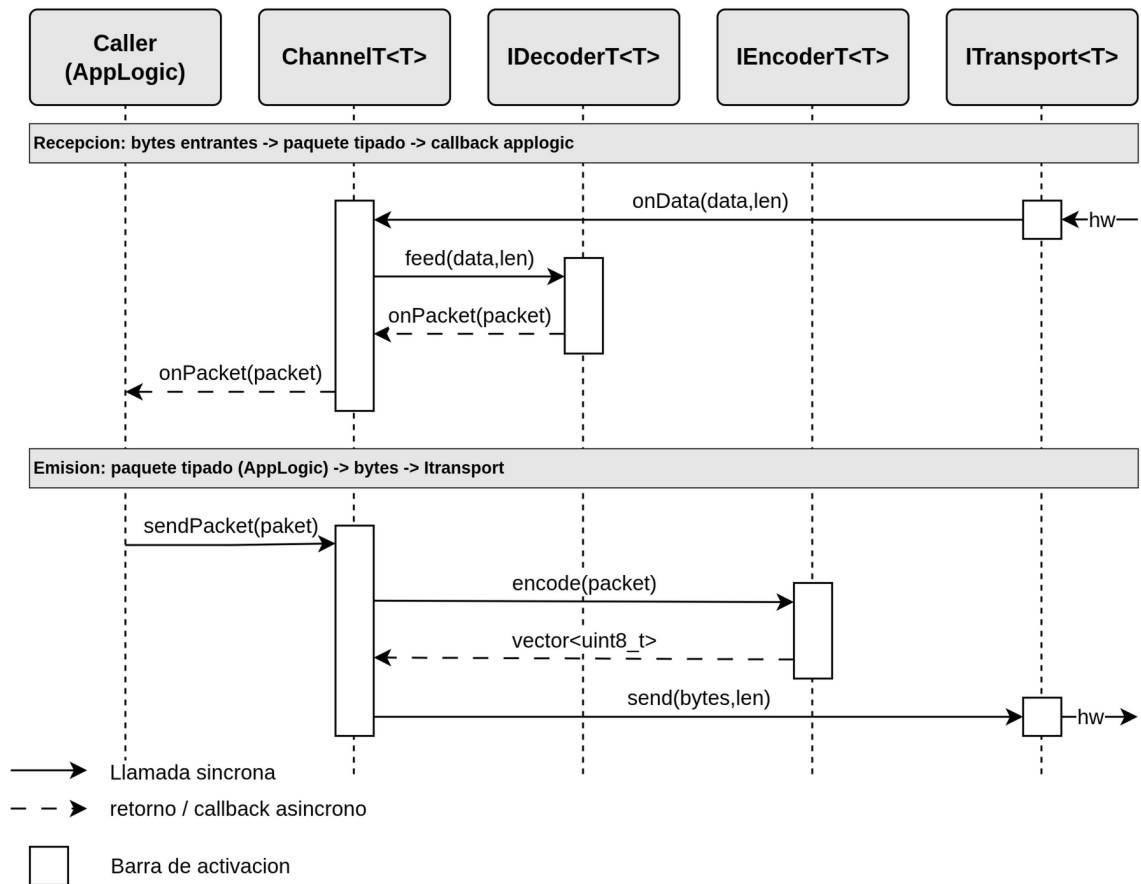


Figura 5. Flujo de datos interno en un `ChannelT<T>`: ciclos de recepción y emisión

6.4.3. Enrutamiento avanzado de canales

Para gestionar topologías de conexión complejas —múltiples clientes GCS simultáneos, multiplexación de sensores sobre un único enlace físico con el BFC— se han diseñado tres variantes de canal sobre la abstracción `IChannelT<T>`:

6.4.3.1. ChannelServer

`ChannelServer<T>` es el análogo del `ITcpListener` de la Capa 1 pero a nivel de canal. Utiliza internamente un `ITcpListener` y, ante cada conexión TCP entrante, instancia automáticamente un `ChannelT<T>` completo —con su encoder y decoder ya configurados— y lo pone a disposición de la capa superior mediante un callback `onNewChannel`.

6.4.3.2. ChannelAggregator

`ChannelAggregatorT<T>` multiplexa N canales del mismo tipo en una única interfaz de entrada. Asigna a cada canal un identificador numérico único, inyecta ese identificador en el

callback `onPacket` de cada canal, y de este modo permite que, cuando el `CommandManager` genera una respuesta, el agregador pueda enrutarla de vuelta al canal de origen exacto. Este mecanismo permite manejar conexiones GCS simultáneas sin que el sistema de comandos tenga que conocer la topología de red.

6.4.3.3. ChannelDisgregator

`ChannelDisgregatorT<T>` realiza la operación inversa al agregador. Un método discriminador —en la práctica, el campo `CommandId` del paquete MSP— determina a cuál de los N canales lógicos debe derivarse cada paquete entrante. Así, un único enlace físico con el BFC (UART en hardware real, TCP en simulación) se descompone en tantos canales lógicos como `DataNodes` hay activos, sin que ningún nodo tenga que conocer la multiplexación subyacente.

6.4.3.4. PersistentChannel

La clase `ChannelPersistentT<T>` añade reconexión automática ante desconexiones inesperadas mediante un backoff de 5 s. Está implementada en el repositorio pero no activa en la compilación actual.

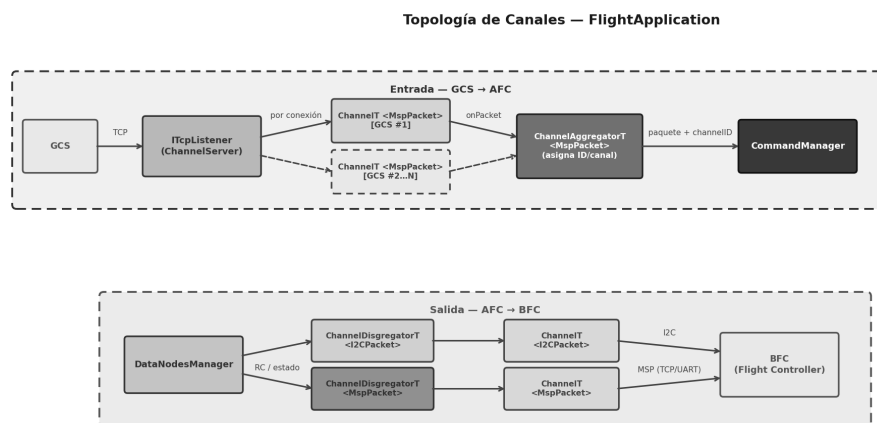


Figura 6. Enrutamiento de canales: `ChannelServer` y `ChannelAggregator` (rama GCS) y `ChannelDisgregator` (rama BFC)

6.5. Capa 4 — Lógica de Aplicación

La Capa 4, ubicada en la carpeta `AppLogic`, es completamente independiente del hardware, de la red y del sistema operativo. No conoce si los datos llegan por UART física o por un socket TCP: solo opera sobre las estructuras de datos tipadas que recibe y produce. Se organiza en tres subsistemas: el blackboard (`AlmacenFlexible`), el sistema de comandos

(CommandManager + AutoCommandFactory) y el sistema de control (ControlMaster + ControlUniversal + DataNodesManager).

6.5.1. El patrón Blackboard: AlmacenFlexible

6.5.1.1. Índice centralizado de datos

AlmacenFlexible implementa el patrón Blackboard: un mapa tipado, thread-safe, indexado por DataID (entero). El acceso al almacén se realiza exclusivamente a través de lambdas obtenidas en la fase de inicialización:

- registrarProductor<T>(id) devuelve una `std::function<void(T)>` que escribe atómicamente el valor en el slot correspondiente y actualiza su timestamp.
- registrarConsumidor<T>(id) devuelve una `std::function<T()>` que lee el último valor almacenado de forma thread-safe.

Este mecanismo desacopla completamente a productores y consumidores: ninguno conoce quién es el otro, solo el DataID que comparten. A continuación se muestra cómo ControlMaster registra sus lambdas durante la inicialización:

```
1 // Extracto de: components/AppLogic/src/Control/ControlMaster.cpp
2
3 void ControlMaster::init(std::shared_ptr<AlmacenFlexible> bb) {
4     // Consumidor: devuelve std::function<RCData()>
5     inRcGetter_ =
6         bb->registrarConsumidor<Core::RCData>(ID_RC_Input);
7
8     // Productores: devuelven std::function<void(T)>
9     outRcNormSetter_ =
10         bb->registrarProductor<Core::RCNORMData>(ID_RC_InputNorm);
11     outRcSetter_ =
12         bb->registrarProductor<Core::RCData>(ID_RC_Output);
13
14     this->control_.init(bb);
15 }
```

Código 2. Registro de productores y consumidores en AlmacenFlexible desde ControlMaster

Adicionalmente, el blackboard registra el timestamp de cada escritura y expone métodos para consultar la antigüedad del dato y su frecuencia de actualización, lo que permite al sistema de control detectar sensores silenciosos sin instrumentación adicional.

6.5.1.2. Metadatos temporales

Los 35 DataIDs activos se agrupan en cinco categorías funcionales:

- **Entradas RC:** ID_RC_Input (1, RCDData crudo desde IBus/UDP), ID_RC_InputNorm (3, RCNORMData normalizado a $[-1, 1]$) e ID_RC_Output (2, referencias finales hacia el BFC).
- **Telemetría raw y alineada:** pares (raw/alineado) para actitud (IDs 5/10), IMU (6/11), magnetómetro (7/12), GPS (8/13) y barómetro (9/14). La alineación hardware corrige la orientación física de cada sensor antes de depositarlo en el slot «alineado» correspondiente.
- **Salidas del sistema de control:** velocidad estimada ID_VEL_Data (15), estimación de yaw ID_YAW_Data (16), y las salidas de los siete bucles PID (IDs 21, 23, 25, 27, 43, 41, 45 para velocidad lateral, frontal, vertical, angular, posición vertical, horizontal y angular respectivamente).
- **Parámetros de control:** nivel de control ID_CTRL_LVL (19), la máscara de bypass por lazo ID_CTRL_LOOP_BYPASS (28, uso en pruebas) y los parámetros PID de entrada para cada eje (IDs 20, 22, 24, 26, 40, 42, 44).
- **Navegación por ruta:** ruta recibida de la GCS ID_ROUTE_CMD (30) y su progreso ID_ROUTE_STATUS (31), descritos en §6.5.3.

Tabla 5. Catálogo de DataIDs del blackboard (AlmacenFlexible) con tipo, productor y consumidor

Val.	DataID	Tipo C++	Productor	Consumidor	Grupo
0	ID_STATUS_Data	StatusData	Nodo_Recepcion_Status	AutoCmd-GET	Estado
1	ID_RC_Input	RCData	FlightApp(UDP) · AutoCmd-SET	ControlMaster · AutoCmd-GET	RC
2	ID_RC_Output	RCData	ControlMaster	Nodo_Emission_RC · AutoCmd-GET	RC
3	ID_RC_InputNorm	RCNORMData	ControlMaster	AutoCmd-GET	RC
5	ID_ATTITUDE_Data_RAW	AttitudeData	Nodo_Recepcion_Attitude	AutoCmd-GET	Sensores RAW
6	ID_IMU_Data_RAW	IMUData	Nodo_Recepcion_IMU	AutoCmd-GET	Sensores RAW
7	ID_MAG_Data_RAW	MagData	Nodo_Recepcion_Mag	AutoCmd-GET	Sensores RAW
8	ID_GPS_Data_RAW	GPSData	FlightApp(GPS channel)	AutoCmd-GET	Sensores RAW
9	ID_BARO_Data_RAW	BaroData	Nodo_Recepcion_Baro	AutoCmd-GET	Sensores RAW
10	ID_ATTITUDE_Data	AttitudeData	Nodo_Recepcion_Attitude	ControlUniversal · AutoCmd	Sensores Alineados
11	ID_IMU_Data	IMUData	Nodo_Recepcion_IMU	ControlUniversal · AutoCmd	Sensores Alineados
12	ID_MAG_Data	MagData	Nodo_Recepcion_Mag	ControlUniversal · AutoCmd	Sensores Alineados
13	ID_GPS_Data	GPSData	FlightApp(GPS alineado)	ControlUniversal · AutoCmd	Sensores Alineados
14	ID_BARO_Data	BaroData	Nodo_Recepcion_Baro	ControlUniversal · AutoCmd	Sensores Alineados
15	ID_VEL_Data	VelocityData	ControlUniversal	AutoCmd-GET	Control Salidas
16	ID_YAW_Data	YawData	ControlUniversal	AutoCmd-GET	Control Salidas
19	ID_CTRL_LVL	uint8_t	AutoCmd-SET (GCS)	ControlUniversal	Control PID
20	ID_CTRL_LATVEL_IN	PidCtrlIn	AutoCmd-SET	ControlUniversal	Control PID
21	ID_CTRL_LATVEL_OUT	PidCtrlOut	ControlUniversal	AutoCmd-GET	Control PID
22	ID_CTRL_FRNTVEL_IN	PidCtrlIn	AutoCmd-SET	ControlUniversal	Control PID
23	ID_CTRL_FRNTVEL_OUT	PidCtrlOut	ControlUniversal	AutoCmd-GET	Control PID
24	ID_CTRL_VERTVEL_IN	PidCtrlIn	AutoCmd-SET	ControlUniversal	Control PID
25	ID_CTRL_VERTVEL_OUT	PidCtrlOut	ControlUniversal	AutoCmd-GET	Control PID
26	ID_CTRL_ANGVEL_IN	PidCtrlIn	AutoCmd-SET	ControlUniversal	Control PID
27	ID_CTRL_ANGVEL_OUT	PidCtrlOut	ControlUniversal	AutoCmd-GET	Control PID
28	ID_CTRL_LOOP_BYPASS	uint16_t	AutoCmd-SET	ControlUniversal	Control PID
29	ID_CTRL_HORPOS_EAST_OUT	PidCtrlOut	ControlUniversal	AutoCmd-GET	Posición Horizontal
30	ID_ROUTE_CMD	RouteData	AutoCmd-SET	ControlUniversal	Ruta
31	ID_ROUTE_STATUS	RouteStatus	ControlUniversal	AutoCmd-GET	Ruta
40	ID_CTRL_HORPOS_IN	PidCtrlIn	AutoCmd-SET	ControlUniversal	Posición Horizontal
41	ID_CTRL_HORPOS_OUT	PidCtrlOut	ControlUniversal	AutoCmd-GET	Posición Horizontal
42	ID_CTRL_VERTPOS_IN	PidCtrlIn	AutoCmd-SET	ControlUniversal	Posición Vertical
43	ID_CTRL_VERTPOS_OUT	PidCtrlOut	ControlUniversal	AutoCmd-GET	Posición Vertical
44	ID_CTRL_ANGPOS_IN	PidCtrlIn	AutoCmd-SET	ControlUniversal	Posición Angular
45	ID_CTRL_ANGPOS_OUT	PidCtrlOut	ControlUniversal	AutoCmd-GET	Posición Angular

6.5.2. Gestor de Comandos (CommandManager)

6.5.2.1. Anatomía de un ICommand

Todos los comandos del sistema implementan la interfaz `ICommand<T>`, que exige dos métodos: `execute(packet, reply)`, que procesa el paquete MSP entrante y construye la respuesta, y `getID()`, que devuelve el `CommandId` MSP al que responde ese comando. La clase `AutoCommand<T>`, generada por `AutoCommandFactory` para cada `DataID` registrado en el blackboard, implementa automáticamente los comandos GET y SET aprovechando los metadatos de `REGISTER_FP_STRUCT: TypeSignature<T>::Get()` proporciona la firma de tipos para la serialización, y `TypeFields<T>::Get()` proporciona los nombres de campo para la descripción del protocolo.

6.5.2.2. Procesamiento de paquetes MSP

`CommandManager<T>` mantiene un mapa de comandos registrados indexado por `CommandId`. Dispone de una cola de entrada a la que el `ChannelAggregator` encola los paquetes MSP recibidos junto con el identificador del canal de origen. El manager extrae paquetes de la cola, localiza el comando correspondiente al `CommandId`, lo ejecuta pasándole el paquete y un

método de respuesta que encola el resultado en el ChannelAggregator para que lo enrute al canal correcto. Si no existe comando registrado para el CommandId recibido, responde negativamente según la especificación del protocolo MSP.

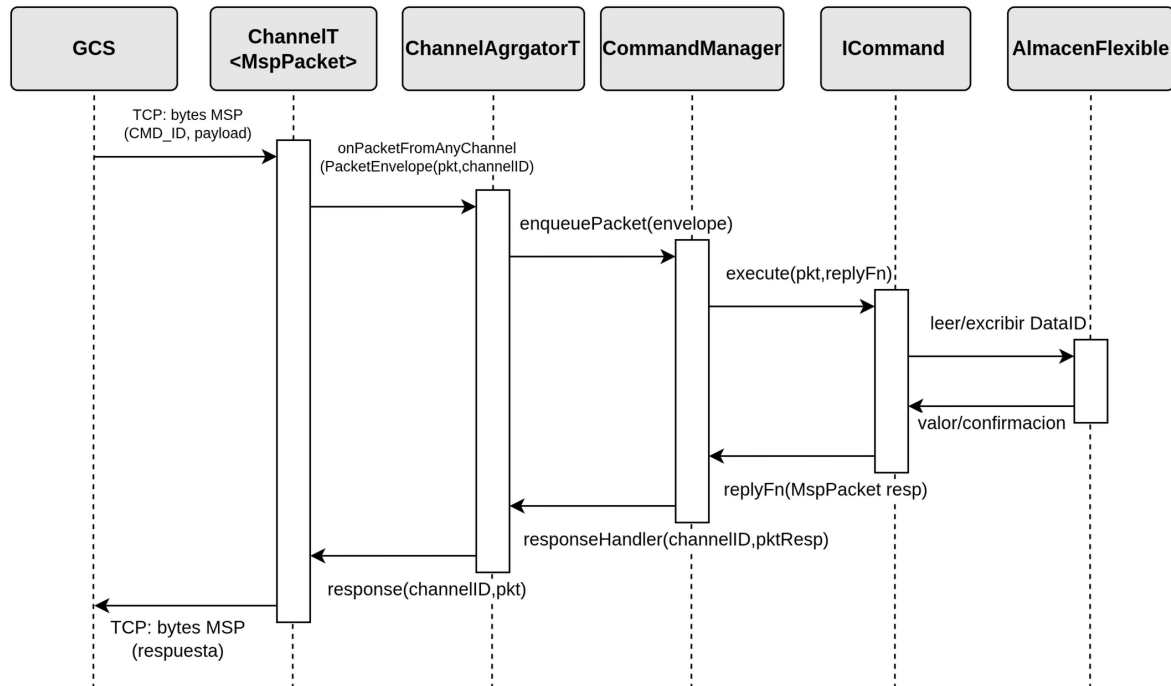


Figura 7. Flujo completo de un comando MSP desde la GCS hasta la respuesta

6.5.3. Gestor de Vuelo (ControlManager)

6.5.3.1. Ciclo de trabajo

ControlMaster es el orquestador del ciclo de control. Durante su inicialización (`init()`), registra todas las lambdas de lectura y escritura en el blackboard que los subsistemas de control necesitarán en operación. En ejecución, `eventLoop()` ejecuta el ciclo de control con el periodo configurado en el slot `ID_CTRL_LVL` del blackboard, lo que permite ajustar la frecuencia de muestreo desde la GCS sin reiniciar el sistema.

6.5.3.2. Leyes de control implementadas

El ciclo de control se articula en torno a `ControlStateMachin`, una máquina de estados con tres estados:

- **DESARMADO:** estado inicial y de reposo. El sistema acepta comandos de configuración pero no genera salidas de actuación activas hacia el BFC.

- **ARMADO_TIERRA:** sistema armado y en tierra. El ciclo de control está activo pero opera con limitaciones de seguridad (umbrales de actuación reducidos para evitar movimientos no deseados durante el armado).
- **ARMADO_VUELO:** sistema en vuelo con control de plena autoridad. Todos los bucles PID operan sin restricciones adicionales.

Máquina de Estados — ControlStateMachine

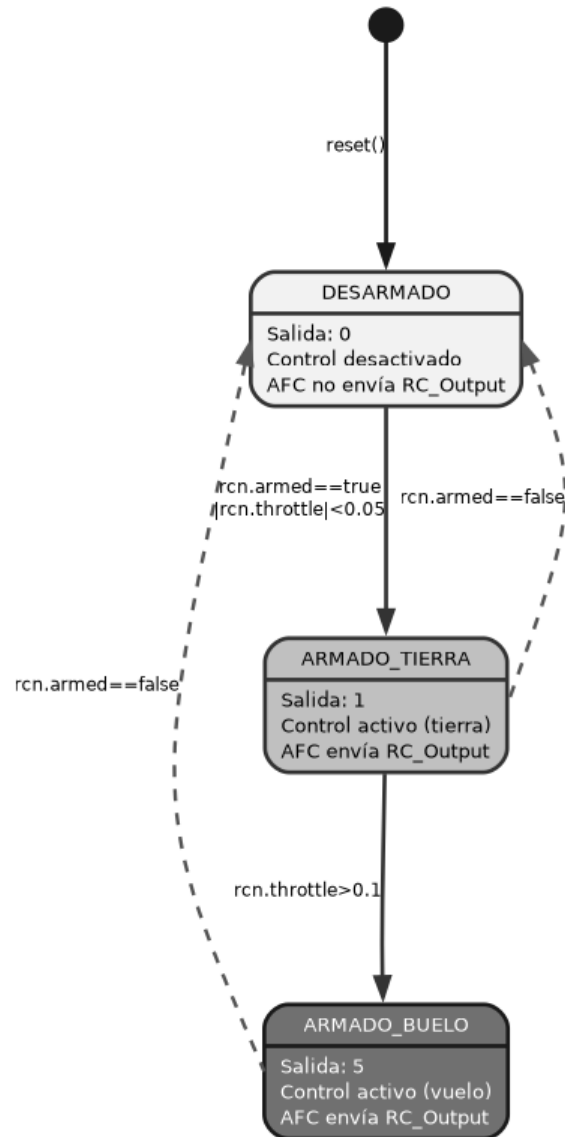


Figura 8. Máquina de estados de ControlStateMachin

ControlRemoteNorm normaliza las referencias RC crudas del slot ID_RC_Input — valores enteros en el rango [1000, 2000] procedentes del protocolo IBus— al rango continuo

$[-1, 1]$ como `RCNORMData`, depositándolas en `ID_RC_InputNorm`. Esta normalización desacopla los valores absolutos del protocolo de radiocontrol del sistema de control, que trabaja en términos adimensionales e independientes del hardware de mando.

`ControlUniversal` implementa el controlador multivariable principal mediante cinco bucles PID en cascada. Los bucles de posición generan referencias de velocidad, que son las consignas de los bucles de velocidad correspondientes:

1. **Posición vertical** (`ID_CTRL_VERTPOS_IN/OUT`, IDs 42/43): control de altitud a partir de los datos del barómetro (`ID_BARO_Data`). La salida actúa como referencia de velocidad vertical.
2. **Velocidad vertical** (`ID_CTRL_VERTVEL_IN/OUT`, IDs 24/25): control de velocidad vertical, con referencia procedente del bucle de posición y realimentación de `ID_BARO_Data`.
3. **Velocidad lateral** (`ID_CTRL_LATVEL_IN/OUT`, IDs 20/21): control de velocidad en el eje lateral del marco de vuelo, con realimentación de actitud e IMU.
4. **Velocidad frontal** (`ID_CTRL_FRNTVEL_IN/OUT`, IDs 22/23): control de velocidad en el eje longitudinal, con realimentación de actitud e IMU.
5. **Velocidad angular (yaw rate)** (`ID_CTRL_ANGVEL_IN/OUT`, IDs 26/27): control de la velocidad de rotación en guiñada, con realimentación de `ID_MAG_Data` e `ID_ATTITUDE_Data`.
6. **Posición horizontal** (`ID_CTRL_HORPOS_IN/OUT`, IDs 40/41 norte, 29 este): control de posición horizontal a partir de `ID_GPS_Data`. Comparte un único juego de ganancias entre los ejes norte y este —la física de la planta es simétrica entre ambos— aunque publica la salida de cada eje por separado. Su salida actúa como referencia de los bucles de velocidad lateral y frontal.
7. **Posición angular** (`ID_CTRL_ANGPOS_IN/OUT`, IDs 44/45): control de orientación (guiñada) a partir de `ID_ATTITUDE_Data`. Su salida actúa como referencia del bucle de velocidad angular.

Los parámetros de cada bucle (ganancias proporcional, integral, derivativa y offset) se configuran en tiempo de ejecución escribiendo los slots `ID_CTRL_*_IN` desde la GCS, lo que habilita la sintonización del control sin necesidad de recompilar ni reflashear el firmware.

6.5.3.3. Integración con el Blackboard

En cada iteración del ciclo de control, `ControlMaster` lee del blackboard todos los datos de telemetría alineada que necesita (`ID_ATTITUDE_Data`, `ID_IMU_Data`, `ID_MAG_Data`, `ID_BARO_Data`, `ID_GPS_Data`) y las referencias de entrada normalizadas (`ID_RC_InputNorm`). Tras ejecutar `ControlStateMachin`, `ControlRemoteNorm`

y ControlUniversal, deposita los resultados en los slots de salida (ID_RC_Output, ID_VEL_Data, ID_YAW_Data y los slots _OUT de cada bucle PID). Este flujo de lectura y escritura exclusivamente a través de lambdas del blackboard garantiza el aislamiento total del sistema de control respecto al resto del firmware.

6.5.3.4. Navegación por ruta

Sobre los lazos de posición se añade la posibilidad de seguir una ruta de varios waypoints (posición norte/este/altura y rumbo), enviada de una vez desde la GCS. El AFC persigue cada waypoint con los lazos de posición anteriores y avanza automáticamente al siguiente en cuanto lo alcanza, sin ningún mecanismo de planificación adicional. Si la ruta recibida cambia respecto a la última procesada, el progreso se reinicia automáticamente, igual que ocurre al entrar en este nivel de control. Los comandos de ruta (ID_ROUTE_CMD/ID_ROUTE_STATUS) se registran con el mismo mecanismo genérico de comandos autodescriptivos ya presentado en este capítulo.

6.5.4. Gestor de Nodos de Datos (DataNodesManager)

6.5.4.1. Anatomía de un DataNode

Los DataNodes son los puentes entre el blackboard y los periféricos externos (sensores y BFC). Todos implementan IDataNodeBase, que exige un único método `transact()`. El flujo es bidireccional: los nodos de recepción envían una petición MSP o I2C por su canal, esperan la respuesta, aplican la alineación hardware mediante `IHwAlignment` y depositan el dato alineado en el blackboard; el nodo de emisión realiza el proceso inverso.

6.5.4.2. Nodos implementados

Los seis DataNodes activos son:

- `Nodo_Recepcion_Attitude`: solicita datos de actitud al BFC vía MSP y los deposita en `ID_ATTITUDE_Data_RAW (raw, ID 5)` e `ID_ATTITUDE_Data (alineado, ID 10)`.
- `Nodo_Recepcion_IMU`: solicita datos de la IMU al BFC vía MSP y los deposita en `ID_IMU_Data_RAW (6) / ID_IMU_Data (11)`.
- `Nodo_Recepcion_Mag`: solicita datos del magnetómetro vía I2C y los deposita en `ID_MAG_Data_RAW (7) / ID_MAG_Data (12)`.
- `Nodo_Recepcion_Baro`: solicita datos del barómetro vía I2C y los deposita en `ID_BARO_Data_RAW (9) / ID_BARO_Data (14)`.
- `Nodo_Recepcion_Status`: solicita el estado del BFC vía MSP y lo deposita en `ID_STATUS_Data (0)`.
- `Nodo_Emision_RC`: extrae `ID_RC_Output (2)` del blackboard y lo transmite al BFC en formato MSP, cerrando el lazo de control.

6.5.4.3. Orquestación y temporización

`DataNodesManager` orquesta estos nodos mediante tres métodos: `addNode(handler, periodo)` registra un nodo con su periodo de muestreo deseado; `start()` arranca el scheduler interno que invoca `transact()` de cada nodo cuando le corresponde según su periodo; `stop()` detiene el scheduler. La interfaz `IHwAlignment` (implementada actualmente por `SimuladorAlignment`) aplica las correcciones de orientación física del sensor —rotaciones del sistema de referencia, inversiones de eje— antes de que el dato sea consumido por el sistema de control. `RealAlignment.h` existe en el repositorio pero está vacío, pendiente de caracterización del hardware real.

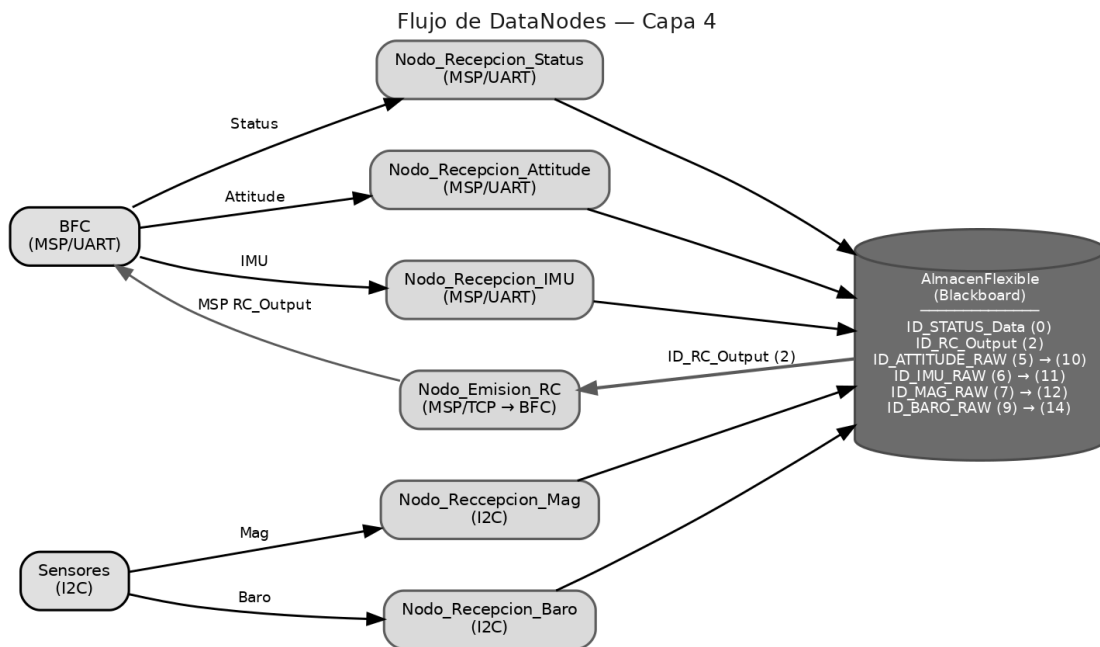


Figura 9. DataNodes activos con sus canales y su relación con el blackboard

6.6. Capa 5 — Ensamblaje

6.6.1. FlightApplication

`FlightApplication` es el único lugar del código donde coexisten instancias de todas las capas. Su responsabilidad es inyectar dependencias, conectar callbacks y mantener vivos los punteros a todos los subsistemas mientras la aplicación está en ejecución. No contiene lógica de negocio: es una función de composición.

Los miembros privados de la clase reflejan directamente la topología de canales del sistema:

```

1 // Extracto de: main/FlightApplication.h
2
3 private:
4     // Canal de entrada TCP (MSP desde GCS) -- puerto :12345
5     int tcpPortInput = 12345;
6     std::shared_ptr<FlightProxy::Channel::ChannelServer<Packet>>
7         channelServerTCP_in;
8     std::shared_ptr<FlightProxy::Channel::ChannelAgregatorT<Packet>>
9         channelAgregatorTCP_in;
10    std::shared_ptr<FlightProxy::AppLogic::Command::CommandManager<Packet>>
11        commandManager;
12
13    // Canal de entrada UDP (IBus RC) -- puerto :12346
14    int udpPortInput = 12346;
15    std::shared_ptr<FlightProxy::Channel::ChannelT<FlightProxy::Core::IBUSPacket>>
16        channelUDP_in;
17
18    // Canal de salida TCP al BFC (MSP) -- puerto :5762
19    std::string droneIp = "127.0.0.1";
20    int dronePort = 5762;
21    std::shared_ptr<FlightProxy::Channel::ChannelDisgregatorT<Packet>>
22        channelDisgregatorTCP_out;
23
24    // Canal I2C (sensores Mag/Baro) -- puerto :5800
25    int droneI2CPort = 5800;
26    std::shared_ptr<FlightProxy::Channel::ChannelDisgregatorT<FlightProxy::Core::I2CPacket>>
27        channelDisgregatorI2C;
28
29    // Canal UART (GPS) -- puerto :5801
30    int droneGpsPort = 5801;
31    std::shared_ptr<FlightProxy::Channel::ChannelT<FlightProxy::Core::GPSData>>
32        channelUart;
33
34    // Subsistemas internos (Capa 4 -- AppLogic)
35    std::shared_ptr<FlightProxy::AppLogic::AlmacenFlexible>                blackboard;
36    std::shared_ptr<FlightProxy::AppLogic::Control::ControlMaster>        controlMaster_;
37    std::shared_ptr<FlightProxy::AppLogic::DataNode::DataNodesManager>    dataNodesManager;
38    std::shared_ptr<FlightProxy::AppLogic::DataNode::HwAlignment::SimuladorAlignment>
39        alineador_;

```

Código 3. Miembros privados de FlightApplication — topología de canales del sistema

6.6.2. Inyección de dependencias y arranque

La inicialización se realiza a través de nueve métodos de configuración especializados: `setupNetwork()` instancia las factories de plataforma y configura las credenciales de red; `setupTCPchannel_in()` construye el `ChannelServer` y el `ChannelAgregator` de entrada MSP; `setupUDPchannel_in()` construye el canal IBus de referencia RC y lo conecta directamente al productor `ID_RC_Input` del blackboard; `setupTCPchannel_out()` construye el `ChannelDisgregator` de salida hacia el BFC; `setupI2Cchannel_in()` construye el canal I2C para magnetómetro y barómetro; `setupGPSchannel_in()` construye el canal GPS UART; `setupCommandSystem()` instancia el `CommandManager` y registra todos los comandos generados por `AutoCommandFactory`; `setupDataNodes()` instancia los seis `DataNodes` y los registra en el `DataNodesManager` con sus periodos de muestreo; y `setupControlSystem()` instancia el `ControlMaster` y lo inicializa con el blackboard.

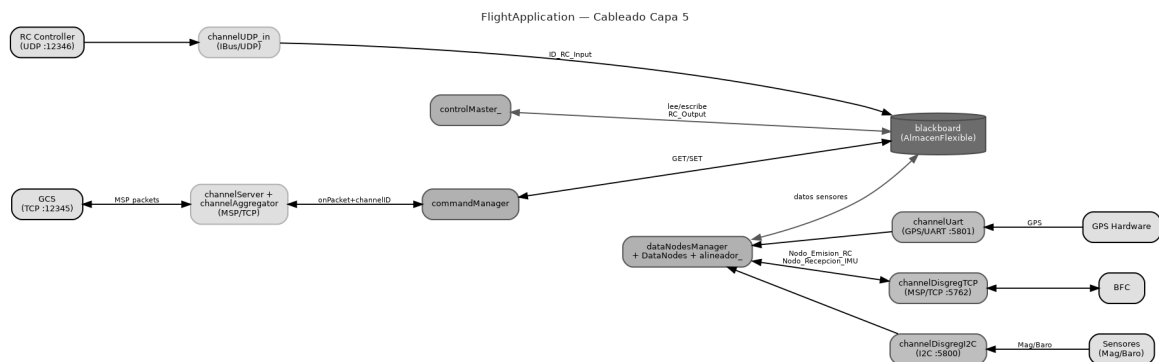


Figura 10. Diagrama de wiring de FlightApplication: canales, blackboard, managers y conexiones de callbacks

6.7. Flujos de datos principales

6.7.1. Entrada y ejecución de comandos desde la GCS

El flujo de un comando MSP desde la GCS hasta la respuesta sigue siete pasos:

1. La GCS abre una conexión TCP al puerto 12345 del AFC.
2. El `ITcpListener`, que está escuchando en ese puerto, detecta la conexión entrante y genera un nuevo objeto `ITransport`.
3. El `ChannelServer<Packet>` recoge ese transporte, instancia un encoder y un decoder MSP v2 y ensambla un `ChannelT<Packet>` completo.
4. El nuevo canal se entrega al `ChannelAgregatorT<Packet>`, que le asigna un identificador único y registra en el callback `onPacket` del canal el método de encolado del `CommandManager` junto con ese identificador.
5. La GCS envía un paquete MSP. El decoder del canal lo decodifica y activa el callback, encolando el paquete y el identificador de canal en la cola del `CommandManager`.
6. El `CommandManager` extrae el paquete de la cola, localiza el `ICommand` registrado para ese `CommandId` y llama a `execute()`, que interactúa con el blackboard (lectura o escritura según el tipo de comando). El comando encola la respuesta en el `ChannelAgregator` junto con el identificador del canal de origen.
7. El `ChannelAgregator` enruta la respuesta por el canal correcto de vuelta a la GCS.

6.7.2. Ingesta continua de referencias (IBus/UDP)

De forma paralela al canal de comandos, la GCS transmite continuamente referencias de vuelo —roll, pitch, throttle, yaw— mediante UDP en formato IBus al puerto 12346. El `TransportUDP` de ese puerto recibe los datagramas y los pasa al `ChannelT<IBUSPacket>` correspondiente. El canal decodifica cada trama mediante el `IbusProtocol`, y el callback `onPacket` tiene registrado directamente el setter del slot `ID_RC_Input` del blackboard, de forma que cada trama válida actualiza las referencias de vuelo sin intermediarios adicionales. El sistema de control, en su siguiente iteración, encontrará el dato más reciente disponible.

6.7.3. Procesamiento interno del control

El `ControlMaster` ejecuta `eventLoop()` en su propia tarea OSAL (`ITask`). En cada iteración del ciclo: (1) lee del blackboard todos los datos de telemetría alineada que necesita y las referencias de entrada normalizadas; (2) pasa esos datos a `ControlStateMachin`, que determina el estado activo y activa o inhibe el control; (3) si el estado es `ARMADO_TIERRA` o `ARMADO_VUELO`, ejecuta `ControlRemoteNorm` para normalizar las referencias crudas, y a continuación `ControlUniversal`, que calcula las salidas de los cinco bucles PID en cascada; (4) deposita los resultados en los slots de salida del blackboard (`ID_RC_Output`, `ID_VEL_Data`, `ID_YAW_Data` y los slots `_OUT` de cada bucle PID).

6.7.4. Envío de referencias al BFC

El `Nodo_Emision_RC`, gestionado por el `DataNodesManager` con su periodo de muestreo configurado, ejecuta su `transact()` periódicamente: (1) lee `ID_RC_Output` del blackboard, obteniendo las referencias de actuación calculadas por el sistema de control; (2) las serializa en un paquete MSP mediante `sendPacket()` en su canal virtual lógico; (3) el paquete llega al `ChannelDisgregatorT<Packet>` de salida; (4) el discriminador —el `CommandId` del paquete— determina el canal lógico de destino dentro del canal físico compartido con el BFC; (5) el dato sale por el transporte subyacente (UART en hardware real, TCP en simulación) hacia el BFC para que ejecute el movimiento.

6.8. Portabilidad y entorno de simulación

La arquitectura de cinco capas permite compilar el firmware AFC sin modificar ningún archivo fuera de la Capa 1. El mecanismo opera a nivel de CMake: al construir el proyecto, la variable de target determina qué componente de plataforma se enlaza. Para el target ESP32 se compila `PlatformESP32`, que contiene `FreeRTOSMutex`, `FreeRTOSQueue`, `FreeRTOSTask` y los transportes basados en `esp-idf`. Para el target Linux se compila `PlatformLinux`, que contiene `LinuxMutex`, `LinuxQueue`, `LinuxTask` y los transportes basados en `sockets` POSIX.

Las capas Core, Channel y AppLogic (Capas 2, 3 y 4) se compilan íntegramente en ambos casos, sin ningún `#ifdef` que altere su comportamiento. La Capa 5 (FlightApplication) también es completamente común: instancia los objetos a través de las factories de plataforma, que ya están especializadas en tiempo de enlace.

Este diseño habilita el entorno SITL descrito en el capítulo de metodología: el AFC corre en Linux, se conecta por TCP al SITL (Betaflight simulado en PC) exactamente como se conectaría por UART al BFC real, y recibe los datos de los sensores del módulo ESS en lugar del hardware físico. La GCS interactúa de la misma manera en ambos entornos. En el modo HIL, únicamente el ejecutable AFC es reemplazado por el binario compilado para ESP32 ejecutándose sobre la placa real; el resto del entorno de simulación permanece idéntico, lo que permite validar que el MCU físico dispone de los recursos de procesamiento y memoria necesarios bajo carga real.

Compilación Dual — Portabilidad del Firmware AFC

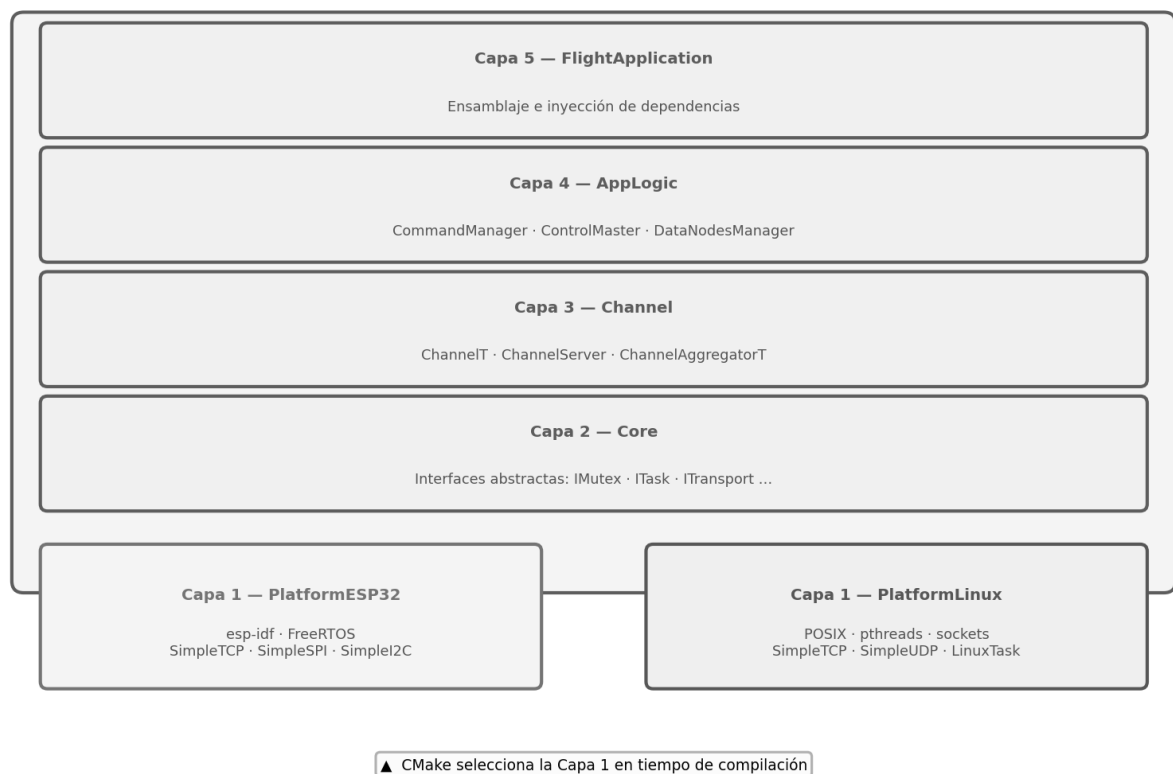


Figura 11. Compilación dual ESP32/Linux: capas compartidas y capa Platform intercambiable

Capítulo 7

Entorno de Simulación

El entorno de simulación es la infraestructura *software* que permite iterar, validar y depurar el *firmware* del AFC sin depender del hardware físico del UAV. La capacidad de que dicho *firmware* se ejecute de forma transparente tanto en el microcontrolador ESP32 como en un proceso Linux nativo es consecuencia directa de las abstracciones de transporte (§6.2.1) y del *Operating System Abstraction Layer* — OSAL — (§6.2.2) introducidos en el Capítulo 6.

El entorno se articula en cuatro bloques *software* que se comunican mediante *sockets* internos: el motor de física **Gazebo Garden**, la instancia **Betaflight SITL** que simula la controladora de vuelo básica, la **ESS** (*External Sensors Simulation*) que emula los sensores periféricos del AFC, y el propio **firmware AFC compilado para Linux**. La **GCS** se conecta al entorno de la misma forma que lo haría con el sistema real. La Figura 12 muestra la disposición completa del sistema y los protocolos y puertos de cada enlace.

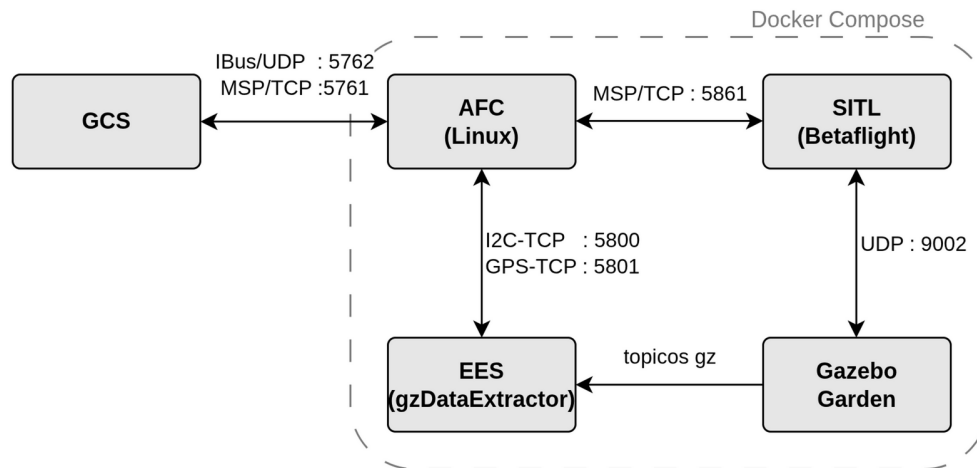


Figura 12. Arquitectura del entorno de simulación completo. Los cuatro bloques internos (AFC en Linux, SITL, Gazebo y ESS) se ejecutan en el mismo host; los dos servicios contenerizados (SITL y gateway) comparten *namespace* de red para que `127.0.0.1` sea común. La GCS se conecta desde el exterior por los puertos publicados por el contenedor *gateway*.

7.1. Motivación y objetivos del entorno de simulación

La decisión de construir un entorno SITL (*Software in the Loop*) propio en lugar de validar el *firmware* exclusivamente sobre el hardware físico responde a tres razones.

Reducción del coste y el tiempo de iteración. Cada sesión de vuelo con el UAV físico implica transportar el equipo, montar los componentes, verificar el estado de las baterías y asumir el riesgo de dañar el hardware si el *firmware* contiene errores. En simulación, ese mismo ciclo se reduce a relanzar el proceso del AFC, en menos de diez segundos.

Reproducibilidad de las condiciones de ensayo. El simulador permite repetir exactamente las mismas condiciones de vuelo — posición inicial, referencia de entrada, estado de los sensores — para comparar versiones del controlador o parametrizaciones PID. Los ensayos reales están sujetos a variaciones de viento, temperatura y batería que dificultan esa comparación.

Verificación de la capacidad del MCU antes del primer vuelo (HIL). El modo *Hardware in the Loop* (HIL), descrito en la Sección 7.6, conecta la placa ESP32 física al entorno de simulación de PC para medir la carga de procesamiento y el consumo de memoria del microcontrolador antes de arriesgar el hardware en vuelo. Corresponde a uno de los objetivos explícitos del TFM: «*HIL (hardware in the loop): poder conectar la placa física al ciclo de simulación para comprobar que el procesador y la memoria del MCU aguantan de verdad*».

Estos tres objetivos son posibles gracias a la arquitectura de capas del AFC: el OSAL abstrae FreeRTOS y Linux como sistemas operativos intercambiables, las abstracciones de transporte reemplazan buses físicos por *sockets* TCP sin que el código de aplicación lo perciba, y las *factories* de plataforma (§6.2.3) seleccionan en tiempo de compilación qué implementaciones concretas se usan.

7.2. Visión de conjunto de la arquitectura de simulación

El entorno está formado por cuatro bloques funcionales, cuya relación se ilustra en la Figura 12 y se detalla en la Tabla 6. Todos ellos se comunican mediante *sockets* de red que reproducen los mismos protocolos que existen en el UAV físico.

Tabla 6. Bloques del entorno de simulación, su rol y los protocolos de comunicación.

Bloque	Simula	Puerto / protocolo	Descripción
Gazebo Garden	Powertrain (motores, hélices, <i>frame</i>)	UDP 9002 (comandos motor)	Motor de física 3D. Recibe velocidades de motor del SITL y publica tópicos de sensores (GPS, magnetómetro, presión, IMU).
Betaflight SITL	BFC (MAMBA F405MINIMK2)	TCP 5761/5762 (MSP)	Instancia de Betaflight 4.5.1 compilada como binario Linux nativo. Ejecuta los algoritmos de estabilización reales y se comunica con Gazebo por UDP.
ESS (<i>gzDataExtractor</i>)	Sensores externos (BMP390, QMC5883L, WS-M181)	TCP 5800 (I2C virtual), TCP 5801 (GPS NMEA)	Lee tópicos de Ga- zebo y expone inter- faces TCP que imi- tan el bus I2C y la UART GPS que el AFC encontraría en el hardware real.
AFC en Linux	Firmware AFC (ESP32)	Todos los anteriores como cliente	El mismo código fuente del firmware, compilado con <code>PlatformLinux</code> en lugar de <code>PlatformESP32</code> . Se conecta a todos los servicios anteriores por TCP/UDP exactamente como el ESP32 se conecta al hardware físico.

La característica más importante del diseño es que el AFC no requiere ninguna rama de código especial para funcionar en simulación. Los únicos valores que cambian son los parámetros

de configuración de los transportes en `FlightApplication` (Capa 5): en el ESP32 apuntan a periféricos del SoC; en Linux, a las direcciones IP y puertos de la Tabla 6. El flujo de datos *end-to-end* completo se desarrolla en la Sección 7.7.

7.3. Componentes del entorno de simulación

De los cuatro bloques de la Tabla 6, Gazebo Garden y Betaflight SITL son herramientas preexistentes de terceros que se integran mediante configuración; la ESS es código Python desarrollado íntegramente para este trabajo y es, por tanto, el componente que se describe con más detalle.

7.3.1. Motor de física — Gazebo Garden

Gazebo Garden¹ se eligió por la existencia del repositorio `vehicle_gateway` de la OSRF, que proporciona de forma integrada el *plugin* `betaflight_gazebo` — responsable de la comunicación entre el SITL de Betaflight y el motor de física — junto con modelos de referencia de UAV. Gazebo Garden se integra con ROS 2 Humble, base del contenedor *gateway* (§7.5).

El mundo virtual usado en la simulación está definido en `betaflight_iris_aruco.sdf` (`tools/simulador/` del repositorio). Sus parámetros clave son: un paso de integración de $\Delta t = 1$ ms y un factor de tiempo real $r = 0,5$, determinado empíricamente como el umbral mínimo de estabilidad para el par Betaflight 4.5.1 + Gazebo Garden — valores más altos no dejan al SITL tiempo suficiente para procesar la IMU entre pasos de simulación; valores más bajos ralentizan en exceso el ensayo. El origen del mundo se fija en las coordenadas geodésicas de Madrid (modelo EARTH_WGS84), de modo que el *plugin* NavSat publica coordenadas GPS reales y las herramientas de visualización de la GCS funcionan sin adaptación. La Tabla 7 lista los nueve *plugins* del sistema activados; los cinco últimos corresponden a los sensores físicos del UAV.

¹Denominado también *Gazebo 7* o *gz-garden* según el contexto de empaquetado.

Tabla 7. *Plugins* del sistema Gazebo Garden habilitados en el mundo de simulación.

Plugin	Función
gz-sim-physics-system	Motor de física (integración ODE/DART)
gz-sim-user-commands-system	API de comandos externos (p.ej. set_pose)
gz-sim-scene-broadcaster-system	Difusión de escena al cliente GUI
gz-sim-sensors-system (ogre2)	Gestión del pipeline de sensores (render ogre2)
gz-sim-imu-system	Sensor IMU (acelerómetro + giróscopo)
gz-sim-air-speed-system	Sensor de velocidad aerodinámica
gz-sim-air-pressure-system	Sensor de presión atmosférica (barómetro)
gz-sim-navsat-system	Sensor GPS (NavSat)
gz-sim-magnetometer-system	Sensor magnetómetro

El modelo del dron es una variante del *iris* cuadricóptero de referencia del ecosistema PX4/Gazebo, modificado con soportes (*standoffs*) para el módulo AFC. Sus parámetros dinámicos son masa $m = 1,5 \text{ kg}$ y momentos de inercia $I_{xx} = 0,008 \text{ kg m}^2$, $I_{yy} = 0,015 \text{ kg m}^2$, $I_{zz} = 0,017 \text{ kg m}^2$, correspondientes a un cuadricóptero con distancia entre ejes de aproximadamente 470 mm; son el punto de partida para el procedimiento de identificación de planta de la GCS (§8). El *plugin* BetaFlightPlugin insertado en el modelo escucha en el puerto UDP 9002 los comandos de velocidad angular de los cuatro rotores, aplica los coeficientes de sustentación y arrastre, y retroalimenta los datos de la IMU al SITL.

La ESS consume tres tópicos publicados por Gazebo (Tabla 8); la IMU no aparece porque la consume directamente el SITL por UDP, sin intermediación de la ESS.

Tabla 8. Tópicos de Gazebo Garden consumidos por la ESS.

Tópico Gazebo	Campos leídos	Uso en la ESS
/gps	latitude_deg, longitude_deg, velocity_east, velocity_north	Generación de tramas NMEA \$GPRMC en <code>simulador_gps.py</code>
/world/.../air_pressure_sensor/ air_pressure ^a	pressure	Emulación del barómetro BMP390 en <code>I2CSim.py</code>
/mag	x, y, z	Emulación del magnetómetro QMC5883L en <code>I2CSim.py</code>

^a Ruta completa: `/world/empty_betaflight_world/model/iris_with_Betaflight/model/iris_with_standoffs/link/imu_link/sensor/air_pressure_sensor/air_pressure`

7.3.2. Simulación del controlador de vuelo — Betaflight SITL

Betaflight SITL es una funcionalidad oficial del proyecto Betaflight que compila su base de código como ejecutable nativo Linux x86_64 en lugar de un binario para microcontroladores ARM. El binario resultante, `betaflight_SITL.elf`, implementa los mismos algoritmos de estabilización — mixer de cuadricóptero y controladores PID — que el *firmware* de la BFC física (MAMBA F405MINIMK2). Desde el punto de vista del AFC, el SITL es indistinguible de la BFC real: se comunica por el protocolo MSP sobre la interfaz `ITransport` (§6.2.1), que en el sistema real usa UART física y en simulación TCP sobre *loopback*, sin que el código de aplicación perciba el cambio.

El proceso de construcción está encapsulado en `Dockerfile.sitl` (`tools/simulador/` del repositorio). El paso más relevante es fijar la versión 4.5.1 de Betaflight, idéntica a la que ejecuta la BFC física: cualquier discrepancia de comportamiento entre simulación y vuelo real puede así atribuirse a la dinámica simulada o a la latencia de comunicación, no al código del controlador. La cadena de herramientas ARM (`arm_sdk_install`) es necesaria aunque el objetivo sea x86, porque los *scripts* de configuración de Betaflight presuponen su presencia en el entorno de *build*.

El arranque del binario dentro del contenedor lo gestiona `start_sitl.sh` (`tools/simulador/` del repositorio): espera un periodo de gracia de diez segundos para que Gazebo abra el puerto UDP 9002, lanza `betaflight_SITL.elf` apuntando a `127.0.0.1:9002`, y mantiene el contenedor vivo. Esa dirección de *loopback* es alcanzable porque el contenedor SITL comparte el *namespace* de red del contenedor *gateway* (`network_mode: service:gateway`, véase §7.5.1).

7.3.3. Emulación de sensores externos — ESS (*gzDataExtractor*)

La ESS hace de puente entre los tópicos que publica Gazebo y las interfaces de hardware que el *firmware* del AFC espera encontrar. Gracias a este puente, los *drivers* de Capa 1 (§6.2) — *SimpleI2C* y *SimpleUart* — usan sus versiones Linux, que implementan *ITransport* sobre *sockets* TCP en lugar de sobre periféricos hardware, sin ninguna rama de código especial para simulación.

Extracción de tópicos. La clase *GazeboTopicLector* (*gztopicextractor.py*, *tools/simulador/gzDataExtractor/* del repositorio) da a los emuladores de sensor un acceso uniforme a cualquier tópico de Gazebo: lanza el subprocesso `gz topicet <topic>` y parsea su salida línea a línea en un hilo *daemon*, actualizando un diccionario de valores actuales que las lecturas del hilo principal consultan de forma segura gracias al GIL de CPython.

Emulador GPS. El módulo *simulador_gps.py* expone un servidor TCP en el puerto 5801 que emula el flujo serie NMEA del módulo GPS WS-M181; el AFC abre una conexión TCP a ese puerto en lugar de la UART física, y el protocolo (*GPSTDecoder* de Capa 2) es idéntico en ambos casos. Cada segundo, el servidor lee la posición del *GazeboTopicLector* y construye una trama *\$GPRMC* (*simulador_gps.py*, *tools/simulador/gzDataExtractor/* del repositorio): convierte latitud/longitud de grados decimales al formato DDMM.MMMM, el vector de velocidad 2D a nudos, calcula el curso verdadero con $\arctan 2$ y añade el *checksum* XOR exigido por el estándar NMEA 0183.

Emulador I2C virtual. El módulo *I2CSim.py* expone en el puerto 5800 un servidor TCP que emula un bus I2C con dos dispositivos: el barómetro BMP390 (0x77) y el magnetómetro QMC5883L (0x0D). Cada transacción usa el protocolo de trama de la Figura 13: cuatro bytes de cabecera — dirección de dispositivo, dirección de registro, flag de lectura/escritura y longitud de *payload* — seguidos del *payload* si la operación es de escritura.



Figura 13. Estructura de trama del protocolo I2C virtual sobre TCP implementado en *I2CSim.py*. La cabecera de 4 bytes es suficiente para identificar el dispositivo, el registro y el tipo de operación. El servidor responde con la misma cabecera más el *payload* de datos del sensor.

El AFC accede a tres registros del BMP390: 0x31 devuelve 21 bytes de coeficientes de calibración *hardcodeados* de un sensor real, sin los cuales la compensación de presión que aplica el *driver* produciría valores erróneos; 0x04 devuelve la presión leída de Gazebo empaquetada en 24 bits, con la temperatura fijada en un valor constante equivalente a $\sim 25^\circ\text{C}$ — simplificación suficiente para la estimación de altitud del AFC; 0x1B acepta el comando

de modo normal sin acción real, solo para que la inicialización del *driver* no falle. Del QMC5883L se emulan el registro 0x00 (ejes X/Y/Z leídos del tópic `/mag` y escalados por un factor 3000,0 de Tesla a unidades ADC) y los registros de configuración 0x0B/0x09, aceptados sin acción real. El código completo de ambos emuladores (`I2CSim.py`) está en `tools/simulador/gzDataExtractor/` del repositorio.

7.4. Integración del firmware AFC en el entorno de simulación

La integración del firmware AFC en el entorno de simulación no requiere ninguna modificación del código de aplicación respecto al firmware que corre en el ESP32. Esta transparencia es consecuencia directa de la arquitectura de capas del Capítulo 6.

En tiempo de compilación, la directiva `CMake` selecciona la plataforma objetivo: si es **ESP32**, `TransportFactory` y `OSALFactory` instancian implementaciones de `FreeRTOS` y `esp-idf` (`FreeRTOSTask`, `FreeRTOSMutex`, `SimpleI2C`, `SimpleUart`); si es **Linux**, las mismas *factories* instancian `LinuxTask`, `LinuxMutex`, `SimpleTCP/SimpleUDP` y `SimpleUart` sobre *sockets*. No existe `SimpleI2C` en Linux: la Capa 1 Linux implementa el acceso al bus I2C como un *socket* TCP cliente al servidor de la ESS.

En la Capa 5 (`FlightApplication`), los parámetros de los transportes son los únicos valores que difieren entre los dos *setups*:

Canal	Setup real (ESP32)	Setup simulación (Linux)
BFC / SITL (MSP)	UART físico (pines GPIO)	TCP 127.0.0.1:5761
Bus I2C (sensores)	I2C hardware (bus 0 ESP32)	TCP 127.0.0.1:5800
GPS (UART NMEA)	UART hardware (pines GPIO)	TCP 127.0.0.1:5801
GCS (IBus/UDP)	WiFi / SIM800L (UDP)	UDP 0.0.0.0:5762
GCS (MSP/TCP)	WiFi / SIM800L (TCP)	TCP listener : 5761

El código de las Capas 2–5 del AFC — `Core`, `Channel`, `AppLogic` y `FlightApplication` — se compila sin ningún cambio: interactúan exclusivamente con las interfaces `ITransport`, `IMutex`, `ITask` e `IQueue`, cuya implementación concreta les es transparente. Esta selección de implementación en tiempo de compilación es la misma mostrada en la Figura 11 del Capítulo 6: aquí se aplica al caso concreto de los transportes de la Tabla anterior en lugar de a las interfaces genéricas del `Core`.

7.5. Orquestación con Docker Compose

El entorno se gestiona mediante `Docker Compose`, que organiza los dos servicios contenerizados (*gateway* y *sitl*) como una unidad desplegable y reproducible. El fichero `docker-compose.yml` completo está en `tools/simulador/` del repositorio.

7.5.1. Servicios, red y dependencias

Servicio *gateway* ejecuta tres procesos: los dos servidores de la ESS (`simulador_gps.py` e `I2CSim.py`), lanzados en segundo plano, y el proceso principal de Gazebo Garden (`gz sim -v 4 -s -r ...betaflight_iris_aruco.sdf`), donde `-s` indica que corre solo como servidor de física, sin GUI integrada —la GUI puede conectarse de forma independiente, véase §7.5.2. El servicio requiere acceso a GPU NVIDIA para el renderizado de sensores virtuales en sistemas con GPU híbrida Intel+NVIDIA. Publica al sistema anfitrión los puertos 5761/5762 (MSP AFC–SITL), 5800 (I2C virtual) y 5801 (GPS NMEA).

La imagen parte de `osrf/ros:humble-desktop` (ROS 2 Humble sobre Ubuntu 22.04). El `Dockerfile.gateway` instala Gazebo Garden desde el repositorio APT de la OSRF, clona y compila con `colcon` los paquetes de `vehicle_gateway` necesarios — entre ellos `betaflight_gazebo`, que aporta el `BetaFlightPlugin` —, copia el mundo personalizado al directorio de mundos instalado, y genera un *entrypoint* que arranca los dos servidores de la ESS antes de ejecutar el comando de `docker-compose`, garantizando que estén listos antes de que Gazebo empiece a publicar tópicos.

Servicio *sitl* ejecuta únicamente `start_sitl.sh` (§7.3.2). Comparte el *namespace* de red del servicio *gateway* mediante `network_mode: service:gateway`, de forma que `127.0.0.1` resuelve al mismo *loopback* en ambos contenedores — mecanismo esencial para que el SITL alcance el puerto UDP 9002 de Gazebo sin pasar por la red *bridge* de Docker. El directorio `./bf_data` se monta para persistir la EEPROM del SITL (tasas PID, calibraciones) entre reinicios del contenedor.

Cuando el AFC se ejecuta en Linux en el mismo anfitrión que Docker, se conecta a `localhost` en los puertos publicados; en modo HIL, el firmware corre en el ESP32 físico y se conecta por WiFi a la dirección IP del anfitrión Docker en la red local.

7.5.2. Scripts auxiliares

`view_simulation.sh` ejecuta `gz simg` dentro del contenedor *gateway* mediante `docker exec`: lanza solo la GUI de Gazebo en modo cliente, conectándose al servidor de física ya en ejecución, sin necesidad de que la GUI estuviera activa desde el inicio.

`reset_position.sh` llama al servicio Gazebo `/world/empty_betaflight_world/set_p` para reponer el dron en su posición inicial ($x = 0, y = 0, z = 0,2$) m, permitiendo repetir ensayos sin reiniciar contenedores ni perder el estado del SITL.

7.6. Modo HIL — Hardware in the Loop

El modo HIL es una variante del entorno SITL en la que el firmware AFC corre en el microcontrolador físico ESP32-WROVER-B del módulo AM-036, en lugar del binario Linux. El

entorno del lado del PC — Gazebo, SITL y ESS — permanece exactamente igual; solo cambia el lado del AFC, que en la Figura 12 pasa de ser el bloque *AFC en Linux* a la placa física, conectada por WiFi a los mismos servidores TCP/UDP del PC.

Las diferencias prácticas respecto al modo SITL puro son tres. **Compilación:** el AFC se compila con `PlatformESP32` en lugar de `PlatformLinux`; las Capas 2–5 son código idéntico, solo cambia la Capa 1. **Transporte por WiFi:** los *sockets* de *loopback* del modo SITL se sustituyen por la IP del anfitrión Docker en la red local; el ESP32 abre sus conexiones TCP (puertos 5800, 5801, 5761) hacia esa dirección, transparente porque esos puertos ya están publicados por el contenedor *gateway*. **Jitter de red:** a diferencia del *loopback*, la WiFi introduce jitter que puede exponer problemas de sincronización en el lazo de control no visibles en el SITL puro, lo que hace del HIL una validación más exigente.

El propósito central del modo HIL es verificar que la capacidad de procesamiento y el espacio de memoria del ESP32-WROVER-B son suficientes para ejecutar el firmware AFC sin desbordamientos de pila, saturación de CPU ni degradación de los periodos de las tareas: es el mismo código de aplicación que se simuló en Linux corriendo ahora sobre FreeRTOS en el microcontrolador real.

7.7. Flujo de datos *end-to-end* en simulación

Esta sección traza el ciclo completo de un ensayo simulado, desde que la GCS envía una referencia de vuelo hasta que el dron virtual actúa y la telemetría de sensores vuelve al AFC para cerrar el lazo de control. El flujo integra todos los componentes descritos en el capítulo y es el equivalente, en simulación, de los flujos de datos del firmware AFC descritos en la Sección 6.7.

El ciclo consta de seis etapas que se ejecutan de forma continua y concurrente:

1. **GCS → AFC (referencias IBus/UDP).** La GCS envía la posición o actitud deseada al AFC mediante el protocolo IBus codificado sobre UDP, dirigido al puerto 5762. El `ChannelIBus` del AFC (§6.4.1.2) recibe el paquete y deposita la referencia en el *Blackboard* bajo el identificador `DataID::RC_Input`.
2. **AFC → SITL (referencias MSP/TCP).** El `DataNode` encargado de la emisión de referencias RC (§6.5.4) lee periódicamente el dato `RC_Output` del *Blackboard* y lo envía al SITL por TCP con protocolo MSP v2. Este flujo es funcionalmente idéntico al descrito en §6.7.4, con la única diferencia de que el transporte es un `SimpleTCP` Linux hacia `127.0.0.1:5761` en lugar de una UART física.
3. **SITL → Gazebo (comandos de motor UDP 9002).** El proceso Betaflight SITL recibe las referencias MSP del AFC, aplica su mixer de cuadricóptero y los controladores PID de actitud, y envía los comandos de velocidad angular de los cuatro rotores al *plugin* `BetaFlightPlugin` de Gazebo por UDP al puerto 9002.

4. **Gazebo (dinámica) → realimentación.** El motor de física de Gazebo integra las ecuaciones de la dinámica del dron con el paso de $\Delta t = 1 \text{ ms}$ y produce dos salidas: (a) los datos de la IMU (acelerómetro y giróscopo) que se devuelven al SITL por el mismo canal UDP 9002, cerrando el lazo interno de estabilización de Betaflight; y (b) los tópicos de sensores (`/gps`, `/mag`, `/air_pressure`) que la ESS consume para alimentar al AFC.
5. **ESS → AFC (sensores por TCP).** El módulo `simulador_gps.py` entrega cada segundo una trama NMEA \$GPRMC al AFC por TCP 5801; el módulo `I2CSim.py` responde a las peticiones de lectura del driver I2C del AFC entregando datos del barómetro BMP390 y del magnetómetro QMC5883L por TCP 5800. Los `DataNodes` correspondientes (§6.5.4) actualizan el *Blackboard* con los identificadores `GPSData`, `BaroData` y `MagData`.
6. **AFC cierra el lazo de control.** El `ControlManager` (§6.5.3) lee periódicamente todos los datos del *Blackboard* (`RC_Input`, `GPSData`, `BaroData`, `MagData`, `IMUData` — este último proveniente del SITL —), ejecuta los algoritmos PID en cascada (§6.5.3.2) y deposita el resultado en `RC_Output`. El `DataNode` de emisión RC lo recoge en el siguiente periodo y el ciclo vuelve a la etapa 2.

Los seis pasos recorren exactamente los enlaces ya identificados en la Figura 12 y la Tabla 6; lo que añade este ciclo es el orden temporal en que se activan: la referencia de la GCS desencadena la cadena AFC→SITL→Gazebo, cuya realimentación de sensores cierra el lazo de nuevo en el AFC.

El flujo descrito muestra por qué la arquitectura de capas del AFC es una precondition del entorno de simulación y no un añadido posterior: si el firmware dependiera directamente de los periféricos hardware, la única forma de simularlo sería mediante capas de emulación parásitas que romperían la correspondencia con el código real. La abstracción por interfaces permite, en cambio, que el código de aplicación sea literalmente el mismo ejecutable en simulación y en vuelo real.

Capítulo 8

Sistema de Control en Tierra (GCS)

La GCS (*Ground Control Station*) es la aplicación Python desarrollada en este trabajo para sustituir al mando RC tradicional y añadir las funciones de telemetría, identificación de planta y diseño de control que constituyen uno de los objetivos centrales del TFM. La aplicación se conecta al AFC por dos canales simultáneos e independientes: comandos de configuración y petición de datos mediante MSP sobre TCP (puerto 12345), y envío continuo de referencias de control mediante IBus sobre UDP (puerto 12346). El Capítulo 7 describe el entorno de simulación sobre el que se valida la propia GCS.

8.1. Introducción y objetivos

El mando RC convencional opera por radiofrecuencia y exige línea de visión directa entre el piloto y el UAV. La GCS abstrae las comunicaciones a TCP/UDP y permite operar desde cualquier red IP, cumpliendo el objetivo de «*controlar el dron de forma remota sin control manual*». Sus tres funciones principales son:

1. **Mando virtual:** envío de referencias de actitud y throttle al AFC, reemplazando al mando RC. Incluye secuencias de ensayo automatizadas para la recopilación de datos.
2. **Telemetría en tiempo real:** petición y visualización de variables del AFC a frecuencia configurable, con exportación a CSV.
3. **Ciclo de identificación y diseño de control:** identificación paramétrica de la planta a partir de datos de ensayo y sintonización PID mediante reglas SIMC, con carga de parámetros al AFC sin recompilar el *firmware*.

La aplicación está desarrollada en Python 3 y emplea **DearPyGui** como biblioteca de interfaz gráfica de modo inmediato, **scipy** para el ajuste de modelos y **numpy** para la simulación discreta. La concurrencia se gestiona con el módulo estándar `threading`. Estas tres funciones se implementan como un backend independiente de la interfaz gráfica (§8.2), lo que permite exponerlas también a clientes automatizados.

8.2. Arquitectura general

La GCS se organiza en cinco pestañas independientes que comparten dos recursos comunes: el objeto `Connection` — que encapsula el socket TCP hacia el AFC — y el `SignalBus` — un bus interno de eventos que permite a las pestañas intercambiarse datos sin referenciarse directamente. La Tabla 9 resume las responsabilidades de cada pestaña y los canales que utiliza.

Tabla 9. Pestañas de la GCS, clase principal, canal hacia el AFC y función.

Pestaña	Clase backend	Canal AFC	Función
Remote	Remote	IBus / UDP	Mando virtual y tests automatizados
Telemetría	TelemetryMgr	MSP / TCP	Petición y visualización de datos
Settings	SettingsTab	MSP / TCP	Nivel de control y parámetros PID
Identificación	PlantIden	(CSV o bus)	Ajuste paramétrico de la planta
Control Design	ControlDesign	→ Settings	Sintonización SIMC y simulación

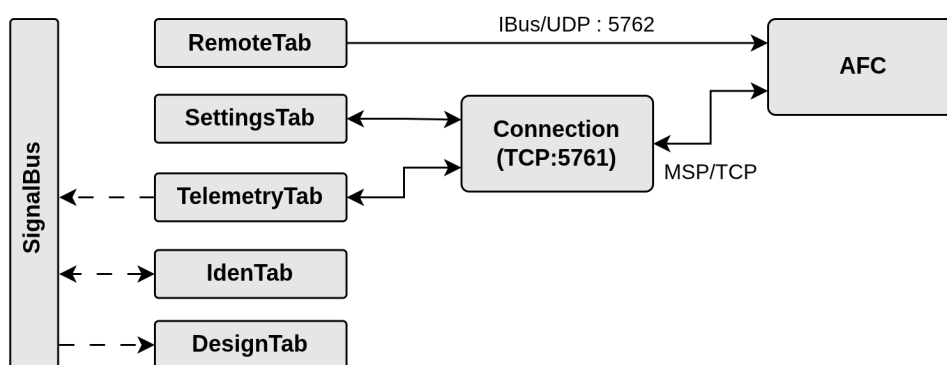


Figura 14. Arquitectura interna de la GCS. Las flechas continuas representan comunicación de red con el AFC; las flechas discontinuas representan solicitudes internas via `SignalBus` entre pestañas.

La lógica de estas pestañas — conexión, mando virtual, telemetría, configuración de parámetros, identificación y diseño de control — vive en las mismas clases backend independientemente de cómo se invoquen: además de la interfaz gráfica de escritorio, un servicio HTTP/JSON expone las mismas funciones por red, sin duplicar lógica. Esta separación entre backend y frontend permite añadir un segundo frontend, pensado para clientes automatizados en vez de para un humano frente a una GUI, sin cambiar una línea del backend — §8.11 detalla este servicio, usado para ejecutar la campaña de ensayos de navegación por ruta del Capítulo 9 de forma reproducible.

8.3. Capa de comunicaciones: protocolo MSP V2

El protocolo MSP (*MultiWii Serial Protocol*) en su versión 2 es el mismo que usa Betaflight para comunicarse con herramientas externas. La GCS lo reutiliza para intercambiar datos estructurados con el AFC, que lo implementa en su Capa 3 (§6.4).

8.3.1. Estructura del paquete MSP V2

Cada trama MSP V2 tiene la estructura de la Tabla 10. Los campos `command` y `size` se transmiten en formato *little-endian* de 16 bits; el *checksum* CRC8-DVB-S2 abarca desde el campo `flag` hasta el último byte del *payload* (excluye los bytes de cabecera \$ y X).

Tabla 10. Campos de la trama MSP V2.

Campo	Tamaño	Descripción
\$	1 B	Byte mágico de inicio
X	1 B	Identificador de versión V2
direction	1 B	< petición, > respuesta, ! error
flag	1 B	Reservado (0x00)
command	2 B	ID del comando (uint16 LE)
size	2 B	Longitud del <i>payload</i> (uint16 LE)
<i>payload</i>	size B	Datos del mensaje
CRC8	1 B	Checksum DVB-S2 sobre [flag... <i>payload</i>]

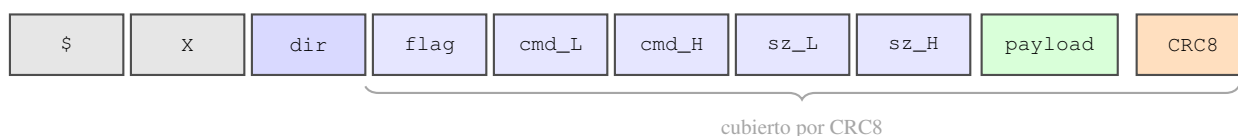


Figura 15. Estructura de trama MSP V2. El *checksum* cubre desde `flag` hasta el final del *payload*, excluyendo los dos bytes de cabecera.

8.3.2. Encoder, decoder y CRC8-DVB-S2

La clase `MspEncoder` (módulo `commands/msp.py`) serializa un `MspPacket` a bytes en cuatro pasos: escritura de la cabecera \$X, serialización del campo `direction`, empaquetado de `command` y `size` con `struct.pack` («H», ...), y cálculo del CRC8 sobre `buffer[3:]` antes de añadirlo al final.

El Listado 8.1 muestra el método de codificación completo.

```
1 def encode(self, packet: MspPacket) -> bytes:
2     buffer = bytearray()
3     buffer.append(ord("$"))
4     buffer.append(ord("X"))
5     buffer.append(packet.direction)
6     flag = 0
7     buffer.append(flag)
8     buffer.extend(struct.pack("<H", packet.command))
9     buffer.extend(struct.pack("<H", len(packet.payload)))
10    if packet.payload:
11        buffer.extend(packet.payload)
12    # CRC sobre buffer[3:] (desde flag, excluye $X)
13    checksum = 0
14    for byte in buffer[3:]:
15        checksum = crc8_dvb_s2(checksum, byte)
16    buffer.append(checksum)
17    return bytes(buffer)
```

Listing 8.1. `MspEncoder.encode()` — serialización de un paquete MSP V2 a bytes crudos (`commands/msp.py`).

El `MspDecoder` es una máquina de estados de nueve estados que procesa el flujo de bytes de entrada de forma robusta ante datos espurios: IDLE → HEADER_X → DIRECTION → FLAG → CMD_L/H → SIZE_L/H → PAYLOAD → CHECKSUM. En el estado IDLE el decoder descarta cualquier byte que no sea \$, lo que le permite resincronizarse automáticamente tras una corrupción parcial del canal.

El polinomio CRC8-DVB-S2 (0xD5) se aplica mediante una tabla precalculada de 256 entradas (`CRC8_DVB_S2_TABLE`), evitando el cálculo polinomial en cada byte recibido.

8.3.3. Gestión de la conexión TCP (`Connection`)

La clase `Connection` (`commands/connection.py`) encapsula el socket TCP con *timeout* de 3 s y expone un único método público de uso frecuente: `transact(payload)`, que envía los bytes y espera la respuesta del AFC. La llamada está protegida con un `threading.Lock` porque la pestaña de telemetría y la de configuración pueden invocarla simultáneamente desde hilos distintos.

Si la operación de red falla, `handle_failure()` cierra el socket, marca el objeto como desconectado y dispara los *callbacks* registrados en `on_disconnect`, de modo que cada pestaña puede reaccionar a la pérdida de conexión sin necesidad de sondear el estado del socket en cada iteración.

8.4. Mando virtual (Remote)

8.4.1. Bucle de envío y estados arm/disarm

La clase `Remote` (`remote/remote.py`) envía paquetes IBus/UDP al AFC mediante el hilo `daemon update_loop`, que se ejecuta a la cadencia configurada (`periodMS`, por defecto 10 ms, es decir, 100 Hz), replicando el ritmo de un receptor RC físico.

El estado interno `armed` determina el valor del canal 5 del paquete IBus: si `True` se envía 2000 μ s; si `False`, 1000 μ s. Betaflight ignora todas las referencias de control mientras el canal de armado esté por debajo del umbral, lo que convierte a `armed` en un interruptor de seguridad de primer nivel.

El paquete IBus completo que construye `get_actual_rc()` tiene la forma:

```
[roll, pitch, throttle, yaw, arm, 1500, 1500, ...]
```

donde los cuatro primeros valores corresponden a los *sliders* de la interfaz y el quinto al canal de armado. Los ocho canales auxiliares se fijan en 1500 μ s.

El mecanismo de seguridad en `unarm()` resetea los cuatro canales a `default_cn` (normalmente `[1500, 1500, 1500, 1500]`) y cancela cualquier secuencia activa antes de que el paquete de parada llegue al AFC. Si `update_loop` detecta que `transact()` devuelve `None` (conexión caída), para el hilo y desarma automáticamente.

8.4.2. Tests automatizados: step test y staircase test

La clase `Remote` implementa dos secuencias de excitación que se ejecutan en hilos `daemon` cancelables mediante la señal `abort_sequence`. Ambas emplean `wait_interruptible()`, que comprueba la señal de cancelación cada 100 ms en lugar de bloquearse con un `time.sleep()` completo, lo que permite abortar cualquier secuencia en menos de 100 ms.

Step test (`play_step`): aplica `start_val` durante `duration_1` segundos y luego `end_val` durante `duration_2` segundos, repitiéndose `repeats` veces. El uso típico es un escalón de *throttle* para registrar la respuesta del altímetro y alimentar la identificación de planta.

Staircase test (`play_staircase`): genera una escalera de subida desde `start_val` hasta `end_val` con paso `step_increment`, permaneciendo `duration_per_step` segundos en cada peldaño, y a continuación ejecuta la misma escalera en sentido descendente. El objetivo es localizar el *throttle* de *hovering* antes de realizar el ensayo de escalón.

8.4.3. Interfaz de usuario (RemoteTab)

La pestaña *Remote* presenta cuatro *sliders* verticales (`roll`, `pitch`, `throttle`, `yaw`) en el rango 1000–2000 μ s, los botones `START SYSTEM` y `ARM`, y dos paneles de configuración de los tests. El estado visual de ambos botones de test se sincroniza con `update_ui_state()`: mientras cualquier secuencia esté activa, ambos muestran `STOP` en color de aviso.

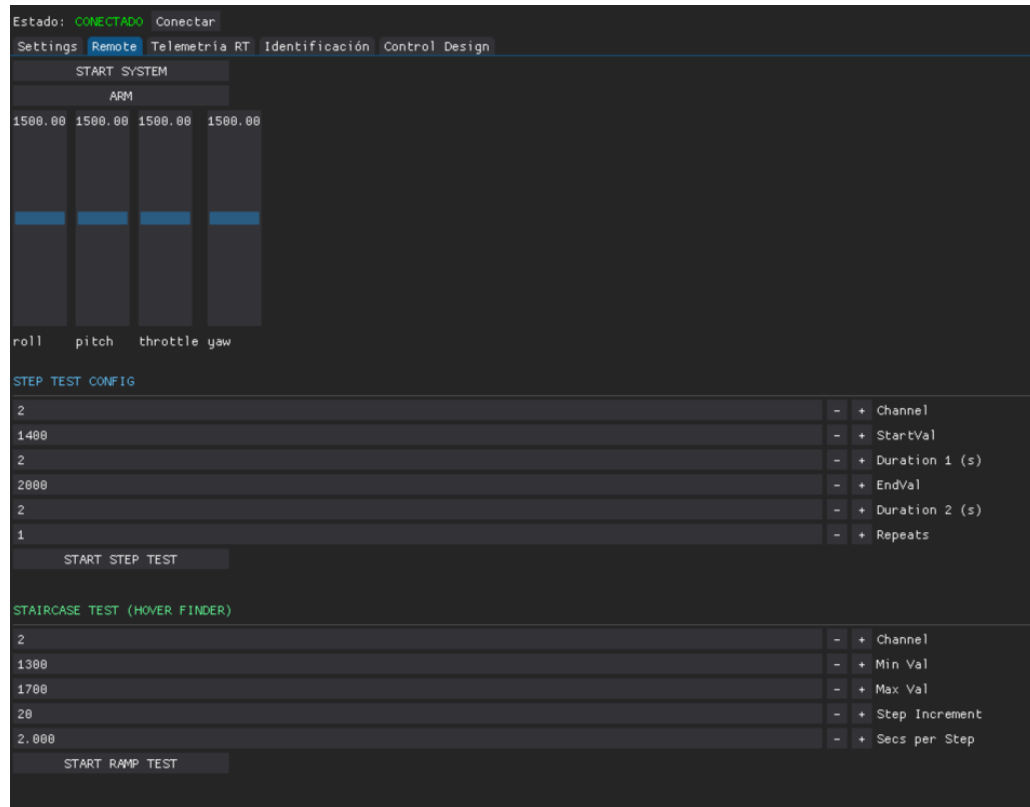


Figura 16. Pestaña *Remote* de la GCS, con los cuatro *sliders* de mando virtual y los paneles de configuración del *step test* y el *staircase test*.

8.5. Telemetría en tiempo real

8.5.1. Suscripción a variables y buffer circular

El `TelemetryMgr` (`telemetry/telemetryMgr.py`) mantiene un registro de comandos MSP disponibles y de las variables activas en cada momento. Cada variable se identifica como «`cmd_id>_<nombre>`» (por ejemplo, «`10_altitude`»).

El mecanismo de suscripción usa un contador de referencias por variable: cuando dos gráficos distintos suscriben la misma variable, el gestor incrementa su contador en lugar de duplicar la petición al AFC. Al desuscribir, decrementa el contador y elimina la petición al AFC solo cuando el contador llega a cero.

8.5.2. Ciclo de muestreo (`TelemetryDataGatherer`)

El `TelemetryDataGatherer` ejecuta un bucle a `samplesperS` Hz (configurable, por defecto 100 Hz) en un hilo daemon. En cada iteración itera los comandos activos, realiza la transacción MSP correspondiente y almacena el *timestamp* junto con los valores en una deque de tamaño fijo:

$$\text{maxlen} = \text{timeframes} \times \text{samplespers} \quad (1)$$

Esta estructura actúa como una ventana temporal deslizante: al alcanzar el límite, los datos más antiguos se descartan automáticamente sin necesidad de gestión explícita. Si cualquier transacción devuelve `None` (pérdida de conexión), el bucle se detiene. El método `update_buffer()` permite reconfigurar el tamaño de la ventana en caliente, deteniendo y reiniciando el hilo.

8.5.3. Interfaz: gráficos configurables y exportación CSV

La pestaña de telemetría (`TelemetryTab`) divide la pantalla en dos columnas: la izquierda muestra el menú de variables agrupadas por comando MSP, con *checkboxes*; la derecha contiene la zona de gráficos dinámica. El botón + Añadir Gráfico crea un nuevo gráfico con su propio selector de variables, lo que permite visualizar simultáneamente señales de distintos comandos MSP.

El refresco de pantalla se realiza a 10 fps (`frame_update = 0,1 s`) desde el hilo principal de la GUI, desacoplando la frecuencia visual de la frecuencia de muestreo de datos. La exportación a CSV transpone los datos de «listas por variable» a «filas por instante temporal» mediante `zip(*data_list)` antes de escribirlos con `csv.writer`.

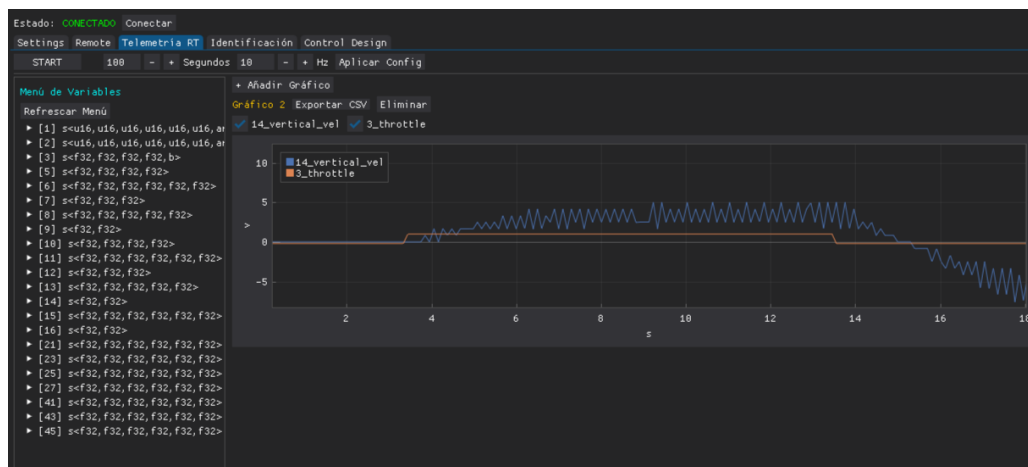


Figura 17. Pestaña de *Telemetría* de la GCS, con el menú de variables agrupadas por comando MSP a la izquierda y la zona de gráficos dinámica a la derecha.

8.6. Configuración del controlador (Settings)

8.6.1. Nivel de control activo

El AFC implementa distintos niveles de control en cascada, gestionados por el `ControlManager` descrito en el Capítulo 6:

- **Velocidades relativas:** control de las velocidades lineales frontal, lateral y vertical en el marco del dron.
- **Velocidades relativas + absolutas:** añade el control de esas mismas velocidades lineales referidas al marco tierra, apoyándose en el nivel anterior como lazo interno.
- **Velocidades relativas + absolutas + posición absoluta de altura:** añade el control de la altura respecto al punto de partida, apoyándose en los dos niveles anteriores en cascada.
- **Velocidades relativas + absolutas + posición absoluta de altura + navegación por ruta:** añade el seguimiento de una ruta de waypoints subida desde la GCS, apoyándose en los tres niveles anteriores en cascada.

La pestaña *Settings* permite cambiar el nivel activo en caliente enviando un entero mediante el comando MSP ID 19 (`set_ctrl1vl`). El cambio surte efecto en el siguiente ciclo del `ControlManager` sin necesidad de reiniciar el AFC.

8.6.2. Parámetros PID por lazo

La Tabla 11 recoge los cinco lazos de control accesibles desde la GCS, con sus identificadores de comando MSP y valores por defecto. El campo `offset` representa el punto de operación del actuador: en el lazo de velocidad vertical, por ejemplo, corresponde al *throttle* de *hovering*.

Tabla 11. Lazos de control configurables desde la pestaña *Settings*.

ID MSP	Lazo	P	I	D	offset
20	Velocidad lateral	0,50	0,05	0,0	0,00
22	Velocidad frontal	0,50	0,05	0,0	-0,12
24	Velocidad vertical	0,40	0,20	0,0	0,54
26	Velocidad angular	0,50	0,05	0,0	0,00
42	Posición vertical	0,50	0,00	0,0	0,00

Cada conjunto [`P`, `I`, `D`, `offset`] se serializa como *payload* del paquete MSP y el AFC lo escribe en la *Blackboard* sin reiniciar, lo que permite una sintonización iterativa en caliente durante el vuelo o la simulación.

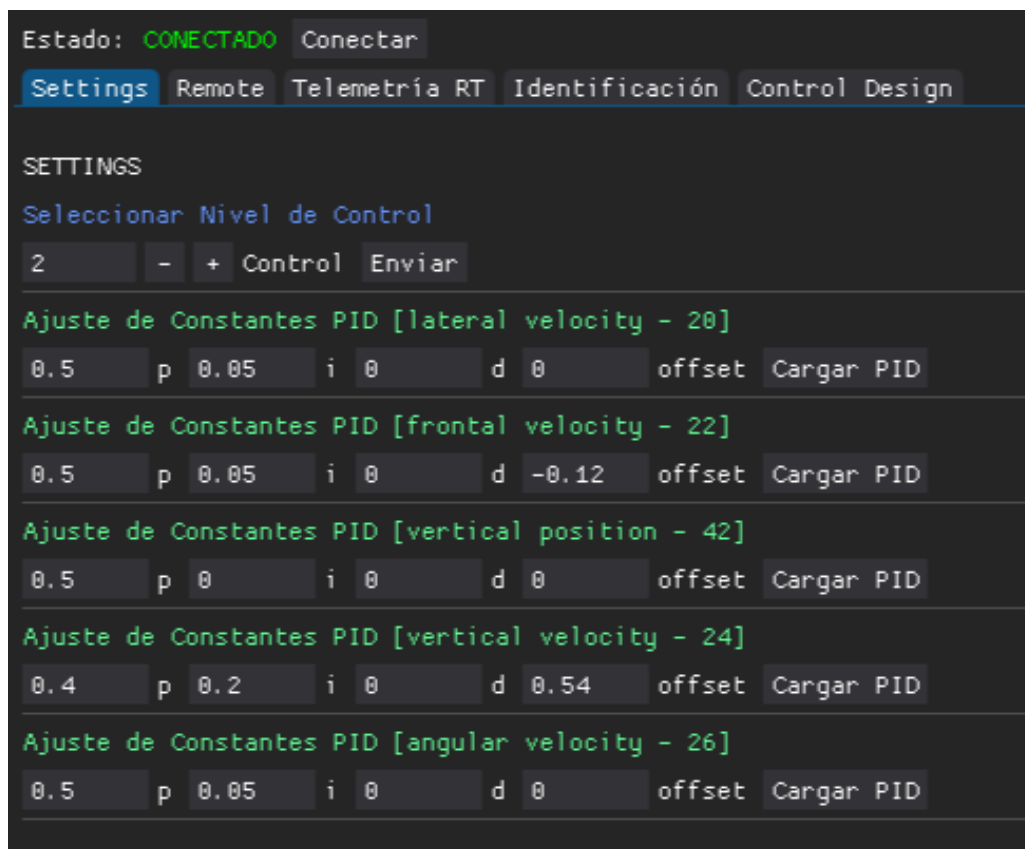


Figura 18. Pestaña *Settings* de la GCS, con el selector de nivel de control activo y los parámetros PID de cada lazo.

Los dos lazos de posición horizontal y angular añadidos para la navegación por ruta (IDs 40 y 44) se configuran de la misma forma vía MSP, aunque no se exponen en esta pestaña de escritorio: se ajustan a través de la API descrita en §8.11.

8.7. Identificación de planta

8.7.1. Modelos soportados y señal de excitación

La clase `PlantIden` (`plantiden/plantiden.py`) admite cuatro modelos de planta de bajo orden. La Tabla 12 los recoge con sus parámetros y el uso típico en los lazos del UAV.

Tabla 12. Modelos de planta soportados por la GCS para la identificación paramétrica.

Modelo	Parámetros	Uso típico
FOPDT	K, τ, θ	Lazos de velocidad con retardo apreciable
FO_SIMPLE	$K, \tau (\theta = 0)$	Lazos rápidos sin retardo
SOPDT	K, τ, ζ, θ	Lazos con dinámica de segundo orden
IPDT	K, θ	Lazo de posición (integrador puro)

La señal de excitación recomendada es el *step test* de la pestaña Remote. El *staircase test* se emplea previamente para localizar el `u_offset` (*throttle de hovering*) necesario para que el escalón parta de un punto de operación estable. Los datos de entrada pueden provenir de un fichero CSV o importarse directamente del buffer de telemetría mediante el `SignalBus` (§8.9).

8.7.2. Ajuste por mínimos cuadrados no lineales

El módulo `plantiden/solver.py` implementa los modelos analíticos de respuesta a escalón y los ajusta a los datos medidos mediante `scipy.optimize.curve_fit`, que minimiza el error cuadrático entre la señal medida y el modelo.

Antes del ajuste, el algoritmo localiza automáticamente el instante del escalón buscando el índice de máxima variación de la señal de entrada:

$$k^* = \arg \max_k |\Delta u_k| = \arg \max_k |u_{k+1} - u_k| \quad (2)$$

La respuesta al escalón del modelo FOPDT que se ajusta es:

$$y(t) = y_0 + K \cdot \Delta u \cdot \left(1 - e^{-(t-t_0-\theta)/\tau}\right) \cdot \mathbf{1}_{t \geq t_0+\theta} \quad (3)$$

donde y_0 es el valor inicial de la salida, Δu la amplitud del escalón y t_0 el instante en que se produce. Una vez ajustados los parámetros, el módulo calcula el coeficiente de determinación R^2 como indicador de la calidad del ajuste.

8.7.3. Flujo de trabajo en la interfaz (IdenTab)

La interfaz de identificación guía al usuario en tres pasos:

1. **Carga de datos:** mediante botón «Cargar CSV» o «RT Telemetry» (importa los datos del buffer activo via `SignalBus`).
2. **Asignación de variables:** combos para seleccionar las columnas t , u e y . Dos líneas de *drag* verticales permiten delimitar visualmente la región de análisis sobre el gráfico.
3. **Cálculo del ajuste:** selección del modelo y botón «Calcular fit». La interfaz muestra los parámetros ajustados, el R^2 y la curva del modelo superpuesta a los datos medidos.

Los resultados se publican en el SignalBus bajo el servicio GET_IDENTIFIED_PLANT para que la pestaña de diseño de control los importe directamente.

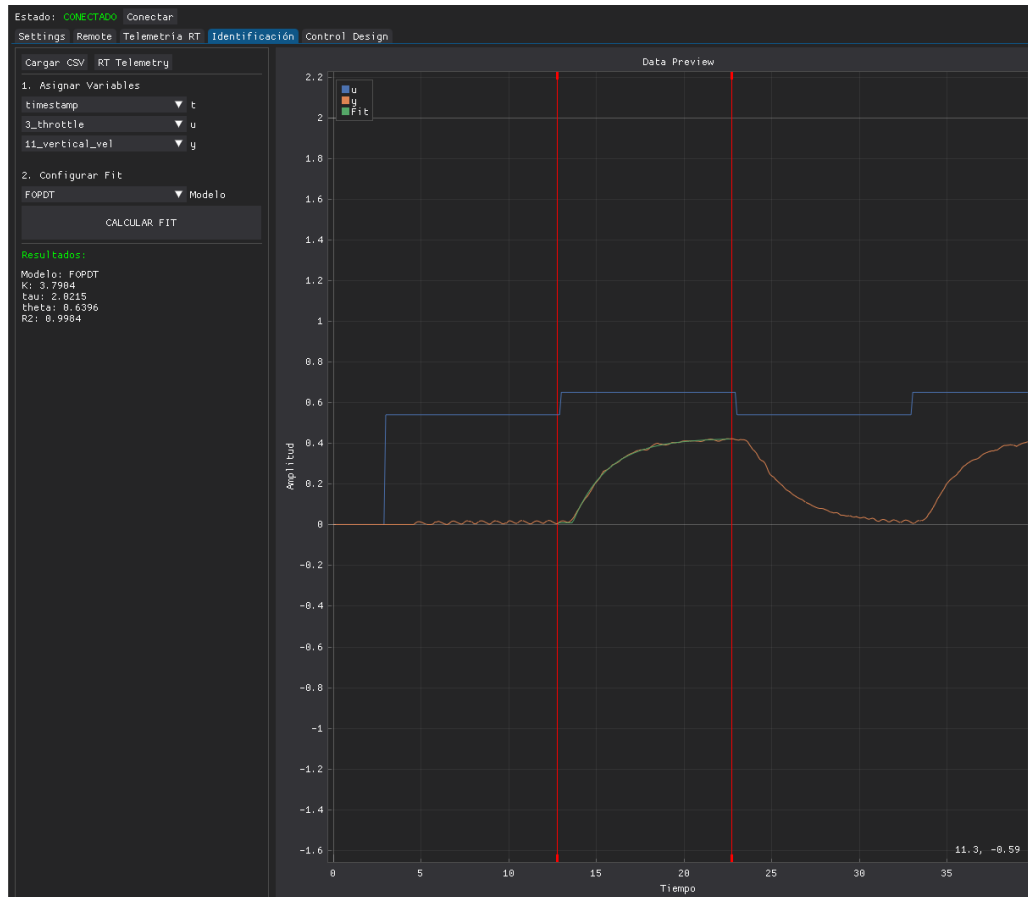


Figura 19. Pestaña de *Identificación* de la GCS, con los datos cargados, la región de análisis delimitada y el resultado del ajuste FOPDT superpuesto.

8.8. Diseño del controlador PID

8.8.1. Reglas SIMC (Skogestad)

La clase `ControlDesign` (`ctrldesign/ctrldesign.py`) aplica las reglas SIMC (*Simplified IMC*) de Skogestad [34], que proporcionan sintonías robustas a partir de los parámetros del modelo identificado. El único parámetro de ajuste del usuario es `factor_agresividad` (rango recomendado 0,5–4,0): valores menores producen respuestas más lentas pero más robustas; valores mayores aceleran la respuesta a costa de menor margen de estabilidad.

Para el modelo FOPDT, las fórmulas son:

$$\tau_c = \max(\theta, 0,1\tau) \cdot \text{factor_agresividad} \quad (4)$$

$$\theta_{eff} = \theta + T_s/2 \quad (5)$$

$$K_p = \frac{1}{K} \cdot \frac{\tau}{\tau_c + \theta_{eff}} \quad (6)$$

$$T_i = \min(\tau, 4(\tau_c + \theta_{eff})), \quad K_i = K_p/T_i \quad (7)$$

$$K_d = 0 \quad (8)$$

El término $T_s/2$ en la ecuación (5) es una corrección por discretización: añade la mitad del período de muestreo al retardo efectivo, ya que en un sistema discreto el actuador no puede responder antes del siguiente instante de muestreo.

Para el modelo IPDT la fórmula de K_p se simplifica a $K_p = (1/K) \cdot 1/(\tau_c + \theta_{eff})$, y $T_i = 4(\tau_c + \theta_{eff})$.

8.8.2. Simulación discreta: saturación y tiempo muerto

Antes de cargar los parámetros al AFC, la GCS permite verificar visualmente la respuesta del lazo cerrado mediante una simulación temporal paso a paso. La planta FOPDT se discretiza con el método de Euler con retención de orden cero (ZOH):

$$a = e^{-T_s/\tau}, \quad b = K(1 - a), \quad y[k] = a y[k - 1] + b u[k - d - 1] \quad (9)$$

donde $d = \lceil \theta/T_s \rceil$ es el número de pasos de retardo. El retardo se implementa con una deque de longitud $d + 1$: en cada paso se añade la señal de control saturada al final y se extrae el elemento del frente para alimentar la planta.

El algoritmo incluye *anti-windup*: si la señal de control calculada supera los límites de saturación $[u_{\min}, u_{\max}]$, se deshace la última acumulación integral (`integral -= error * Ts`) para evitar la saturación prolongada del integrador.

8.8.3. Flujo completo: de la identificación a la carga de parámetros

La pestaña *Control Design* (`ctrlldesign/ctrlldesign_front.py`) encadena cuatro pasos hasta obtener parámetros listos para cargar en el AFC:

1. **Importar planta:** el botón «Importar ID» solicita los datos al SignalBus mediante `GET_IDENTIFIED_PLANT` y rellena automáticamente los campos K , τ , θ (y ζ si el modelo es SOPDT).
2. **Configurar hardware:** el usuario introduce `u_offset`, `u_min` y `u_max` (límites físicos del actuador).
3. **Simular:** el botón «Simular» ejecuta `tune_simc()` y `simulate()`, mostrando las curvas de variable de proceso y señal de control para el escalón de referencia configurado. El usuario puede ajustar `factor_agresividad` e iterar.

4. **Cargar al AFC:** los valores K_p , K_i y K_d obtenidos se copian a la pestaña *Settings* y se envían al AFC via MSP. El AFC los escribe en la *Blackboard* sin reiniciar, completando el ciclo de «sintonización on-the-fly».

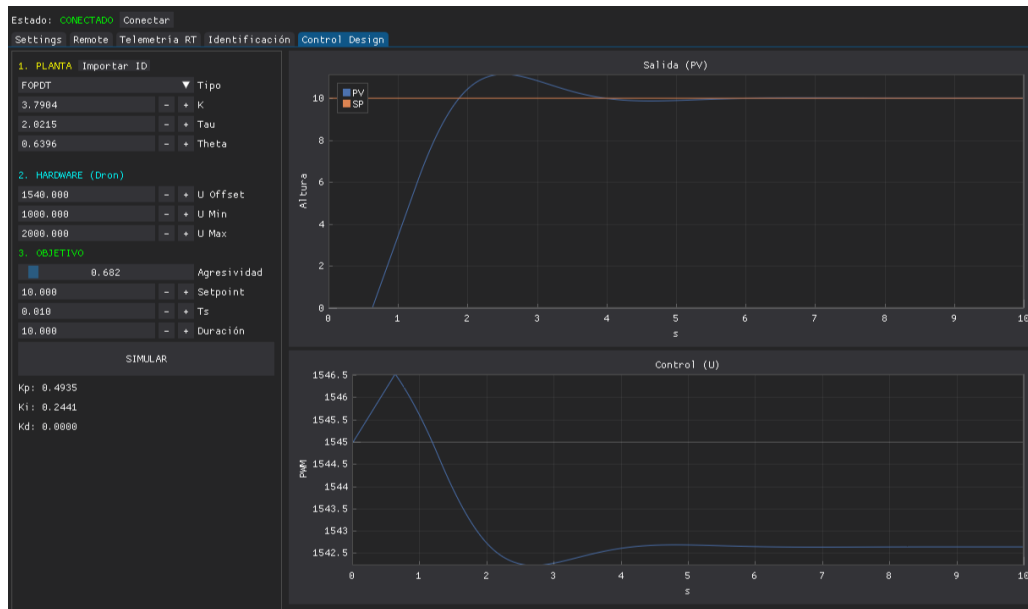


Figura 20. Pestaña *Control Design* de la GCS, con las gráficas de simulación de la variable de proceso y la señal de control para los parámetros SIMC sintonizados.

8.9. Bus interno de eventos (SignalBus)

Las pestañas de la GCS necesitan intercambiarse datos sin referenciarse directamente entre sí, lo que crearía acoplamiento entre componentes. El `SignalBus` (`utils/bus.py`) implementa un patrón *provider/consumer* minimalista: un proveedor registra una función *callback* bajo un nombre de servicio, y cualquier consumidor puede solicitar ese dato mediante el mismo nombre.

```
1 # Proveedor (IdenTab) registra su servicio
2 bus.register_provider("GET_IDENTIFIED_PLANT",
3     self._provide_plant_params)
4
5 # Consumidor (DesignTab) solicita el dato
6 data = bus.request("GET_IDENTIFIED_PLANT")
```

Listing 8.2. `SignalBus` — registro de proveedor y solicitud de servicio (`utils/bus.py`).

La Tabla 13 lista los dos servicios activos en la aplicación. El desacoplamiento que proporciona el `SignalBus` permite añadir nuevas pestañas que consuman o publiquen datos sin modificar las existentes.

Tabla 13. Servicios registrados en el SignalBus de la GCS.

Nombre del servicio	Proveedor	Consumidor
GET_IDENTIFIED_PLANT	IdenTab	DesignTab
GET_TELEMETRY_DATA	TelemetryTab	IdenTab

8.10. Flujo de trabajo integrado

Esta sección describe el procedimiento completo desde el primer encendido hasta la validación de un lazo de control sintonizado. Los pasos se pueden ejecutar tanto con el entorno de simulación del Capítulo 7 como con el UAV físico.

1. **Conexión:** iniciar el entorno de simulación (o conectar el AFC físico) y pulsar START SYSTEM en la pestaña *Remote*.
2. **Localizar el punto de *hovering*:** ejecutar el *Staircase Test* en el canal de *throttle*; identificar el valor al que el dron queda estacionario — ese valor es `u_offset`.
3. **Ensayo de escalón:** configurar el *Step Test* con ese `u_offset` como `start_val` y un incremento moderado como `end_val`.
4. **Recoger telemetría:** en la pestaña *Telemetría*, suscribir las variables de altitud y *throttle* y pulsar START para iniciar el muestreo.
5. **Identificar la planta:** en la pestaña *Identificación*, importar los datos del buffer («RT Telemetry»), seleccionar el modelo FOPDT y calcular el ajuste. Verificar $R^2 > 0,9$.
6. **Diseñar el controlador:** importar los parámetros identificados en la pestaña *Control Design*, ajustar `factor_agresividad` hasta obtener una respuesta temporal aceptable en la simulación.
7. **Cargar los PID:** transferir K_p , K_i , K_d a la pestaña *Settings* y pulsar «Cargar PID». El AFC aplica los nuevos parámetros sin reiniciar.
8. **Validar:** ejecutar un nuevo *Step Test* y comparar la respuesta en telemetría con la simulada. Iterar desde el paso 6 si es necesario.

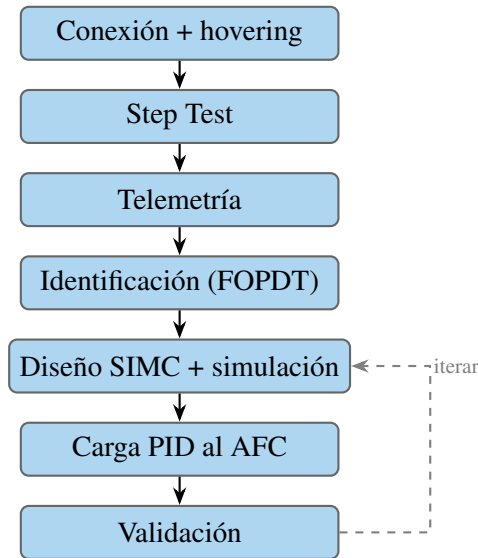


Figura 21. Flujo de trabajo integrado de la GCS: desde el ensayo de escalón hasta la validación del lazo de control sintonizado. La flecha discontinua indica la posibilidad de iterar ajustando `factor_agresividad` sin repetir el ensayo.

8.11. API para automatización

El mismo backend descrito en este capítulo —conexión, mando virtual, telemetría, configuración, identificación y diseño de control— se expone también por HTTP/JSON, sin duplicar lógica, para poder operarlo desde scripts automáticos en vez de desde la interfaz gráfica de escritorio. Sobre ese backend compartido se añaden dos funciones más: enviar la ruta de waypoints y reposicionar el dron en el entorno de simulación entre ensayos. Este backend compartido es el que sostiene la campaña de ensayos automatizada del Capítulo 9.

Capítulo 9

Ensayos

9.1. Ensayos en SITL

9.1.1. Verificación de los flujos de comunicación

Con el sistema completo levantado (SITL + AFC + GCS, todo sobre el mismo host) se capturan con Wireshark, sobre la interfaz de *loopback* `lo`, un intercambio MSP completo y una trama IBus, para comprobar que el protocolo descrito en el Capítulo 8 (Tabla 10) se cumple a nivel de bytes, no solo a nivel funcional.

9.1.1.1. MSP (TCP, puerto 12345)

Se captura un intercambio GET completo: una petición GCS → AFC de 9 bytes de *payload* (Figura 22) y la respuesta inmediata del AFC de 37 bytes (Figura 23), en el mismo *stream* TCP. El campo `cmd = 302` ($= 300 + 2$) corresponde a la convención autodescriptiva de la GCS (§8.10): el prefijo $300 + id$ solicita el valor actual del comando `id` (aquí, `id = 2`).

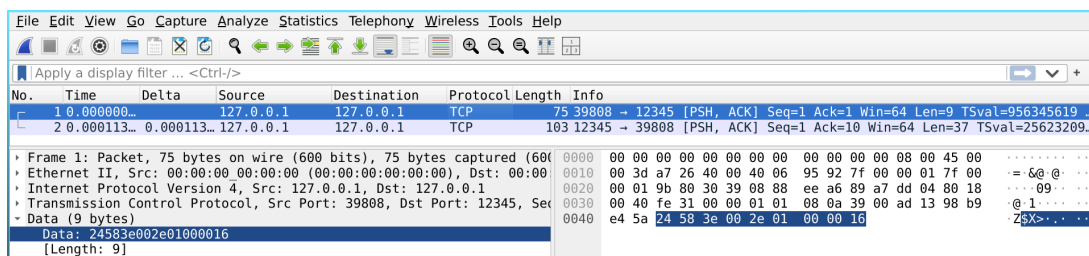


Figura 22. Captura Wireshark de la petición MSP (GCS → AFC), puerto TCP 12345. Data (9 bytes): 24 58 3e 00 2e 01 00 00 16.

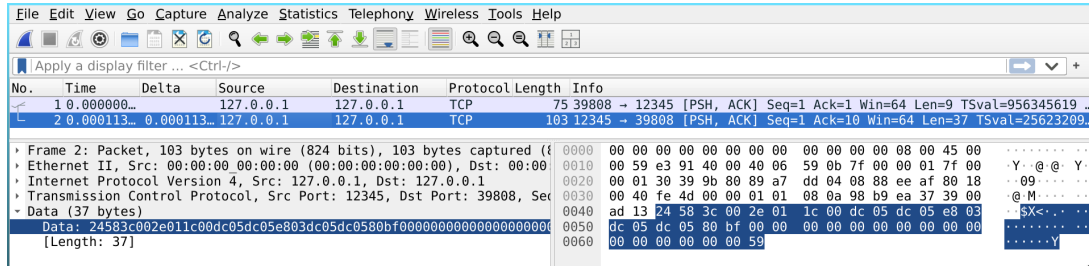


Figura 23. Captura Wireshark de la respuesta MSP (AFC → GCS) al mismo intercambio. Data (37 bytes), con cmd = 302 igual que en la petición.

Tabla 14. Bytes capturados del intercambio MSP GET ($id = 2$) y verificación de CRC8-DVB-S2.

Frame	cmd	size (B)	CRC	Bytes (hex)
Petición (>)	302	0	0x16 ✓	24 58 3e 00 2e 01 00 00 16
Respuesta (<)	302	28	0x59 ✓	24 58 3c 00 2e 01 1c 00 ...59

Ambos frames se descomponen a mano según la Tabla 10 (cabecera \$X, dirección, flag, cmd en 16 bits, size en 16 bits, *payload* y CRC8) y el CRC se recalcula reutilizando la función `crc8_dvb_s2` real del proyecto (`tools/msp/commands/msp.py`) sobre los bytes capturados, coincidiendo en ambos sentidos con el byte transmitido. El campo cmd de la respuesta coincide con el de la petición, confirmando que el AFC referencia el mismo comando en su contestación.

9.1.1.2. IBus (UDP, puerto 12346)

La Figura 24 muestra una trama IBus capturada con el envío continuo activo (*START SYSTEM* en la pestaña *Remote*): un datagrama UDP de 32 bytes hacia el puerto 12346, con cabecera 0x20 0x40 y 14 canales de 2 bytes en *little-endian*.

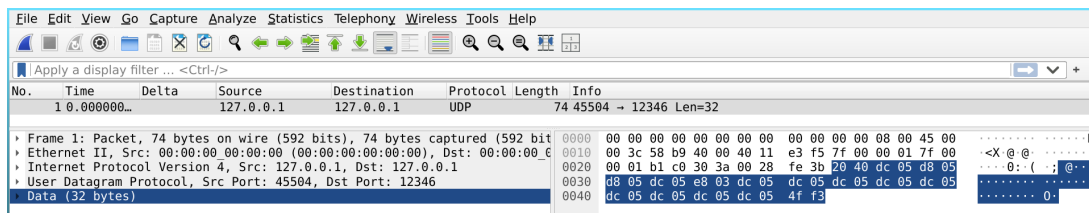


Figura 24. Captura Wireshark de una trama IBus (GCS → AFC), puerto UDP 12346. Data (32 bytes).

El *checksum* estándar IBus ($0xFFFF - \sum \text{bytes}$, calculado sobre los primeros 30 bytes) se recalcula a mano sobre los bytes capturados y coincide con los dos últimos bytes de la

trama (0xf34f), confirmando que la implementación del AFC (`IbusProtocol.h`) sigue el estándar FlySky sin desviaciones. Los 14 canales decodificados corresponden a los *sticks* en reposo (roll/pitch $\approx 1500 \mu s$, throttle = $1000 \mu s$), consistente con el mando virtual de la GCS sin entrada del usuario en el momento de la captura.

Ambas capturas quedan archivadas en `code/ensayos/ensayo_1/` (ficheros `.pcapng` completos y aislados por intercambio).

9.2. Ensayos HIL

Los ensayos de esta sección se ejecutan sobre la placa física AM-036 (ESP32), conectada por WiFi al mismo segmento de red que el resto del sistema (SITL, GCS), con conexión estable a Betaflight SITL, I2Csim y GPSSim expuestos por la red.

9.2.1. Uso de CPU en el ESP32

El `DataNode` `Nodo_Emission_SystemHealth` (ID 46, 1 Hz) publica el % de CPU consumido por cada una de las tres tareas principales de la aplicación (`CtrlMgr`, `DataNodes`, `CmdMgr`), calculado a partir del delta de `uxTaskGetSystemState()` entre muestras (`configGENERATE_RUN_TIME_STATS`, ya activo en el `sdkconfig` del proyecto). La Figura 25 (panel inferior) muestra las tres señales durante una captura de 43 s.

Tras un arranque de ~ 5 s en el que `CtrlMgr` sube desde el 2,5 % hasta su régimen estacionario, las tres tareas se estabilizan en el rango 2,5–7 %. La Tabla 15 resume los estadísticos de la captura completa. La carga es muy baja en las tres tareas, coherente con el *footprint* estático de §9.2.2.1: la lógica de vuelo es una fracción pequeña del binario y consume una fracción igualmente pequeña de CPU.

Tabla 15. Uso de CPU por tarea durante 43 s de vuelo HIL (AM-036).

Tarea	Mín. (%)	Media (%)	Máx. (%)
CtrlMgr	2,48	5,13	6,02
DataNodes	2,62	3,45	3,83
CmdMgr	0,01	5,57	7,17

9.2.2. Uso de memoria (heap y stack)

9.2.2.1. Footprint estático

Antes de medir consumo en *runtime* se caracteriza el *footprint* estático del binario, obtenido con `idf.py size` sobre el mismo código fuente compilado para el target ESP32 (sin flashear la placa). La Tabla 16 resume el uso de memoria del binario resultante.

Tabla 16. Uso de memoria estático del binario TransponderESP32.

Región	Usado (B)	Total (B)	Uso (%)
IRAM	97 186	131 072	74,15
DRAM	35 152	180 736	19,45
Flash (código + datos)	993 442	—	—

El binario completo ocupa 1 107 140 bytes de flash, un 27 % de la partición de aplicación disponible (0x3f0000 B), quedando un 73 % libre para crecimiento futuro (waypoints, telemetría adicional, etc.).

El desglose por archivo objeto (`idf.py size-components`) permite separar qué parte de ese *footprint* corresponde al código propio del AFC frente a las bibliotecas de ESP-IDF. Agrupando los componentes del proyecto (`main`, `AppLogic`, `PlatformESP32`, `Connectivity`, `Core`) frente a la pila de comunicaciones (`net80211`, `lwip`, `mbedtls`, `mbedcrypto`, `wpa_supplicant`, `esp_wifi`, `esp_netif`) y el resto de componentes de sistema de ESP-IDF (RTOS, HAL, *drivers*, *libc*), se obtiene la Tabla 17.

Tabla 17. Distribución del *footprint* por origen del código.

Origen	Tamaño (B)	% del total
Código propio del AFC	254 391	20,8
Pila de comunicaciones (WiFi/TLS/IP)	439 093	35,8
Resto de ESP-IDF (RTOS, HAL, <i>libc</i> ...)	532 164	43,4

La lógica de vuelo propiamente dicha (Capas 3-5, §6) representa apenas una quinta parte del binario final: la mayor parte del *footprint* la ocupa la infraestructura de comunicaciones necesaria para que el transporte de red (§6.2) funcione sobre WiFi, no la aplicación en sí.

9.2.2.2. Uso en *runtime*

El mismo `SystemHealthData` de §9.2.1 publica `esp_get_free_heap_size()`, `esp_get_minimum_free_heap_size()` y el *stack high-water mark* (`uxTaskGetStackHighWaterMark()`) de las tres tareas, expresado aquí como % de uso sobre los 4096 B configurados para cada una. La Figura 25 recoge las tres magnitudes — heap, pila y CPU — de la misma captura de 43 s.

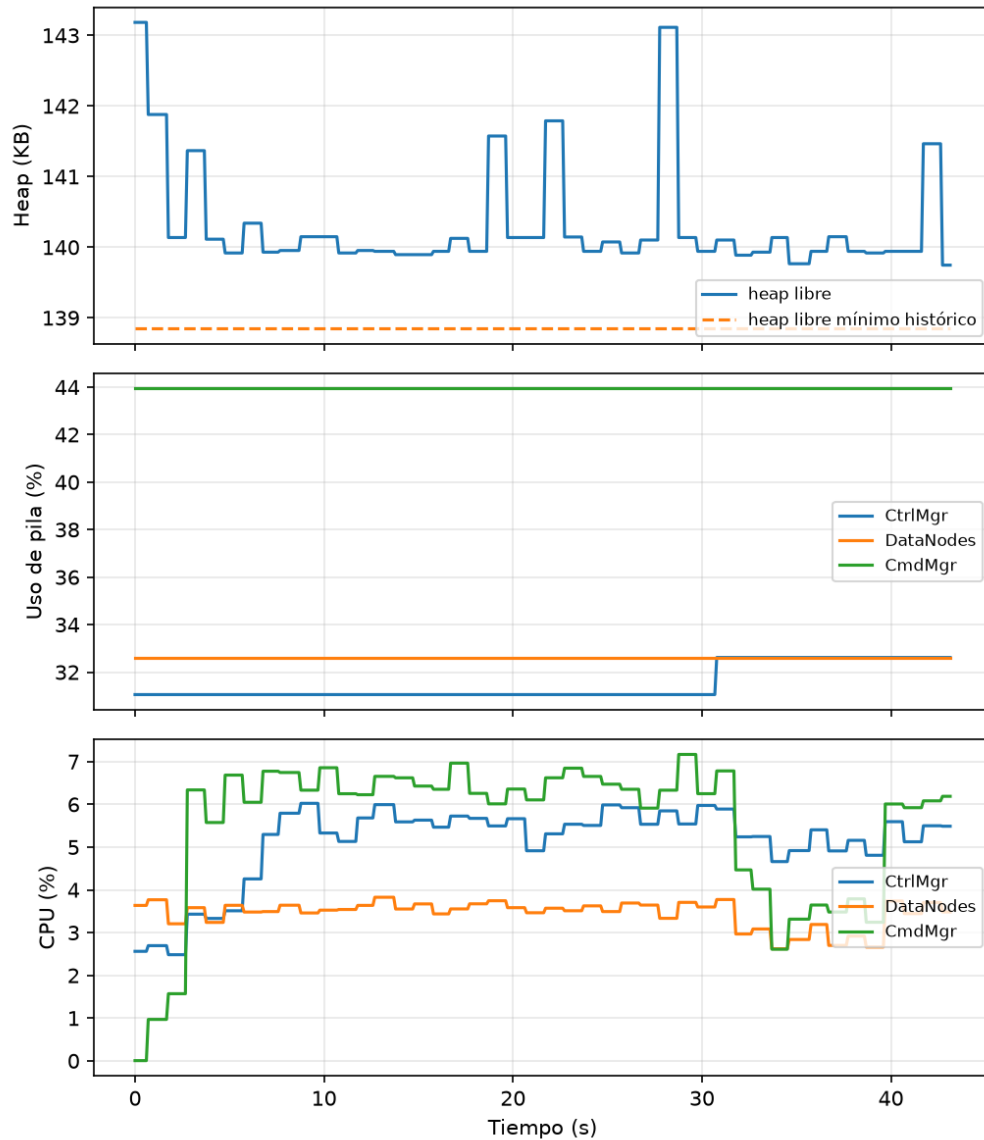


Figura 25. Instrumentación HIL en *runtime* (AM-036, 43 s): heap libre (arriba), uso de pila por tarea (centro) y uso de CPU por tarea (abajo, ya presentado en §9.2.1).

El heap libre se mantiene estable en torno a 140–144 KB, con picos puntuales de ~1–2 KB coincidiendo con actividad de la pila WiFi/TLS (reconexiones, *keep-alive*); el mínimo histórico de la sesión (142 180 B) se alcanza en los primeros segundos y no vuelve a bajar, sin indicios de fuga en esta ventana de 43 s. El uso de pila es constante en las tres tareas y queda lejos del límite: 31–33 % en CtrlMgr/DataNodes y 44 % en CmdMgr, la tarea con menos margen pero aun así con más de la mitad de sus 4096 B libres.

Los datos completos (CSV exportado desde la pestaña *Telemetry* de la GCS) y el script de generación de la figura están en `code/ensayos/ensayo_hil2/`.

9.3. Ciclo completo de identificación y diseño de control

Esta sección ejecuta de principio a fin el procedimiento descrito en §8.10 sobre una planta real del entorno SITL, sin reutilizar datos de ensayos anteriores. La planta elegida es el lazo de **altura: throttle** como entrada y **velocidad vertical** (variable `9_vertical_vel` del Blackboard) como salida, por ser más representativa de la dinámica de vuelo real que un lazo de actitud.

9.3.1. Localización del punto de *hovering*

Se ejecuta un *Staircase Test* en el canal de *throttle* para localizar el valor de `u_offset` en el que el UAV queda estacionario (Figura 26). El valor identificado es $u_{offset} = 0,54$.

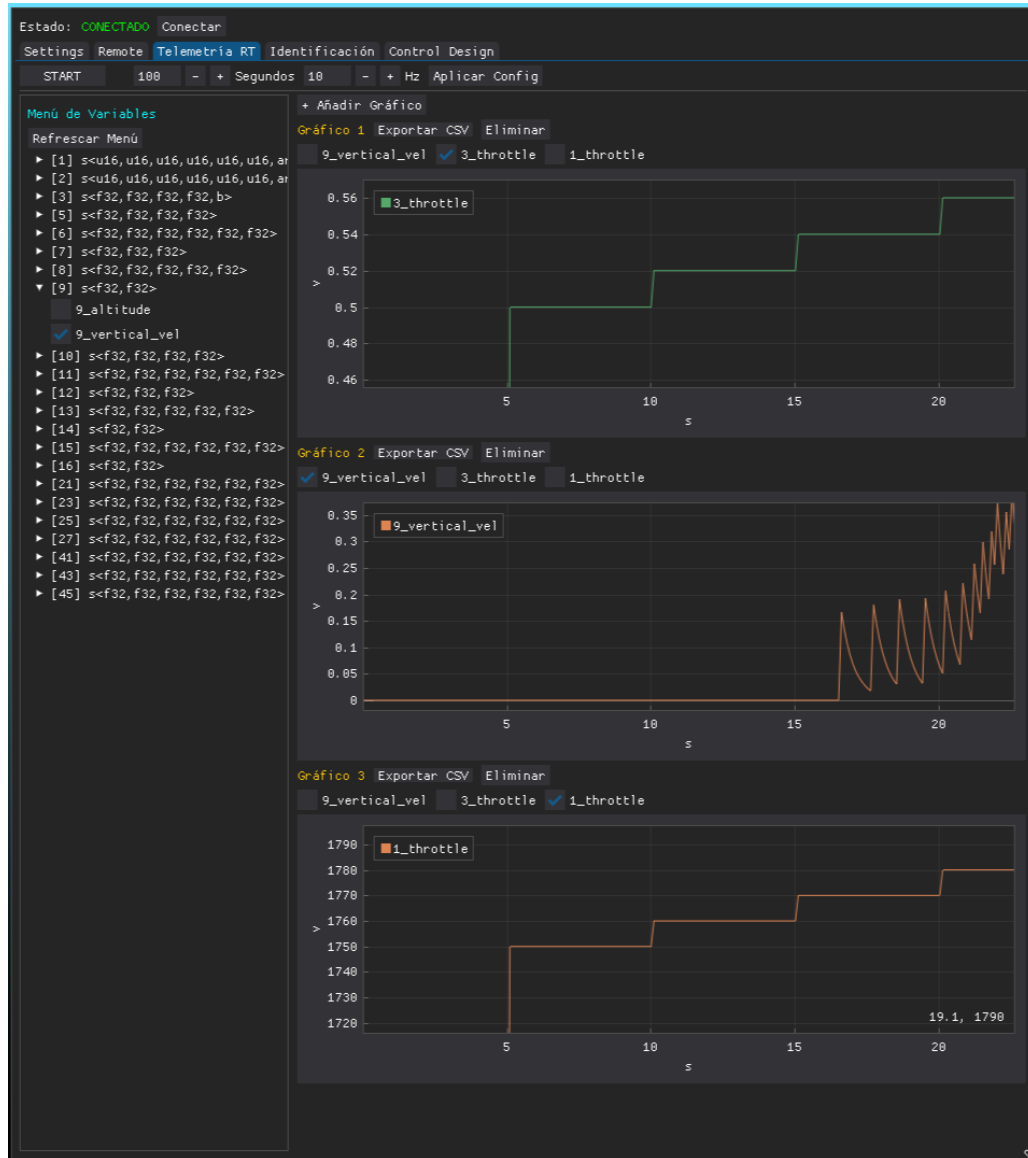


Figura 26. Staircase test sobre el canal de *throttle*. El escalonado de 1_throttle/3_throttle (paneles inferior y superior) permite localizar u_{offset} a partir de la respuesta de 9_vertical_vel (panel central).

9.3.2. Ensayo de escalón

Partiendo de u_{offset} , se aplica un escalón de *throttle* (Figura 27). La respuesta de 9_vertical_vel muestra la subida esperada de primer orden.

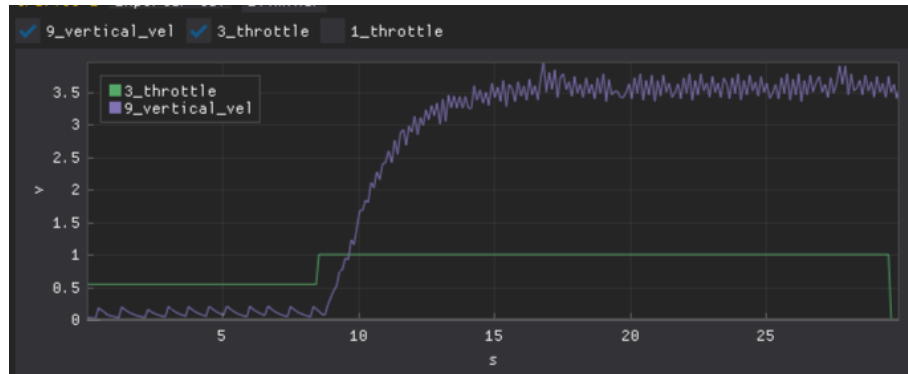


Figura 27. Ensayo de escalón: *throttle* (verde) y *9_vertical_vel* (violeta).

9.3.3. Identificación de la planta

Los datos del ensayo de escalón se cargan en la pestaña *Identificación* con *3_throttle* como entrada u y *9_vertical_vel* como salida y . Siguiendo la Tabla 12 (§8.7), al ser un lazo de velocidad se ajusta un modelo **FOPDT**. El resultado (Figura 28) es:

Tabla 18. Parámetros identificados del modelo FOPDT *throttle* $\rightarrow$ velocidad vertical.

K	τ (s)	θ (s)	R^2
7,588	1,804	0,470	0,978

El ajuste cumple sobradamente el criterio habitual en identificación de plantas, $R^2 > 0,9$.

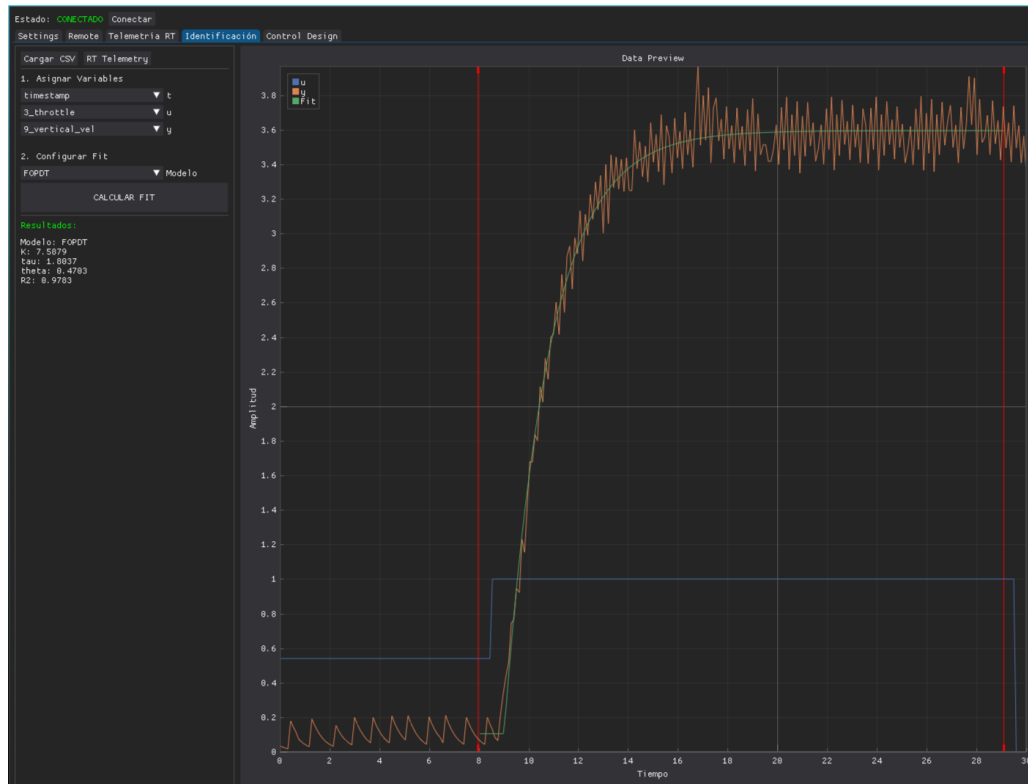


Figura 28. Pestaña *Identificación* con el ajuste FOPDT superpuesto a los datos medidos de *throttle* → velocidad vertical.

9.3.4. Diseño del controlador y validación

Con el modelo identificado se calculan los parámetros SIMC en la pestaña *Control Design* con $\text{factor_agresividad} = 4$ (Figura 29), obteniendo:

Tabla 19. Parámetros PID sintonizados por SIMC para el lazo de velocidad vertical.

K_p	K_i	K_d
0,1007	0,0558	0,0000

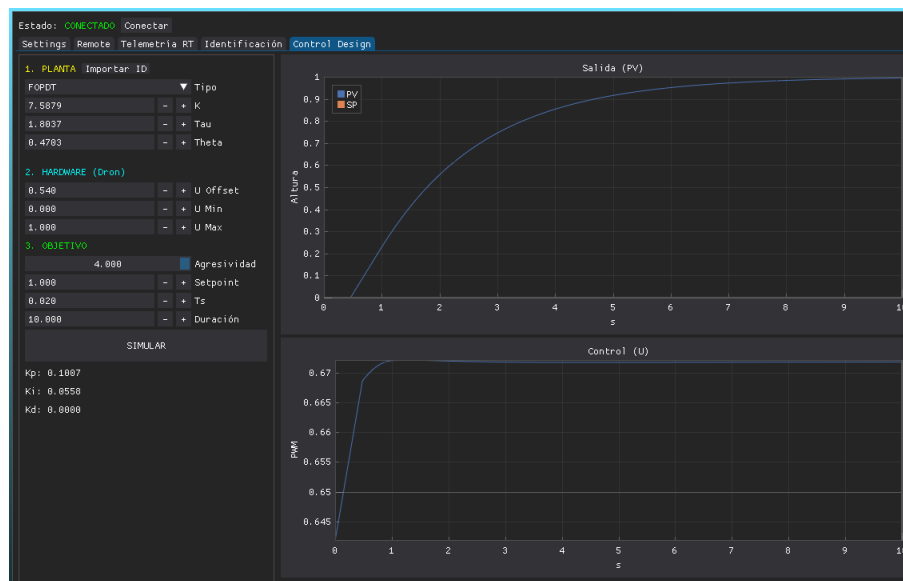


Figura 29. Pestaña *Control Design*: simulación de la variable de proceso (arriba) y de la señal de control (abajo) antes de cargar los parámetros al AFC.

Tras cargar los parámetros al AFC en caliente, se ejecuta un ensayo de validación tipo *doblet* (Figura 30). La velocidad vertical sigue la referencia ± 1 m/s sin *overshoot* apreciable, validando el diseño SIMC.



Figura 30. Ensayo de validación (*doblet* ± 1 m/s) con el PID sintonizado ya cargado en el AFC.

9.3.5. Sintonización del resto de lazos

El procedimiento anterior —identificación, diseño SIMC y validación— se ha repetido para el resto de lazos de control del sistema, incluidos los dos lazos de posición usados en la navegación por ruta (§6.5.3.2). La Tabla 20 resume el modelo identificado y los parámetros PID finales de cada lazo.

Tabla 20. Sintonización de todos los lazos de control del sistema.

Lazo (ID)	Modelo identificado	K_p	K_i
vertical_vel (24)	FOPDT: $K = 18,49$, $\tau = 3,06$ s, $\theta = 0,67$ s	0,122	0,040
position_vertical (42)	IPDT: $K = 1,011$, $\theta = 0,081$ s	0,397	0,08
frontal_vel (22)	IPDT: $K = 1,019$, $\theta = 0,28$ s	0,695	0,123
lateral_vel (20)	IPDT: $K = 0,775$, $\theta = 0,49$ s	0,866	0,145
angular_vel (26)	Oscilación natural sostenida, no describible por SIMC — ganancias ajustadas a mano	0,075	0,05
horizontal_pos (40)	IPDT: $K = 1,025$, $\theta = 1,98$ s, $R^2 = 0,994$	0,245	0,0154
angular_pos (44)	IPDT: $K = 0,391$, $\theta = 2,56$ s, $R^2 = 0,965$	0,499	0,0244

Todos los parámetros se calculan con las reglas SIMC de §8.8.1 y se cargan en caliente en el AFC, sin recompilar ni reflashear el firmware (§8.8.3).

9.4. Ensayo de navegación por ruta

Con todos los lazos sintonizados, se sube una ruta de 6 waypoints (hexágono de radio 3 m, altitud oscilando entre 3 y 4 m, rumbo recorriendo los cuatro cuadrantes) y el AFC la recorre de forma autónoma. Se realizan $n = 10$ repeticiones con el mismo controlador, sin variar ningún parámetro entre corridas. Las 10 corridas completan el circuito íntegro.

La Tabla 21 resume el error de seguimiento (*cross-track error*, XTE) y el sobrepico en el primer waypoint.

Tabla 21. Resultados del ensayo de navegación por ruta ($n = 10$).

Métrica	Media $\pm$ desv.
xte_{rms} (m)	$0,599 \pm 0,222$
xte_{max} (m)	$1,406 \pm 0,588$
Sobrepico en WP0 (m)	$0,870 \pm 0,805$

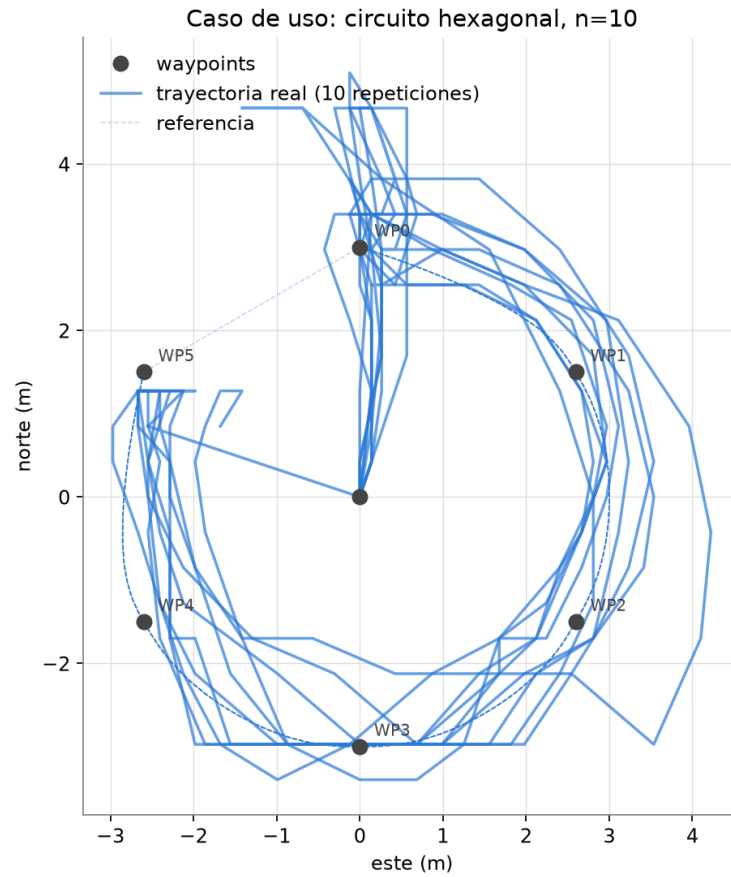


Figura 31. Las 10 trayectorias reales superpuestas a la referencia, sobre el circuito hexagonal de 6 waypoints.

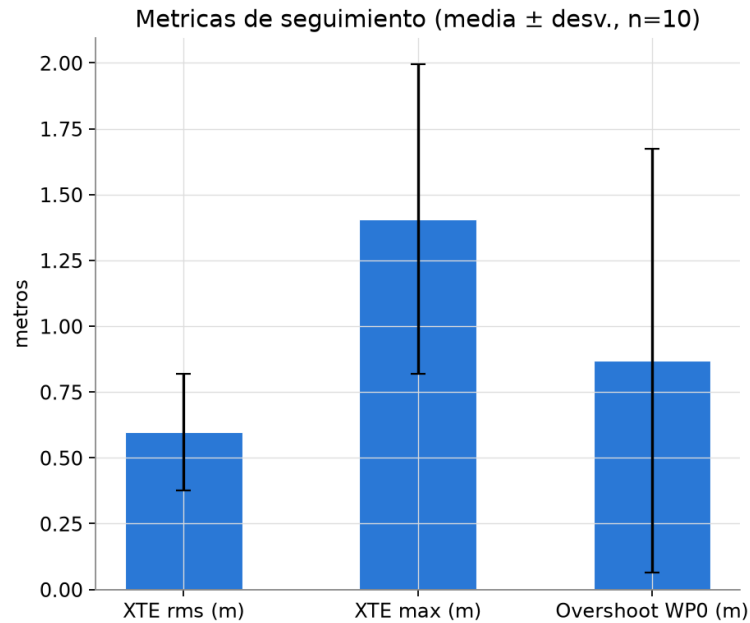


Figura 32. Métricas de seguimiento del ensayo, media $\pm$ desviación sobre las 10 corridas.

Dos corridas superan la media más una desviación típica en xte_{max} — variabilidad normal de simulación entre corridas del mismo controlador y protocolo, no un patrón sistemático distinto. La Figura 33 muestra la trayectoria media de las 8 corridas restantes, con las corridas individuales de fondo para visualizar la dispersión real.

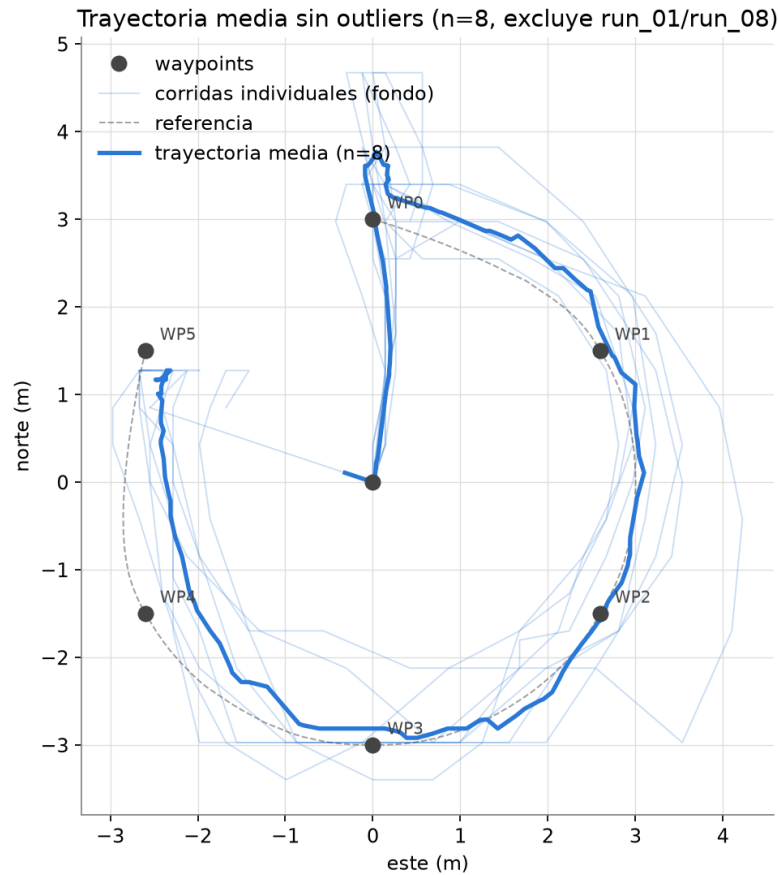


Figura 33. Trayectoria media de las 8 corridas dentro de rango, con las corridas individuales de fondo.

La plataforma completa — arquitectura en capas, protocolo MSP, GCS desacoplada y control PID en cascada — sostiene así una misión autónoma tridimensional exigente (giros y cambios de altitud) de forma repetible.

Capítulo 10

Conclusiones y Trabajo Futuro

10.1. Cumplimiento de los objetivos

Los objetivos fijados en el Capítulo 3 guiaron el diseño de la arquitectura (Capítulo 6), el entorno de simulación (Capítulo 7) y la GCS (Capítulo 8). Los ensayos del Capítulo 9 aportan evidencia concreta de que el sistema construido los cumple.

10.1.1. Software lo más flexible posible

La abstracción OSAL/HAL (§3.2.1) no es solo un diseño sobre el papel: el mismo código de las Capas 3–5 compila y se ejecuta sin cambios tanto en Linux (SITL) como en la placa AM-036 (ESP32/FreeRTOS), seleccionando la implementación de Capa 1 en tiempo de compilación (TransportFactory/OSALFactory, Figura 11, Tabla 4). Los ensayos de comunicaciones (§9.1.1) y de HIL (§9.2.1, §9.2.2) validan esta portabilidad sobre los dos entornos, sin que la lógica de aplicación distinga entre ellos.

La incorporación de navegación autónoma por ruta de waypoints (§6.5.3, §9.4) confirma esta flexibilidad de forma práctica: se añade una capacidad no prevista al fijar los objetivos, reutilizando únicamente los mecanismos de extensión que ya describen los capítulos anteriores (blackboard, sistema de comandos autodescriptivo, GCS desacoplada), sin tocar sus cimientos.

10.1.2. Control del dron sin mando manual

La GCS (Capítulo 8) sustituye por completo al mando RC y se ha usado para ejecutar todos los ensayos de esta memoria. El protocolo MSP/IBus que la conecta al AFC se ha verificado a nivel de bytes, no solo de comportamiento funcional (§9.1.1). La independencia respecto a la red subyacente queda demostrada por el mismo protocolo funcionando sin cambios de código sobre *loopback* en SITL y sobre WiFi real en HIL (Capítulo 9).

10.1.3. Entorno de desarrollo ágil

El SITL ha sido el entorno de trabajo principal durante todo el desarrollo, y el HIL sobre la placa AM-036 se ha usado para la validación final de recursos: 73 % de flash libre y un uso de CPU de un solo dígito por tarea (§9.2.2.1, §9.2.1). La sintonización *on-the-fly* se ha demostrado de principio a fin en el ciclo de identificación (§9.3): parámetros SIMC calculados en la GCS y cargados en caliente en el AFC sin recompilar ni reflashear, validados con un ensayo de *doblet* (Figura 30).

10.2. Conclusiones

El sistema desarrollado demuestra que un microcontrolador de bajo coste puede asumir el papel de *companion computer* que tradicionalmente exige un ordenador embebido de gama superior (Raspberry Pi o similar), sin renunciar a telemetría, identificación de planta y control remoto. El *footprint* estático y el consumo de CPU en *runtime* (§9.2.2.1, §9.2.1) confirman que la lógica de vuelo ocupa una fracción mínima de los recursos disponibles, dejando margen de crecimiento sin cambiar de hardware.

La arquitectura en capas con inversión de dependencias (Capítulo 6) ha permitido desarrollar y depurar la mayor parte del sistema en SITL, reservando el hardware físico para la validación final — un flujo de trabajo que reduce el riesgo y el tiempo de iteración frente a programar directamente sobre la placa. El protocolo de comunicaciones diseñado para este trabajo, verificado a nivel de bytes y puesto a prueba en dos entornos de red distintos, es la base que hace posible ese flujo.

10.3. Trabajo futuro

1. **Robustez de red.** Ensayo formal de reconexión tras caída de la conexión TCP de la GCS, comportamiento del sistema cuando el canal IBus deja de recibir referencias, y reinicio del entorno SITL con el AFC en ejecución.
2. **Latencia y jitter SITL vs. HIL.** Comparación cuantitativa de la frescura de los datos del *blackboard* entre el transporte de *loopback* y el transporte WiFi real.
3. **Migración del módulo celular a LTE** (SIM7600 o similar), para operación fuera de cobertura WiFi.
4. **Validación en hardware real.** Repetir la navegación autónoma por ruta de waypoints (§9.4) sobre la placa AM-036 en HIL, para confirmar en dron real el comportamiento ya validado en SITL.

Capítulo 11

Alineación con los Objetivos de Desarrollo Sostenible

Los Objetivos de Desarrollo Sostenible (ODS) de la Agenda 2030 de Naciones Unidas proporcionan un marco de referencia para evaluar el impacto social, económico y medioambiental de los proyectos de ingeniería. A continuación se describe cómo este TFM contribuye a los ODS más directamente relacionados.

11.1. ODS 9 — Industria, innovación e infraestructura

El trabajo propone una arquitectura de bajo coste que democratiza el acceso a UAVs autónomos para investigación y aplicaciones industriales. La reducción del coste y el peso del sistema respecto a las soluciones convencionales con *companion computers* de altas prestaciones facilita que equipos con recursos limitados puedan experimentar con tecnología aeronáutica avanzada.

Asimismo, la integración de conectividad celular (IoT) y el entorno de simulación completo (SITL/HIL) promueven prácticas de desarrollo ágil y seguro, reduciendo la necesidad de recursos físicos y el riesgo de pérdida de material durante el desarrollo.

11.2. ODS 4 — Educación de calidad

El código, la arquitectura y los entornos de simulación desarrollados en este TFM están concebidos como herramientas de aprendizaje reutilizables. El entorno SITL permite a estudiantes e investigadores experimentar con algoritmos de control y comunicaciones aeronáuticas sin disponer de hardware real, reduciendo así la barrera de entrada al campo.

11.3. ODS 11 — Ciudades y comunidades sostenibles

Los UAVs de bajo coste con conectividad global tienen aplicaciones directas en monitorización medioambiental, inspección de infraestructuras y operaciones de emergencia en áreas de difícil acceso. La arquitectura propuesta habilita estas aplicaciones a un coste accesible para municipios y organismos con presupuesto limitado.

11.4. ODS 13 — Acción por el clima

La conectividad celular y las capacidades de telemetría continua del sistema desarrollado lo convierten en una plataforma idónea para misiones de monitorización medioambiental a largo alcance: medición de calidad del aire, seguimiento de focos de incendio forestal o vigilancia de zonas de difícil acceso sin exponer a personal en terreno.

El bajo coste del sistema —tanto en hardware como en infraestructura de operación, al apoyarse en redes celulares existentes— permite desplegar flotas de unidades para cubrir áreas extensas, algo inviable con las plataformas convencionales de mayor precio. Esta accesibilidad facilita que organismos públicos y ONG con recursos limitados incorporen UAVs a sus herramientas de respuesta ante emergencias climáticas y catástrofes naturales.

Bibliografía

- [1] E. Ebeid, M. Skriver, and J. Jin, “A survey of open-source UAV autopilots,” *Electronics*, vol. 13, no. 23, p. 4785, 2024.
- [2] E. Ebeid, M. Skriver, K. H. Terkildsen, K. Jensen, and U. P. Schultz, “A survey of open-source UAV flight controllers and flight simulators,” *Microprocessors and Microsystems*, vol. 61, pp. 11–20, 2018.
- [3] L. Y. Sørensen, L. T. Jacobsen, and J. P. Hansen, “Low cost and flexible UAV deployment of sensors,” *Sensors*, vol. 17, no. 1, p. 154, 2017.
- [4] J. Muñoz-Morera *et al.*, “An open-source hardware/software architecture for UAV autonomy.” arXiv:2407.06320, 2024.
- [5] Betaflight Contributors, “Betaflight — open source flight controller firmware.” <https://github.com/betaflight/betaflight>, 2025.
- [6] L. Beaudoin, L. Avanthey, and C. Villard, “Porting ArduPilot to ESP32: Towards a universal open-source architecture for agile and easily replicable multi-domains mapping robots,” in *International Archives of the Photogrammetry, Remote Sensing and Spatial Information Sciences*, vol. XLIII-B2-2020, pp. 933–938, Copernicus Publications, 2020.
- [7] M. Panunzio and T. Vardanega, “An architectural approach with separation of concerns to address extra-functional requirements in the development of embedded real-time software systems,” *Journal of Systems Architecture*, vol. 60, no. 7, pp. 578–590, 2014.
- [8] E. Ebeid, M. Skriver, and J. Jin, “Embedded computation architectures for autonomy in unmanned aircraft systems (UAS),” *Sensors*, vol. 21, no. 4, p. 1115, 2021.
- [9] S. Kounev *et al.*, “Inversion of control container for safety-critical embedded systems,” in *Proceedings of the 19th European Conference on Pattern Languages of Programs (EuroPLoP)*, ACM, 2015.
- [10] F. Ahmed and M. Jenihhin, “A survey on UAV computing platforms: A hardware reliability perspective,” *Sensors*, vol. 22, no. 16, p. 6286, 2022.
- [11] R. Bolimera, S. Y. Pasha, A. Thomas, S. S. Kumar, and V. Mathsyagiri, “Implementation of drone using ESP-32,” *International Journal of Data Science and IoT Management System*, vol. 5, no. 1, pp. 19–21, 2026.

-
- [12] G. Yang *et al.*, “A telecom perspective on the internet of drones: From LTE -advanced to 5G.” arXiv:1803.11048, 2018.
- [13] X. Lin *et al.*, “The sky is not the limit: LTE for unmanned aerial vehicles,” *IEEE Communications Magazine*, vol. 56, no. 4, pp. 204–210, 2018.
- [14] R. Amorim *et al.*, “Reliable aerial mobile communications with RSRP & RSRQ prediction models for the internet of drones,” *Sensors*, vol. 22, no. 14, 2022. PMC9331756.
- [15] H. X. Nguyen *et al.*, “Mobile network performance and technical feasibility of LTE -powered unmanned aerial vehicle,” *Sensors*, vol. 21, no. 8, p. 2848, 2021. PMC8073316.
- [16] A. Colpaert, E. Vinogradov, and S. Pollin, “Aerial coverage analysis of cellular systems at LTE and mmWave frequencies using 3D city models,” *Sensors*, vol. 18, no. 12, p. 4311, 2018. PMC6308414.
- [17] Anonymous, “Implementation of cellular-based drone module using cloud services,” *Journal of Electrical, Electronic, Information, and Communication Technology*, 2023.
- [18] ArduPilot Dev Team, “Multiwii serial protocol (MSP) — copter documentation.” <https://ardupilot.org/copter/docs/common-msp-overview.html>, 2024. Última consulta: Mayo 2025.
- [19] Grokipedia, “MultiWii serial protocol.” <https://grokipedia.com/multiwii-serial-protocol>, 2026. Última consulta: Mayo 2025.
- [20] C. Rauch, “msp: Implementation of the MultiWii serial protocol.” GitHub. <https://github.com/christianrauch/msp>, 2024. Última consulta: Mayo 2025.
- [21] O. Liang, “FPV protocols explained (CRSF, SBUS, DSHOT, ACCST, PPM, PWM and more).” <https://oscarliang.com/rc-protocols>, 2024. Última consulta: Mayo 2025.
- [22] ivcan.com, “UART MSP protocol in drone radio link.” <https://ivcan.com>, 2025. Última consulta: Mayo 2025.
- [23] AllPCB / ElectroHub, “Embedded system software and hardware frameworks.” <https://allpcb.com>, 2026. Última consulta: Mayo 2025.
- [24] NASA, “Operating system abstraction layer (GSC-18370-1).” <https://software.nasa.gov>, 2019. Core Flight System (cFS). Última consulta: Mayo 2025.
- [25] T. Koç and G. Ünlü, “A modular software architecture for UAVs,” in *Proceedings of the 38th Annual Conference of the IEEE Industrial Electronics Society (IECON)*, IEEE, 2012.
- [26] Wikipedia, “Dependency inversion principle.” https://en.wikipedia.org/wiki/Dependency_inversion_principle, 2025. Basado en R. C. Martin, 1994–2003. Última consulta: Mayo 2025.
-

-
- [27] Wikipedia, “Blackboard (design pattern).” [https://en.wikipedia.org/wiki/Blackboard_\(design_pattern\)](https://en.wikipedia.org/wiki/Blackboard_(design_pattern)), 2025. Última consulta: Mayo 2025.
 - [28] D. Weyns *et al.*, “Blackboard architecture for detecting and notifying failures for component-based unmanned systems,” *Journal of Intelligent & Robotic Systems*, vol. 91, no. 1, pp. 1–18, 2018.
 - [29] G. Foxwell, “Blackboard pattern.” Medium / Coinmonks. <https://medium.com/coinmonks/blackboard-pattern>, 2018. Última consulta: Mayo 2025.
 - [30] USPTO, “Blackboard-centric layered software architecture for an embedded aircraft fuel gauging subsystem.” US Patent 6,256,679, 2001.
 - [31] R. Antonini *et al.*, “Vision-based software-in-the-loop-simulation for UAVs using Gazebo and PX4,” in *IEEE International Conference on Automation Science and Engineering (CASE)*, IEEE, 2019.
 - [32] F. L. Razón *et al.*, “Software- and hardware-in-the-loop verification of flight dynamics model and flight control simulation of a fixed-wing UAV,” *Journal of Intelligent & Robotic Systems*, 2015.
 - [33] T. A. Nguyen *et al.*, “Development of hardware-in-the-loop simulation based on Gazebo and Pixhawk for UAVs,” *International Journal of Aeronautical and Space Sciences*, vol. 19, no. 2, pp. 529–540, 2018.
 - [34] S. Skogestad, “Simple analytic rules for model reduction and PID controller tuning,” *Journal of Process Control*, vol. 13, no. 4, pp. 291–309, 2003.